



Data Warehouse and Business Intelligence

Tugas Kelompok: Multidimensional Modeling untuk Sistem DWBI Healthcare

Engelbertus Rande, Gilbert Fandiliam Mooy, Sinta Siti Nuriah, Vicky Mahfudy

**Dosen Pengampu
Khabib Mustofa**

May 27, 2026



Daftar Isi

I	Pendahuluan	1
1	Pendahuluan	2
1.1	Deskripsi Domain	2
1.2	Latar Belakang	3
1.3	Tujuan	4
II	Analisis Kebutuhan Bisnis	5
2	Analisis Kebutuhan Bisnis	6
2.1	Business Questions	6
2.2	Stakeholder	8
2.3	Proses Bisnis Utama	8
III	Perancangan Dimensional Modeling	10
3	Perancangan Dimensional Modeling	11
3.1	Penentuan Grain	11
3.2	Rancangan Fact Table	12
3.3	Rancangan Dimension Table	12
3.4	Star Schema Diagram	13
3.5	Perancangan Arsitektur Staging dan Alur ETL	14
IV	Synthetic Data Generation	17
4	Synthetic Data Generation	18
4.1	Strategi Generator Data Sintetik	18



4.2	Kode Generator Utama	18
4.3	Contoh Data Mentah (Output source_kunjungan.csv)	21
4.4	Skema Sumber (Data Dictionary)	21
V	Implementasi Orkestrasi Apache Airflow	22
5	Implementasi Orkestrasi Apache Airflow	23
5.1	Arsitektur DAG (Directed Acyclic Graph)	23
5.2	Topologi dan Penjelasan <i>Task Dependencies</i>	23
5.3	Panduan Dokumentasi Antarmuka Airflow UI	26
VI	Implementasi dbt (Data Build Tool)	29
6	Implementasi dbt (Data Build Tool)	30
6.1	Struktur Proyek dbt	30
6.2	Penjelasan Tiap Lapisan (<i>Layering Architecture</i>)	31
6.3	Hasil dbt Docs dan Lineage Graph	33
VII	Pembangunan Data Mart	34
7	Pembangunan Data Mart	35
7.1	Skema Final Data Mart	35
7.2	Dokumentasi Tabel dan Kolom (Kamus Data)	36
7.3	Koneksi ke Business Intelligence (BI) Tools	38
7.4	Lapisan Semantik (<i>Semantic Layer</i>)	46
VIII	Verifikasi dan Validasi Data Mart	48
8	Verifikasi dan Validasi	49
8.1	Query Analitik terhadap Data Mart	49
8.1.1	Query 1: Total Pendapatan Berdasarkan Tipe Fasilitas (BQ-01)	49
8.1.2	Query 2: Kinerja Dokter per Spesialisasi (BQ-02)	50
8.1.3	Query 3: Distribusi Metode Pembayaran terhadap Total Biaya (BQ-03)	51
8.1.4	Query 4: Rata-rata Length of Stay dan Pengaruhnya terhadap Biaya (BQ-04)	52



8.1.5	Query 5: Tren Bulanan Volume Kunjungan Pasien (BQ-05)	54
8.1.6	Query 6: Proporsi Status Keluar Pasien Rawat Inap (BQ-06)	55
8.1.7	Query 7: Breakdown Komponen Biaya Pelayanan (BQ-07)	56
8.1.8	Query 8: Distribusi Kunjungan Berdasarkan Kota Asal Pasien (BQ-08)	58
8.1.9	Query 9: Profil Demografi Kunjungan Berdasarkan Jenis Kelamin dan Golongan Darah (BQ-09)	60
8.1.10	Query 10: Perbandingan Rata-Rata Biaya Obat IGD vs. Rawat Jalan (BQ-10)	61
8.1.11	Query 11: Peringkat Lima Dokter Teraktif (BQ-11)	62
8.1.12	Query 12: Rata-Rata Biaya per Transaksi Kunjungan — <i>Average Ticket Size</i> (BQ-12)	63
8.2	Hasil dbt Test	66
8.2.1	Konfigurasi Pengujian (schema.yml)	67
8.2.2	Output Eksekusi dbt Test	69
8.3	Pemeriksaan Kualitas Data (<i>Data Quality Check</i>)	76
8.3.1	Metodologi Pemeriksaan	76
8.3.2	Dimensi Pemeriksaan	76
8.3.3	Kueri Validasi PostgreSQL	76
8.3.4	Ringkasan Hasil Pemeriksaan	80
IX	Kesimpulan dan Rekomendasi	81
9	Kesimpulan dan Refleksi	82
9.1	Ringkasan Pencapaian Proyek	82
9.2	Tantangan yang Dihadapi	82
9.3	Keputusan Desain Utama	83
9.4	Kesiapan Data Mart untuk Tugas BI (Visualisasi Data)	83

Part I

Pendahuluan



BAB 1

Pendahuluan

Anggota kelompok

1. Engelbertus Rande (NIM 25/573149/PPA/07211)
2. Gilbert Fandiliam Mooy (NIM 25/574767/PPA/07259)
3. Sinta Siti Nuriah (NIM 25/575177/PPA/07274)
4. Vicky Mahfudy (NIM 25/573019/PPA/07207)

1.1 DESKRIPSI DOMAIN

Domain yang menjadi fokus penelitian ini adalah bidang kesehatan (*healthcare*), khususnya pengelolaan data operasional rumah sakit yang mencakup registrasi pasien, penugasan tenaga medis, serta transaksi kunjungan ke fasilitas kesehatan. Domain ini bersifat multidimensional karena melibatkan berbagai aspek yang saling berkaitan, yaitu profil demografis pasien, klasifikasi keahlian dokter, serta rincian pelayanan medis baik di unit gawat darurat (IGD), rawat jalan, maupun rawat inap.

Populasi dalam dataset ini adalah populasi umum pasien yang melakukan kunjungan ke rumah sakit. Cakupan operasional dataset berfokus pada dinamika pelayanan dan administrasi penagihan (*billing*), di mana setiap kunjungan akan menghasilkan beban biaya yang berbeda bergantung pada tipe fasilitas, durasi perawatan, serta tindakan medis yang diberikan. Tiga aspek utama yang menjadi pilar analisis dalam domain ini adalah:

- **Kunjungan Pasien** — mencakup frekuensi penggunaan tipe fasilitas (rawat inap, rawat jalan, IGD), lama perawatan (*Length of Stay*), status pemulihan saat keluar, dan metode pembayaran.
- **Tenaga Medis** — mencakup data spesialisasi dokter, kepemilikan izin praktek (STR), dan struktur biaya konsultasi medis.
- **Rincian Biaya** — mencakup rincian penagihan yang terdiri dari biaya konsultasi, obat, tindakan, dan sewa kamar rawat inap yang kemudian diakumulasi menjadi total biaya transaksi.

Gambaran Umum Dataset

Berikut adalah karakteristik lengkap dataset yang digunakan dalam penelitian ini:



Tabel 1.1: Karakteristik Dataset Operasional Rumah Sakit

Aspek	Keterangan
Nama Dataset	Healthcare Operations Synthetic Dataset
Sumber Data 1	source_pasien.csv — Data profil demografis pasien (2.500 baris, 7 kolom)
Sumber Data 2	source_dokter.csv — Data master dokter (80 baris, 5 kolom)
Sumber Data 3	source_kunjungan.csv — Data transaksi layanan medis (12.000 baris, 13 kolom)
Total Record Kunjungan	12.000 baris
Rentang Waktu Kunjungan	1 Januari 2025 – Awal 2026
Tipe Fasilitas	IGD, Rawat Jalan, Rawat Inap
Metode Pembayaran	BPJS, Asuransi Swasta, Mandiri
Format Output	CSV (<i>Comma Separated Values</i>)

Dataset final didistribusikan ke dalam tiga entitas CSV independen yang saling berelasi. Pengelompokan atribut dan informasi kolom ditunjukkan pada tabel berikut.

Tabel 1.2: Pengelompokan Kolom pada 3 Tabel Sumber

Kelompok Kolom	Jml.	Contoh Kolom
Identitas & Kunci	3	id_pasien, id_dokter, id_kunjungan
Demografi Pasien	6	nama_pasien, jenis_kelamin, tanggal_lahir, alamat, kota, golongan_darah
Informasi Medis Dokter	4	nama_dokter, spesialisasi, biaya_konsultasi, nomor_izin_praktek
Utilisasi Layanan	4	tanggal_kunjungan, tipe_fasilitas, lama_rawat_hari, status_keluar
Finansial Medis	5	metode_pembayaran, biaya_obat, biaya_tindakan, biaya_kamar, total_biaya

1.2 LATAR BELAKANG

Manajemen operasional rumah sakit merupakan tantangan analitik terbesar di sistem pelayanan kesehatan. Transaksi operasional seringkali menghasilkan volume data klinis dan finansial yang sangat besar dari berbagai fasilitas (IGD, Poliklinik, dan Bangsal Inap). Fragmentasi data operasional di berbagai sistem pendaftaran, penjadwalan dokter, dan unit kasir menjadi hambatan utama dalam menghasilkan analitik keuangan dan utilitas fasilitas yang komprehensif.

Sistem Data Warehouse dan Business Intelligence (DWBI) hadir sebagai solusi untuk mengintegrasikan, menyimpan, dan menganalisis data transaksional dari berbagai sumber tersebut secara terstruktur dalam suatu model dimensional, sehingga menghasilkan wawasan operasional yang dapat ditindaklanjuti oleh pemangku kepentingan.

Penggunaan data sintesis dalam penelitian dan pengembangan DWBI ini dipilih untuk menghindari kendala regulasi perlindungan data privasi yang membatasi akses terhadap rekam medis nyata, namun tetap mempertahankan logika bisnis klinis operasional melalui teknik *synthetic data generation* berbasis aturan kondisional menggunakan bahasa pemrograman Python. Berikut merupakan identifikasi permasalahan:

1. Data pendaftaran pasien, jadwal dokter, dan *billing* layanan tersebar di berbagai sumber yang belum



terintegrasi, sehingga menyulitkan analisis performa rumah sakit secara holistik.

2. Belum tersedianya infrastruktur Data Warehouse yang mampu mengonsolidasikan ketiga aspek tersebut ke dalam satu skema *Star Schema* yang efisien untuk pelaporan bisnis.
3. Kurangnya kapabilitas untuk melacak rincian komponen biaya medis (obat, tindakan, kamar, jasa dokter) berdasarkan fasilitas dan spesialisasi secara *real-time*.
4. Keterbatasan akses terhadap data riil operasional akibat regulasi privasi memicu kebutuhan pembuatan *pipeline* generator data sintetik berbasis aturan medis (*rule-based*).

1.3 TUJUAN

Tujuan Umum

Membangun sistem Data Warehouse dan Business Intelligence (DWBI) operasional rumah sakit yang mengintegrasikan profil pasien, data master dokter, dan log transaksi kunjungan dari sumber terpisah ke dalam suatu pemodelan dimensional untuk mendukung pengambilan keputusan klinis dan finansial.

Tujuan Khusus

1. Mengintegrasikan data dari tiga sumber heterogen (`source_pasien.csv`, `source_dokter.csv`, dan `source_kunjungan.csv`) menjadi satu model data terpadu (12.000 transaksi).
2. Menganalisis pola utilisasi fasilitas layanan kesehatan (rawat inap, rawat jalan, IGD) dan mengevaluasi efisiensi operasional berbasis indikator durasi *Length of Stay* (LOS).
3. Menganalisis distribusi pendapatan operasional rumah sakit yang dirincikan berdasarkan komponen biaya, spesialisasi tenaga medis, dan metode pembayaran klaim.
4. Membangun skema dimensional (*Star Schema*) yang terfokus pada kemudahan ekstraksi pelaporan OLAP tingkat lanjut.
5. Menyediakan dataset sintetis berkualitas tinggi sebagai infrastruktur data untuk pengembangan, pengujian, dan demonstrasi sistem DWBI tanpa pelanggaran regulasi privasi pasien.

Part II

Analisis Kebutuhan Bisnis



BAB 2

Analisis Kebutuhan Bisnis

Anggota kelompok

1. Engelbertus Rande (NIM 25/573149/PPA/07211)
2. Gilbert Fandiliam Mooy (NIM 25/574767/PPA/07259)
3. Sinta Siti Nuriah (NIM 25/575177/PPA/07274)
4. Vicky Mahfudy (NIM 25/573019/PPA/07207)

2.1 BUSINESS QUESTIONS

Business questions merupakan pertanyaan-pertanyaan bisnis kritis yang diformulasikan berdasarkan kebutuhan analitik *stakeholder* manajemen rumah sakit serta karakteristik aktual dari struktur dataset operasional. Dua belas *business questions* berikut ditetapkan sebagai panduan utama dalam perancangan sistem DWBI, yang mencakup enam domain analitik utama: utilisasi fasilitas, kinerja tenaga medis, analisis finansial/penagihan, efisiensi klinis, metode pembayaran, dan demografi pasien.



Tabel 2.1: Business Questions Sistem DWBI Operasional Rumah Sakit

No	Business Question	Kategori	KPI / Metrik
1	Berapa total pendapatan rumah sakit dan bagaimana distribusinya berdasarkan tipe fasilitas (IGD, Rawat Jalan, Rawat Inap)?	Finansial	total_biaya, tipe_fasilitas
2	Spesialisasi dokter manakah yang menghasilkan volume transaksi kunjungan tertinggi dan total biaya penagihan terbesar?	Kinerja Dokter	Count(id_kunjungan), Sum(total_biaya) per spesialisasi
3	Bagaimana distribusi penggunaan metode pembayaran (BPJS, Asuransi Swasta, Mandiri) terhadap total biaya pelayanan?	Finansial & Payer	Persentase akumulasi total_biaya per metode_pembayaran
4	Berapa rata-rata lama rawat inap (<i>Length of Stay</i>) pasien dan bagaimana pengaruhnya terhadap biaya kamar serta total biaya transaksi?	Efisiensi Klinis	Rata-rata lama_rawat_hari, biaya_kamar, total_biaya
5	Bagaimana tren bulanan jumlah kunjungan pasien di rumah sakit sepanjang periode tahun 2025 hingga awal 2026?	Tren Kunjungan	Count(id_kunjungan) berdasarkan tren tanggal_kunjungan
6	Bagaimana proporsi status keluar pasien (Sembuh, Dirujuk, Pulang Paksa, Meninggal) khusus pada tipe fasilitas Rawat Inap?	Mutu Pelayanan	Rasio status_keluar where tipe_fasilitas = 'Rawat Inap'
7	Bagaimana rincian kontribusi masing-masing komponen biaya (konsultasi, obat, tindakan, kamar) terhadap struktur total penagihan?	Analisis Biaya	Breakdown rata-rata biaya_konsultasi, biaya_obat, biaya_tindakan, biaya_kamar
8	Kota asal pasien manakah yang memiliki beban pemanfaatan layanan dan frekuensi kunjungan tertinggi di rumah sakit?	Geografi	Count(id_kunjungan), Sum(total_biaya) per kota
9	Bagaimana profil distribusi kunjungan pasien jika dikelompokkan berdasarkan variabel jenis kelamin dan golongan darah?	Demografi	<i>Cross-tabulation</i> Count(id_kunjungan) per jenis_kelamin dan golongan_darah
10	Berapa rata-rata biaya obat yang dikeluarkan oleh pasien pada fasilitas IGD dibandingkan dengan fasilitas Rawat Jalan?	Farmasi & Biaya	Rata-rata biaya_obat per tipe_fasilitas (IGD vs Rawat Jalan)
11	Siapakah 5 dokter teratas (<i>Top 5 Doctors</i>) yang memiliki kontribusi pelayanan transaksi kunjungan paling aktif di rumah sakit?	Kinerja Dokter	Ranking 5 teratas nama_dokter berdasarkan Count(id_kunjungan)
12	Berapa rata-rata pengeluaran total biaya per satu kali transaksi kunjungan pasien (<i>average ticket size</i>) di rumah sakit?	Finansial	Rata-rata total_biaya dari seluruh baris transaksi

Business questions di atas dirancang agar dapat dijawab secara multidimensional melalui pembuatan tabel fakta transaksi (Fact_Kunjungan) yang akan terhubung dengan tabel-tabel dimensi pendukung seperti Dimensi Pasien, Dimensi Dokter, Dimensi Waktu, dan Dimensi Fasilitas pada arsitektur pemodelan data.



2.2 STAKEHOLDER

Identifikasi *stakeholder* diperlukan untuk memastikan sistem DWBI yang dibangun mampu menyajikan informasi yang relevan, akurat, dan sesuai dengan wewenang masing-masing pengguna di rumah sakit. Berdasarkan arsitektur data operasional yang tersedia, terdapat tujuh kelompok *stakeholder* utama:

Tabel 2.2: Daftar Stakeholder dan Kebutuhan Informasi

Stakeholder	Peran	Kebutuhan Informasi
Direktur / Manajemen RS	Decision Maker (Eksekutif)	Laporan KPI makro operasional, tren total pendapatan bulanan, efisiensi utilitas tempat tidur, dan evaluasi performa bisnis makro.
Kepala Bidang Pelayanan Medis	Clinical Manager	Monitoring beban kerja dokter berdasarkan spesialisasi, evaluasi hasil klinis pasien lewat rasio status keluar (Sembuh/Meninggal).
Manajer Keuangan / Akuntan	Financial Analyst	Analisis profitabilitas komponen penagihan (obat, tindakan, kamar), pemantauan perputaran arus kas penagihan rumah sakit.
Manajer Operasional & Fasilitas	Operational Analyst	Pengawasan utilitas tipe fasilitas (IGD, Rawat Jalan, Rawat Inap) dan analisis efisiensi operasional harian berdasarkan <i>Length of Stay</i> (LOS).
Unit Penjaminan & Klaim (Casemix)	Payer Analyst	Analisis segmentasi metode pembayaran (klaim BPJS, klaim Asuransi Swasta, dan Mandiri) untuk optimalisasi proses verifikasi penagihan.
Kepala Instalasi Farmasi RS	Pharmacy Supervisor	Evaluasi biaya obat yang terserap dalam transaksi pelayanan pasien di berbagai unit fasilitas (IGD dan Rawat Jalan).
Tim Data Engineer / IT RS	Sistem Administrator	Pemantauan kualitas data transaksi, optimasi performa <i>query</i> database OLAP, serta pemeliharaan <i>pipeline</i> ETL dari file CSV sumber.

Sistem DWBI ini menjembatani kebutuhan informasi dari tingkat eksekutif yang memerlukan data agregasi tinggi untuk keputusan strategis (seperti total pendapatan atau tren kunjungan), hingga tingkat operasional yang membutuhkan analisis granularitas rendah (seperti rincian biaya per transaksi).

2.3 PROSES BISNIS UTAMA

Enam proses bisnis utama berikut diidentifikasi sebagai rantai nilai operasional (*value chain*) yang menjadi sumber data transaksional dan konteks analitik utama di dalam lingkungan rumah sakit:



Tabel 2.3: Proses Bisnis Utama Domain Operasional Rumah Sakit

No	Proses Bisnis	Deskripsi	Kolom Terkait
1	Pendaftaran & Profil Pasien	Proses pencatatan data demografis, identitas unik, dan karakteristik fisik pasien saat pertama kali terdaftar di rumah sakit.	id_pasien, nama_pasien, jenis_kelamin, tanggal_lahir, alamat, kota, golongan_darah
2	Manajemen Data Master Dokter	Pengelolaan data ketenagakerjaan tenaga medis, legalitas izin praktek, penentuan spesialisasi, dan standarisasi tarif jasa dokter.	id_dokter, nama_dokter, spesialisasi, biaya_konsultasi, nomor_izin_praktek
3	Pelayanan Kunjungan Fasilitas	Proses penerimaan pasien di gerbang pelayanan (IGD, Rawat Jalan, Rawat Inap) pada waktu tertentu untuk dialokasikan ke dokter yang bertugas.	id_kunjungan, tanggal_kunjungan, tipe_fasilitas
4	Perawatan Rawat Inap (Opname)	Proses pemantauan durasi hari menginap pasien di bangsal perawatan serta pencatatan kondisi atau status keluar akhir pasien dari perawatan.	lama_rawat_hari, status_keluar
5	Perhitungan <i>Billing</i> & Biaya Medis	Proses kalkulasi penagihan otomatis yang mengakumulasikan komponen biaya penunjang medis (jasa dokter, konsumsi obat, dan biaya tindakan).	biaya_konsultasi, biaya_obat, biaya_tindakan, biaya_kamar
6	Penyelesaian Pembayaran (Klaim)	Proses pelunasan seluruh tagihan transaksi kunjungan pasien berdasarkan metode penjaminan yang disepati oleh unit kasir.	metode_pembayaran, total_biaya

Keenam proses bisnis di atas merepresentasikan siklus operasional harian rumah sakit yang terintegrasi. Setiap transaksi yang divalidasi oleh sistem akan terekam ke dalam file `source_kunjungan.csv` yang memuat relasi kunci asing ke `source_pasien.csv` dan `source_dokter.csv`. Pemahaman mendalam terhadap rantai proses bisnis ini menjadi fondasi utama dalam menentukan *grain*, *measures*, dan dimensi pendukung dalam perancangan model dimensional (*Star Schema*) pada bab berikutnya.

Part III

Perancangan Dimensional Modeling



BAB 3

Perancangan Dimensional Modeling

Anggota kelompok

1. Engelbertus Rande (NIM 25/573149/PPA/07211)
2. Gilbert Fandilam Mooy (NIM 25/574767/PPA/07259)
3. Sinta Siti Nuriah (NIM 25/575177/PPA/07274)
4. Vicky Mahfudy (NIM 25/573019/PPA/07207)

3.1 PENENTUAN GRAIN

Grain adalah pernyataan tegas mengenai apa yang direpresentasikan oleh satu baris data (*record*) di dalam tabel fakta (*fact table*). Penentuan *grain* yang tepat merupakan langkah paling kritis dalam perancangan model dimensional karena secara langsung akan menentukan *measures* apa saja yang dapat dihitung, dimensi apa saja yang diperlukan, serta tingkat kedalaman analisis yang dapat dieksplorasi.

Tabel 3.1: Definisi Grain Fact Table Operasional

Komponen	Definisi / Nilai
Subjek Ukur	Satu transaksi kunjungan pelayanan medis yang dialami pasien dari satu dokter pada satu tanggal tertentu
Granularitas Waktu	Tingkat harian / stempel waktu berdasarkan tanggal_kunjungan
Granularitas Pasien	Per pasien unik, diidentifikasi oleh kunci id_pasien
Granularitas Dokter	Per dokter unik, diidentifikasi oleh kunci id_dokter
Pernyataan Grain	Satu baris mewakili satu transaksi kunjungan medis seorang pasien ke suatu unit fasilitas yang ditangani oleh dokter tertentu pada tanggal tertentu, lengkap dengan rincian biaya penagihannya.
Implikasi	Satu pasien dapat memiliki banyak baris fakta jika melakukan kunjungan ulang ke rumah sakit pada tanggal atau fasilitas yang berbeda.

Grain pada tingkat transaksi per-kunjungan dipilih karena memberikan fleksibilitas analisis yang maksimal dari level makro (seperti laporan pendapatan total per bulan) hingga ke level mikro (seperti beban biaya obat per individu pasien di fasilitas IGD).



3.2 RANCANGAN FACT TABLE

Jenis Fact Table

Tabel fakta utama dalam sistem DWBI ini diberi nama **Fact_Kunjungan** dan tergolong sebagai **Transaction Fact Table**, di mana setiap baris mewakili satu kejadian (*event*) transaksi yang berdiri sendiri dan bersifat final ketika pasien telah melunasi penagihan dan keluar dari fasilitas rumah sakit.

Terdapat dua jenis *measures* di dalam tabel ini: (1) **Additive Measures** — nilai numerik yang dapat dijumlahkan secara logis ke seluruh dimensi, seperti komponen biaya medis dan nilai tagihan; serta (2) **Semi-Additive Measures** — nilai ukur yang hanya bisa diolah pada dimensi tertentu namun tidak masuk akal jika dijumlahkan pada dimensi waktu, seperti hari lama rawat inap (*length of stay*).

Daftar Measures dan Foreign Keys

Tabel 3.2: Rancangan Fact_Kunjungan (5 FK + 6 Measures)

ID	Nama Kolom	Tipe Measure	Tipe Data	Keterangan
FK1	pasien_key	Foreign Key	VARCHAR	Relasi ke entitas Dim_Pasien
FK2	dokter_key	Foreign Key	VARCHAR	Relasi ke entitas Dim_Dokter
FK3	date_key	Foreign Key	INT	Relasi penanggalan ke Dim_Waktu
FK4	fasilitas_key	Foreign Key	INT	Relasi ke Dim_Fasilitas
FK5	bayar_key	Foreign Key	INT	Relasi ke Dim_Pembayaran
M1	lama_rawat_hari	Semi-Additive	INT	Durasi opname pasien (0 jika selain Rawat Inap)
M2	biaya_konsultasi	Additive	DECIMAL	Beban tarif jasa pemeriksaan oleh dokter
M3	biaya_obat	Additive	DECIMAL	Beban persepan obat farmasi dari kunjungan
M4	biaya_tindakan	Additive	DECIMAL	Beban prosedur diagnostik atau tindakan operatif
M5	biaya_kamar	Additive	DECIMAL	Beban biaya sewa kasur unit inap per kelas
M6	total_biaya	Additive	DECIMAL	Akumulasi tagihan akhir pasien (M2+M3+M4+M5)

3.3 RANCANGAN DIMENSION TABLE

Jenis Slowly Changing Dimension (SCD)

Slowly Changing Dimension (SCD) adalah teknik pengarsipan yang digunakan untuk menangani perubahan atribut deskriptif dimensi seiring berjalannya waktu agar rekam jejak finansial di masa lalu tidak mengalami kerusakan data.



Tabel 3.3: Konfigurasi Tipe SCD pada Sistem DWBI

Tipe SCD	Nama Lain	Karakteristik	Contoh Kasus
Tipe 0	Retain / Fixed	Atribut tetap abadi dan tidak akan pernah diperbarui setelah pertama direkam.	Data kalender hari dan bulan pada Dim_Waktu
Tipe 1	Overwrite	Nilai lama langsung dihapus dan diganti nilai baru tanpa riwayat histori.	Perbaikan salah ketik ejaan nama / <i>update</i> izin STR pada Dim_Dokter
Tipe 2	Add New Row	Setiap pembaruan menghasilkan penambahan baris data baru beserta rentang penanda tanggal aktifnya.	Perubahan lokasi domisili / kota / status demografi pada Dim_Pasien

Rancangan Lima Dimension Table

Tabel 3.4: Rancangan Detail Tabel Dimensi

Dimensi	Tipe SCD	Primary Key	Atribut / Kolom Utama
Dim_Pasien	SCD Tipe 2	pasien_key	id_pasien, nama_pasien, jenis_kelamin, tanggal_lahir, alamat, kota, golongan_darah
Dim_Dokter	SCD Tipe 1	dokter_key	id_dokter, nama_dokter, spesialisasi, nomor_izin_praktek
Dim_Waktu	SCD Tipe 0	date_key	tanggal, nama_hari, bulan, kuartal, tahun, is_weekend_flag
Dim_Fasilitas	SCD Tipe 1	fasil_key	tipe_fasilitas (IGD, Rawat Inap, Rawat Jalan), status_keluar
Dim_Pembayaran	SCD Tipe 1	bayar_key	metode_pembayaran (BPJS, Swasta, Mandiri)

Dim_Pasien berperan sebagai dimensi subjek pelayanan yang menyimpan riwayat perpindahan domisili demografis menggunakan skema penambahan baris data baru (SCD Tipe 2). Sementara Dim_Waktu diturunkan sepenuhnya dari atribut operasional tanggal_kunjungan ke dalam struktur hierarki agregasi yang luas (hari ke minggu, bulan, dan tahun) secara permanen.

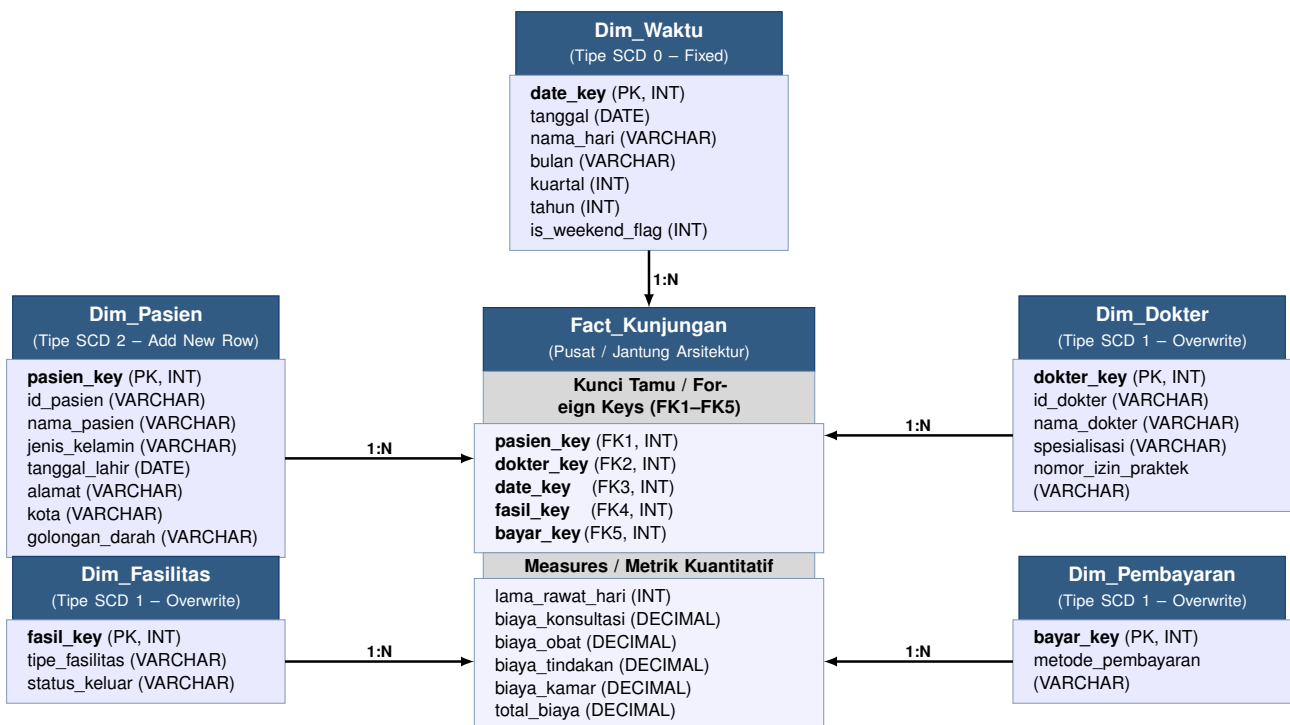
3.4 STAR SCHEMA DIAGRAM

Pendekatan topologi Data Warehouse diwujudkan melalui skema bintang (*Star Schema*). Arsitektur ini dipilih dibandingkan dengan skema hibrida seperti *Snowflake Schema* karena menawarkan kecepatan komputasi dan kesederhanaan eksekusi *query* OLAP melalui denormalisasi penuh ke arah tabel sentral.



Tabel 3.5: Pemetaan Komponen Arsitektur Star Schema

Nama Tabel	Jenis Model	Keterangan Arsitektur
Fact_Kunjungan	Fact Table	Inti perhitungan finansial dan kunjungan (5 kunci tamu + 6 kalkulasi numerik)
Dim_Pasien	Dimensi	Menjelaskan karakteristik pasien, terikat melalui pasien_key
Dim_Dokter	Dimensi	Menjelaskan identitas & spesialisasi medis, terikat melalui dokter_key
Dim_Waktu	Dimensi	Menentukan batasan agregasi <i>time-series</i> , terikat melalui date_key
Dim_Fasilitas	Dimensi	Memetakan pengelompokan jenis layanan unit, terikat via fasilitas_key
Dim_Pembayaran	Dimensi	Memetakan segmentasi arus kas/penjamin, terikat via bayar_key



Catatan Star Schema

Fact_Kunjungan berada di jantung arsitektur. Secara relasional, setiap rincian transaksi finansial tagihan medis diapit secara langsung (melalui konjungsi kunci indeks numerik tunggal) oleh penjelasan informatif dari lima dimensi pendukung: identitas Pasien yang dilayani, Dokter yang bertugas, Waktu kalender perawatan, letak Fasilitas penanganan, hingga model Pembayaran yang disepakati.

3.5 PERANCANGAN ARSITEKTUR STAGING DAN ALUR ETL

Untuk memastikan data transaksional dapat mengalir dari sistem sumber (file generator) ke dalam *Star Schema* dengan aman dan terstruktur, dirancang sebuah arsitektur *Extract, Transform, Load* (ETL) yang memanfaatkan *Staging Area*.



Konsep dan Fungsi Staging Area

Staging Area merupakan area penyimpanan sementara (*buffer zone*) yang menjembatani file CSV sumber dengan repositori akhir Data Warehouse. Penggunaan *staging area* dalam perancangan ini bertujuan untuk:

- **Ekstraksi Cepat:** Memindahkan data dari file `source_*.csv` ke dalam database relasional secepat mungkin untuk meminimalisasi beban baca pada file sumber.
- **Transformasi & Standarisasi:** Memvalidasi format tipe data (seperti format tanggal dan angka desimal) sebelum disuntikkan ke skema utama.
- **Pembentukan Surrogate Key:** Tempat pemetaan kunci alami (*Natural Key* seperti `id_pasien`) menjadi kunci pengganti numerik (*Surrogate Key*) untuk mempercepat performa *query* OLAP.

Pemetaan Tabel Staging

Skema *staging* dirancang dengan prinsip relasi satu-banding-satu (1:1) terhadap file CSV yang dihasilkan oleh Python. Data akan dimuat dengan metode *Truncate and Load* ke dalam tiga tabel *staging* berikut:

Tabel 3.6: Pemetaan File Sumber ke Tabel Staging

Nama File Sumber	Tabel Staging	Karakteristik Pemindahan Data
<code>source_pasien.csv</code>	<code>stg_pasien</code>	Memuat profil pasien mentah; tipe data string dan date dipertahankan.
<code>source_dokter.csv</code>	<code>stg_dokter</code>	Memuat data master dokter beserta spesialisasi dan biaya dasar konsultasi.
<code>source_kunjungan.csv</code>	<code>stg_kunjungan</code>	Memuat seluruh 12.000 transaksi penagihan; bertindak sebagai tabel beban terberat.

Alur Transformasi ke Star Schema

Setelah data mentah mendarat di tabel *staging*, proses *Transform* dan *Load* akan mendistribusikan data tersebut ke dalam 5 Tabel Dimensi dan 1 Tabel Fakta dengan urutan logis sebagai berikut:

Langkah 1: Pengisian Tabel Dimensi

Tabel dimensi wajib diisi lebih dahulu untuk menjaga integritas *Foreign Key*.

- **Dim_Waktu:** Sistem mengekstrak nilai *distinct* `tanggal_kunjungan` dari `stg_kunjungan`, lalu memecahnya menjadi komponen hari, bulan, tahun, dan *weekend flag*.
- **Dim_Fasilitas & Dim_Pembayaran:** Sistem mengambil nilai unik (IGD/Inap/Jalan) dan metode pembayaran (BPJS/Mandiri/Swasta) dari `stg_kunjungan`.
- **Dim_Pasien & Dim_Dokter:** Data dari `stg_pasien` dan `stg_dokter` dipindahkan ke dimensi utama. Pada tahap ini, sistem menghasilkan *Surrogate Key* (`pasien_key`, `dokter_key`) dan mengevaluasi status perubahan data untuk SCD.

Langkah 2: Pengisian Tabel Fakta



Pembentukan tabel `Fact_Kunjungan` dieksekusi melalui *query JOIN* antara `stg_kunjungan` dengan kelima tabel dimensi yang telah diperbarui.

- Sistem mengambil metrik nominal (*Measures*) secara langsung: `biaya_konsultasi`, `biaya_obat`, `biaya_tindakan`, `biaya_kamar`, `total_biaya`, dan `lama_rawat_hari`.
- Sistem melakukan proses *Lookup* pada setiap dimensi untuk mengganti *Natural Key* (misal: PSN-00001) menjadi *Surrogate Key* numerik.
- Seluruh kombinasi *Surrogate Key* dan *Measures* disuntikkan sebagai baris baru ke dalam `Fact_Kunjungan`.

Orkestrasi Pipeline ETL

Proses pergerakan data dari file CSV ke *Staging* hingga ke *Star Schema* akan diotomatisasi secara terjadwal menggunakan Apache Airflow. Hal ini menjamin siklus pembaruan laporan DWBI di rumah sakit berjalan harian secara konsisten tanpa intervensi manual.

Part IV

Synthetic Data Generation



BAB 4

Synthetic Data Generation

4.1 STRATEGI GENERATOR DATA SINTETIK

Proses perancangan ekosistem data rumah sakit dalam skala volume 12.000 transaksi mensyaratkan strategi simulasi logis yang merepresentasikan probabilitas keadaan lapangan yang sesungguhnya (*Constraint Enforcement*). Metodologi augmentasi Python berlandaskan aturan berikut:

- **Sinergi Kredensial Tarif Medis:** Beban tarif pada biaya_konsultasi bukanlah angka yang diacak sepenuhnya, melainkan dikunci berdasarkan nilai kamus tarif spesialisasi.
- **Isolasi Logika Fasilitas Non-Inap:** Jika alokasi unit merujuk ke “IGD” atau “Rawat Jalan”, algoritma menginstruksikan kalkulasi durasi inap absolut di angka 0 dan tidak membebankan biaya sewa ruangan.
- **Pembobotan Pemulihan Pasien Inap:** Pasien yang dialokasikan ke “Rawat Inap” memiliki hari rawat yang disimulasikan (1–14 hari) dan luaran klinis yang ditentukan oleh probabilitas empiris: 80% dinyatakan Sembuh, dan hanya sisanya Dirujuk/Meninggal.
- **Logika Integritas Keuangan (*Billing*):** total_biaya dieksekusi secara matematis ketat melalui penjumlahan komponen linier: jasa dokter + obat farmasi + tindakan alat + sewa ruangan, meniadakan kesalahan agregasi fiskal pada Data Warehouse.

4.2 KODE GENERATOR UTAMA

Berikut adalah *source code* Python yang dikompilasi secara deterministik untuk merakit data sumber transaksi rumah sakit operasional ke dalam direktori file CSV:

Listing 4.1: Generator Data Sintetik Operasional Rumah Sakit

```
1 import os
2 import random
3 from datetime import datetime, timedelta
4 import pandas as pd
5 from faker import Faker
6
7 # Inisialisasi Faker dengan lokalisasi Indonesia
8 fake = Faker('id-ID')
9 Faker.seed(42)
10 random.seed(42)
11
```



```

12 # Konfigurasi Jumlah Data
13 TOTAL_PASIEN = 2500
14 TOTAL_DOKTER = 80
15 TOTAL_TRANSAKSI_KUNJUNGAN = 12000
16
17 print("Memulai pembuatan dataset sintetik rumah sakit...")
18
19 # =====
20 # 1. GENERATE TABEL SUMBER: PASIEN
21 # =====
22 data_pasien = []
23 golongan_darah_opsi = ['A', 'B', 'AB', 'O']
24 jenis_kelamin_opsi = ['Laki-laki', 'Perempuan']
25
26 for pasien_id in range(1, TOTAL_PASIEN + 1):
27     jk = random.choice(jenis_kelamin_opsi)
28     nama = fake.name_male() if jk == 'Laki-laki' else fake.name_female()
29     data_pasien.append({
30         'id_pasien'       : f'PSN-{pasien_id:05d}',
31         'nama_pasien'     : nama,
32         'jenis_kelamin'   : jk,
33         'tanggal_lahir'   : fake.date_of_birth(minimum_age=0, maximum_age=85)
34                             .strftime('%Y-%m-%d'),
35         'alamat'         : fake.street_address(),
36         'kota'           : fake.city(),
37         'golongan_darah' : random.choice(golongan_darah_opsi)
38     })
39 df_pasien = pd.DataFrame(data_pasien)
40
41 # =====
42 # 2. GENERATE TABEL SUMBER: DOKTER
43 # =====
44 data_dokter = []
45 spesialisasi_opsi = {
46     'Umum': 30, 'Anak': 40, 'Penyakit Dalam': 50,
47     'Bedah': 60, 'Jantung': 80, 'Saraf': 70, 'Mata': 45
48 }
49
50 for dokter_id in range(1, TOTAL_DOKTER + 1):
51     spesialisasi = random.choice(list(spesialisasi_opsi.keys()))
52     biaya_konsultasi = spesialisasi_opsi[spesialisasi] * 1000
53     data_dokter.append({
54         'id_dokter'       : f'DKT-{dokter_id:03d}',
55         'nama_dokter'     : fake.name(),
56         'spesialisasi'    : spesialisasi,
57         'biaya_konsultasi' : biaya_konsultasi,
58         'nomor_izin_praktek' : f'STR-{random.randint(100000, 999999)}'
59     })
60 df_dokter = pd.DataFrame(data_dokter)
61
62 # =====
63 # 3. GENERATE TABEL SUMBER: KUNJUNGAN
64 # =====
65 data_kunjungan = []
66 tipe_fasilitas_opsi = ['IGD', 'Rawat Jalan', 'Rawat Inap']
67 status_keluar_opsi = ['Sembuh', 'Dirujuk', 'Pulang Paksa', 'Meninggal']
68 metode_pembayaran_opsi = ['BPJS', 'Asuransi Swasta', 'Mandiri']

```



```

69 start_date = datetime(2025, 1, 1)
70
71 for kunjungan_id in range(1, TOTAL_TRANSAKSI_KUNJUNGAN + 1):
72     pasien = random.choice(data_pasien)
73     dokter = random.choice(data_dokter)
74     tipe_fasilitas = random.choice(tipe_fasilitas_opsi)
75     metode_pembayaran = random.choice(metode_pembayaran_opsi)
76
77     # Logika Lama Rawat Inap (Length of Stay)
78     if tipe_fasilitas == 'Rawat Inap':
79         lama_rawat = random.randint(1, 14)
80         status_keluar = random.choices(
81             status_keluar_opsi, weights=[80, 12, 6, 2])[0]
82         biaya_kamar = lama_rawat * random.choice([250000, 500000,
83             1200000])
84     else:
85         lama_rawat = 0
86         status_keluar = 'Sembuh'
87         biaya_kamar = 0
88
89     # Simulasi Komponen Biaya Medis
90     biaya_konsul = dokter['biaya_konsultasi']
91     biaya_obat = random.randint(50000, 750000)
92     if tipe_fasilitas != 'IGD':
93         biaya_obat = random.randint(30000, 200000)
94     biaya_tindakan = random.choice([0, 150000, 300000, 1500000])
95     if tipe_fasilitas in ['IGD', 'Rawat Inap'] else 0)
96     total_biaya = biaya_konsul + biaya_obat + biaya_tindakan +
97         biaya_kamar
98     tgl_kunjungan = start_date + timedelta(days=random.randint(0, 450))
99
100     data_kunjungan.append({
101         'id_kunjungan' : f'TX-{kunjungan_id:06d}',
102         'id_pasien' : pasien['id_pasien'],
103         'id_dokter' : dokter['id_dokter'],
104         'tanggal_kunjungan' : tgl_kunjungan.strftime('%Y-%m-%d %H:%M:%S'),
105         'tipe_fasilitas' : tipe_fasilitas,
106         'lama_rawat_hari' : lama_rawat,
107         'status_keluar' : status_keluar,
108         'metode_pembayaran' : metode_pembayaran,
109         'biaya_konsultasi' : biaya_konsul,
110         'biaya_obat' : biaya_obat,
111         'biaya_tindakan' : biaya_tindakan,
112         'biaya_kamar' : biaya_kamar,
113         'total_biaya' : total_biaya
114     })
115
116 df_kunjungan = pd.DataFrame(data_kunjungan)
117
118 # =====
119 # 4. MENYIMPAN DATA KE CSV
120 # =====
121 OUTPUT_DIR = '/opt/airflow/data_generator/output'
122 os.makedirs(OUTPUT_DIR, exist_ok=True)
123
124 df_pasien.to_csv(os.path.join(OUTPUT_DIR, 'source_pasien.csv'), index=False)
125 df_dokter.to_csv(os.path.join(OUTPUT_DIR, 'source_dokter.csv'), index=False)
126 df_kunjungan.to_csv(os.path.join(OUTPUT_DIR, 'source_kunjungan.csv'), index=

```




```

False)
124
125 print("Pembuatan dataset selesai!")
126 print(f"    - Total Pasien    : {len(df_pasien)} baris")
127 print(f"    - Total Dokter    : {len(df_dokter)} baris")
128 print(f"    - Total Kunjungan: {len(df_kunjungan)} baris")

```

4.3 CONTOH DATA MENTAH (OUTPUT SOURCE_KUNJUNGAN.CSV)

Berikut adalah cuplikan simulasi hasil output transaksi yang memperlihatkan keutuhan integritas perhitungan *billing* matematis dalam *pipeline* DWBI ini.

Tabel 4.1: Contoh Data Mentah Transaksi Kunjungan

ID Kunjun- gan	Fasilitas	LOS	Biaya Kon- sul	Biaya Obat	Biaya Tin- dakan	Biaya Ka- mar	Total (Rp)
TX-000001	Rawat Inap	4	50.000	630.000	300.000	1.000.000	1.980.000
TX-000002	IGD	0	80.000	75.000	1.500.000	0	1.655.000
TX-000003	Rawat Jalan	0	30.000	125.000	0	0	155.000

4.4 SKEMA SUMBER (DATA DICTIONARY)

Skema akhir luaran generator diklasifikasikan menjadi tiga repositori utama yang saling terhubung menggunakan *Primary Key* (PK) dan *Foreign Key* (FK).

Sumber 1 — source_pasien.csv

Data master yang berisi informasi profil identitas pasien, meliputi: *id_pasien* (PK), *nama_pasien*, *jenis_kelamin*, *tanggal_lahir*, *alamat*, *kota*, dan *golongan_darah*.

Sumber 2 — source_dokter.csv

Data master yang berisi parameter profesi kesehatan dokter, meliputi: *id_dokter* (PK), *nama_dokter*, *spesialisasi*, *biaya_konsultasi*, dan *nomor_izin_praktek*.

Sumber 3 — source_kunjungan.csv

Data transaksi historis operasional yang menautkan kunci referensial dan parameter beban tagihan pasien, meliputi: *id_kunjungan* (PK), *id_pasien* (FK), *id_dokter* (FK), *tanggal_kunjungan*, *tipe_fasilitas*, *lama_rawat_hari*, *status_keluar*, *metode_pembayaran*, *biaya_konsultasi*, *biaya_obat*, *biaya_tindakan*, *biaya_kamar*, dan *total_biaya*.

Part V

Implementasi Orkestrasi Apache Airflow



BAB 5

Implementasi Orkestrasi Apache Airflow

5.1 ARSITEKTUR DAG (DIRECTED ACYCLIC GRAPH)

Apache Airflow bertindak sebagai mesin pengatur dan penjadwal utama (*enterprise orchestrator*) untuk mengotomatisasi seluruh siklus data dalam sistem DWBI rumah sakit ini. Di dalam Airflow, rangkaian alur kerja didefinisikan sebagai **Directed Acyclic Graph (DAG)** bernama `02_healthcare_data_pipeline`. Karakteristik "*directed*" menandakan alur eksekusi memiliki arah ketergantungan yang jelas dari awal hingga akhir, sedangkan "*acyclic*" menjamin bahwa tidak ada tugas yang berjalan berulang atau membentuk putaran tak berujung (*infinite loop*).

Konfigurasi parameter operasional dan manajemen risiko dari DAG ini dirancang menggunakan argumen bawaan (*default args*) sebagai berikut:

- **Owner** (`kelompok_healthcare`): Digunakan sebagai identitas pengelola aset *pipeline* untuk mempermudah kategorisasi, audit log, dan pemfilteran pada dasbor Web UI Airflow.
- **Depends on Past (False)**: Keberhasilan eksekusi *pipeline* hari ini tidak digantungkan pada status keberhasilan eksekusi hari sebelumnya, sehingga mencegah kemacetan massal (*data blocking*).
- **Retries (1) & Retry Delay (2 menit)**: Jika sebuah *task* gagal akibat masalah jaringan atau database PostgreSQL yang sedang sibuk, Airflow akan menjadwalkan ulang eksekusi otomatis satu kali lagi setelah jeda 2 menit.
- **Schedule Interval (None) & Catchup (False)**: *Pipeline* dikonfigurasi untuk berjalan berdasarkan pemicu manual (*Manual Trigger*). Penonaktifan *catchup* mencegah Airflow melakukan eksekusi susulan (*backfilling*) terhadap jadwal historis masa lalu.

5.2 TOPOLOGI DAN PENJELASAN TASK DEPENDENCIES

Pipeline dipecah menjadi 6 *task* diskrit yang dikelola oleh dua operator utama: `BashOperator` untuk mengeksekusi perintah di lingkungan *shell* Linux, dan `PythonOperator` untuk mengeksekusi fungsi Python di dalam memori proses Airflow. Keenam *task* diikat menggunakan operator dependensi sekuensial linear (`»`), membentuk urutan kronologis sebagai berikut:

Listing 5.1: Definisi alur dependensi keseluruhan pipeline



```
1 task_generate_data >> task_ingest_data >> task_auto_generate_models \
2 >> task_dbt_run >> task_dbt_test >> task_dbt_docs >> task_dbt_docs_serve
```

Task 1: run_data_generator (BashOperator)

Airflow memicu lingkungan *shell* Linux untuk menjalankan skrip pembuat data sintetik secara langsung melalui interpreter Python yang tersedia di dalam kontainer Docker.

Listing 5.2: Definisi Task run_data_generator

```
1 task_generate_data = BashOperator(
2     task_id='run_data_generator',
3     bash_command='python /opt/airflow/data_generator/src/generator.py',
4 )
```

Airflow memastikan skrip *generator.py* selesai dieksekusi sepenuhnya dan menghasilkan tiga file CSV (*source_pasien.csv*, *source_dokter.csv*, *source_kunjungan.csv*) di direktori *shared storage* */opt/airflow/data_generator/output* sebelum mengizinkan aliran data berpindah ke *task* berikutnya.

Task 2: ingest_csv_to_database (PythonOperator)

Airflow memanggil fungsi Python *ingest_csv_to_postgres* secara langsung di dalam memori proses *worker* untuk memindahkan data dari file *.csv* ke tabel relasional PostgreSQL.

Listing 5.3: Definisi Task ingest_csv_to_database

```
1 task_ingest_data = PythonOperator(
2     task_id='ingest_csv_to_database',
3     python_callable=ingest_csv_to_postgres,
4 )
```

Fungsi ini menggunakan *connection string* *postgresql+psycopg2://airflow:airflow@postgres:5432/airflow* untuk terhubung ke PostgreSQL via SQLAlchemy. Data dibaca melalui *DataFrame* Pandas, kemudian ditulis ke tabel *src_pasien*, *src_dokter*, dan *src_kunjungan* dengan strategi pemuatan *idempotent*: jika tabel sudah ada, isinya dikosongkan terlebih dahulu (TRUNCATE) sebelum data baru dimasukkan (*if_exists='append'*); jika tabel belum ada, tabel dibuat otomatis (*if_exists='fail'* pada eksekusi pertama). Seluruh operasi ini dibungkus dalam satu blok transaksi *engine.begin()* untuk menjamin konsistensi data.

Task 3: auto_generate_dbt_files (PythonOperator)

Airflow memanggil fungsi *auto_generate_dbt_models* untuk mengunduh konfigurasi model dbt secara dinamis dari repositori GitHub dan membuat seluruh file SQL model secara otomatis di dalam volume Docker.

Listing 5.4: Definisi Task auto_generate_dbt_files

```
1 task_auto_generate_models = PythonOperator(
2     task_id='auto_generate_dbt_files',
3     python_callable=auto_generate_dbt_models,
4 )
```

Fungsi ini mengunduh file *manifest* YAML dari *raw URL* repositori GitHub (*dbt_manifest.yml*), kemudian mengiterasi setiap entri untuk membuat file *.sql* model dbt di direktori */opt/airflow/dbt_project/models/*



dengan sub-folder *staging*, *dimensions*, dan *marts*. Secara bersamaan, fungsi mendeteksi nama tabel sumber melalui ekspresi reguler pada pemanggilan `source()`, lalu secara otomatis menghasilkan file `sources.yml` agar dbt dapat mengenali skema PostgreSQL tanpa konfigurasi manual.

Task 4: dbt_run_transformation (BashOperator)

Airflow memerintahkan dbt sebagai *third-party transformation tool* untuk memproses seluruh model SQL dan membangun struktur *Star Schema* secara nyata di dalam database.

Listing 5.5: Definisi Task dbt_run_transformation

```
1 task_dbt_run = BashOperator(  
2     task_id='dbt_run_transformation',  
3     bash_command=(  
4         'cd /opt/airflow/dbt_project && '  
5         'dbt run --target docker_env --profiles-dir .'6     ),  
7 )
```

Airflow mengirimkan perintah CLI `dbt run` yang menginstruksikan dbt untuk mengompilasi dan mengeksekusi seluruh model SQL mulai dari lapisan *staging*, *dimensions*, hingga *marts*, sehingga menghasilkan tabel fakta dan dimensi (*Star Schema*) yang persisten di dalam PostgreSQL.

Task 5: dbt_data_quality_test (BashOperator)

Airflow mengeksekusi rangkaian pengujian kualitas data yang telah didefinisikan dalam skema YAML dbt sebelum hasil transformasi dinyatakan siap dikonsumsi oleh lapisan analitik.

Listing 5.6: Definisi Task dbt_data_quality_test

```
1 task_dbt_test = BashOperator(  
2     task_id='dbt_data_quality_test',  
3     bash_command=(  
4         'cd /opt/airflow/dbt_project && '  
5         'dbt test --target docker_env --profiles-dir .'6     ),  
7 )
```

Perintah `dbt test` mengeksekusi seluruh pengujian integritas yang telah dikonfigurasi, mencakup pemeriksaan `unique`, `not_null`, `accepted_values`, dan `relationships` antar tabel. Apabila salah satu pengujian menghasilkan kegagalan, Airflow akan menandai *task* ini dengan status *Failed* (ditampilkan dalam warna merah pada UI) dan menghentikan seluruh *pipeline* secara otomatis, sehingga data yang tidak valid tidak sampai ke lapisan pelaporan.

Task 6: dbt_generate_docs (BashOperator)

Airflow mengeksekusi proses pembangkitan dokumentasi dbt secara otomatis setelah seluruh pengujian kualitas data dinyatakan lulus.

Listing 5.7: Definisi Task dbt_generate_docs

```
1 task_dbt_docs = BashOperator(  
2     task_id='dbt_generate_docs',  
3     bash_command=(  
4         'cd /opt/airflow/dbt_project && '  
5     )
```



```

5      'dbt docs generate --target docker_env --profiles-dir .'
6    ),
7  )

```

Perintah `dbt docs generate` mengompilasi seluruh metadata model, kolom, deskripsi, dan hasil pengujian menjadi artefak statis berupa file `manifest.json` dan `catalog.json` yang menjadi fondasi portal dokumentasi interaktif dbt.

Task 7: dbt_serve_docs (BashOperator)

Airflow menjalankan server dokumentasi dbt di *background* agar *task* dapat langsung berstatus *Success* tanpa memblokir proses Airflow.

Listing 5.8: Definisi Task dbt_serve_docs

```

1 task_dbt_docs_serve = BashOperator(
2     task_id='dbt_serve_docs',
3     bash_command='''
4         fuser -k 8081/tcp || true
5         cd /opt/airflow/dbt_project && \
6         nohup dbt docs serve --port 8081 \
7             > /opt/airflow/logs/dbt_docs.log 2>&1 &
8     ''',
9 )

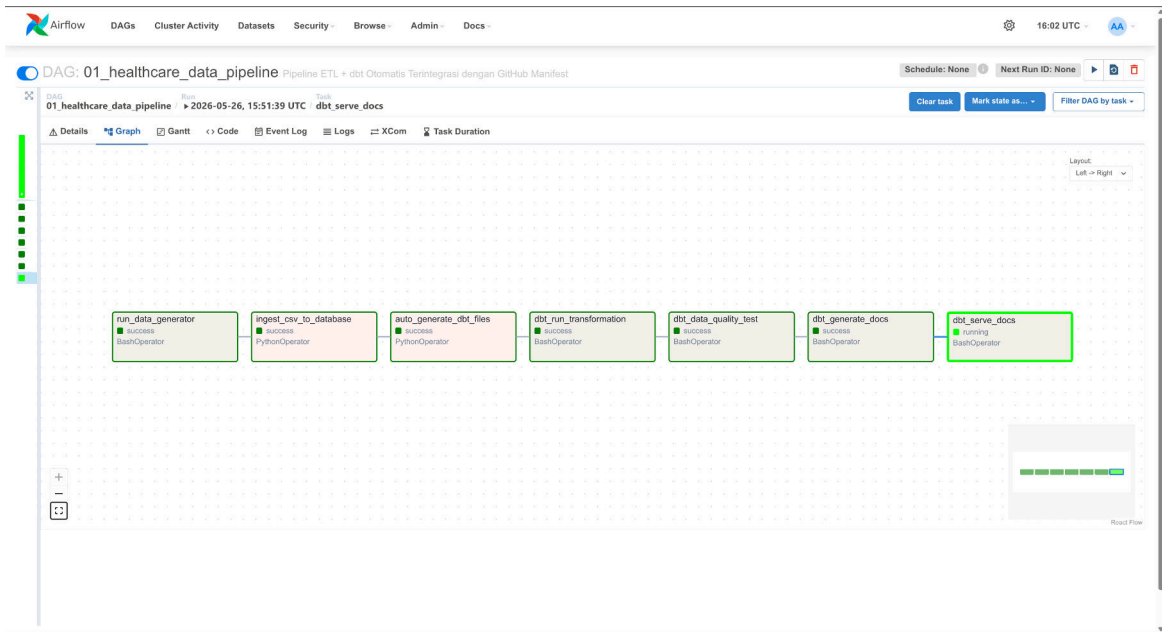
```

Task ini terlebih dahulu memeriksa apakah *port* 8081 sudah aktif; jika ya, proses lama dimatikan via `fuser -k` sebelum server baru dijalankan. Server dbt kemudian dijalankan menggunakan `nohup ... &` agar berjalan sebagai proses *daemon* yang tidak bergantung pada siklus hidup proses Airflow. Portal dokumentasi dapat diakses melalui `http://localhost:8081` setelah *task* ini berhasil.

5.3 PANDUAN DOKUMENTASI ANTARMUKA AIRFLOW UI

Untuk membuktikan bahwa arsitektur orkestrasi ini berjalan dengan sukses di lingkungan Docker, lampirkan beberapa tangkapan layar dari Apache Airflow Web UI (akses via `localhost:8080`) dengan penjelasan sebagai berikut:

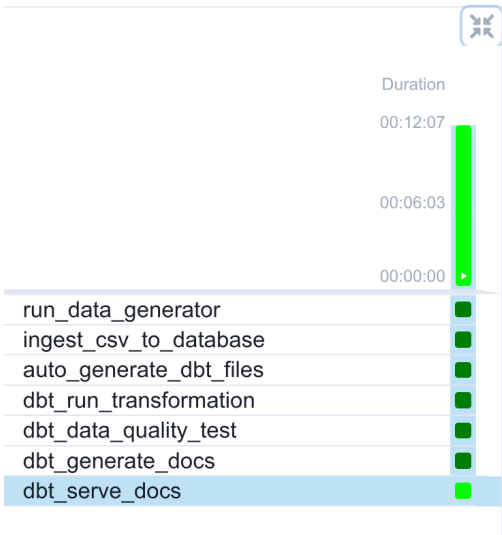
- **Screenshot Graph View:**



Gambar 5.1: Gambar 5.1 Tampilan Graph View Ketergantungan Task.

Gambar ini mendokumentasikan visualisasi topologi linear dari pipeline. Terlihat jelas arah panah eksekusi dari kiri ke kanan yang dimulai dari pembuatan data hingga pengujian kualitas data. Seluruh bingkai kotak (node) harus dipastikan berwarna hijau tua (dark green), yang menjadi indikator absolut di Airflow bahwa seluruh perintah script sukses dieksekusi tanpa interupsi.

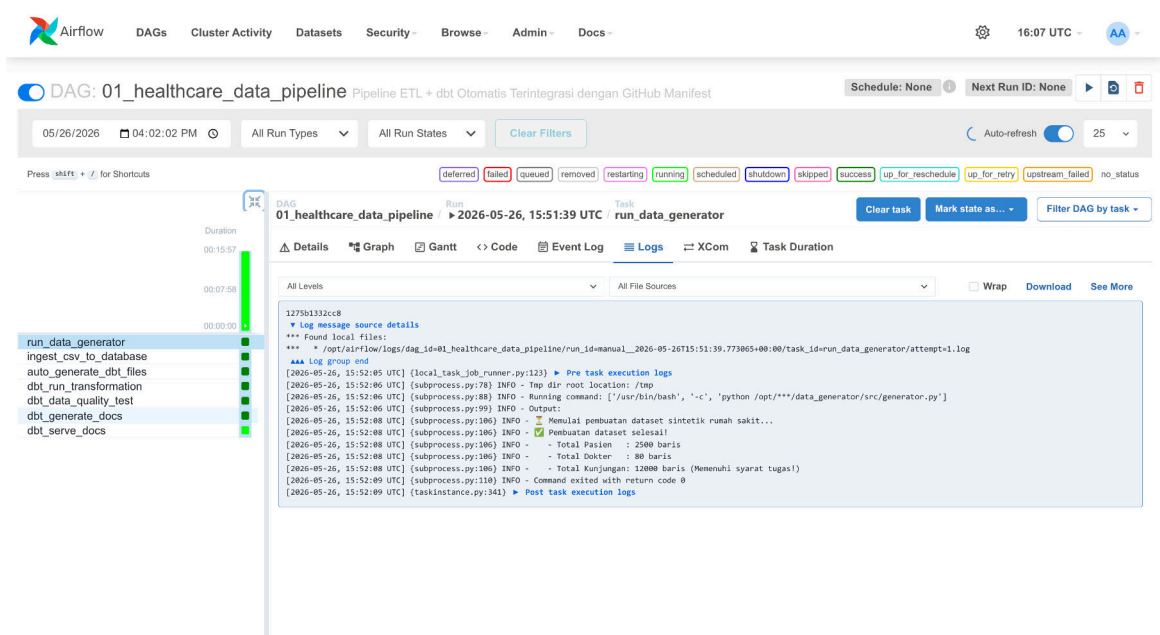
• Screenshot Grid / Tree View:



Gambar 5.2: Gambar 5.2 Tampilan Grid View Histori Eksekusi Pipeline.

Dokumentasi ini memperlihatkan rekam jejak (log run) dari eksekusi DAG. Lajur kotak vertikal berwarna hijau penuh menandakan keberhasilan run secara utuh pada baris waktu tertentu. Melalui tampilan ini, pengelola Data Warehouse dapat memantau durasi performa kecepatan eksekusi tiap task dan mendeteksi jika terjadi lonjakan beban komputasi (bottleneck).

• Screenshot Task Log:



Gambar 5.3: Gambar 5.3 Tampilan Log Output Terminal Eksekusi Task.

Dokumen log menunjukkan status keluaran sistem yang bersih dengan kode balik INFO - Command exited with return code 0 pada task run_data_generator, membuktikan Airflow berhasil menyelesaikan pemanggilan fungsi eksternal dengan sukses.

Part VI

Implementasi dbt (Data Build Tool)



BAB 6

Implementasi dbt (Data Build Tool)

Proses transformasi data dalam arsitektur Data Warehouse kesehatan ini menerapkan pendekatan ELT (*Extract-Load-Transform*). Setelah data mentah (*raw data*) dimuat ke dalam database pementasan oleh Apache Airflow, seluruh lapisan transformasi dan pemodelan data diserahkan kepada **dbt** (*Data Build Tool*).

Peran dbt di dalam sistem ini adalah mengelola kode SQL transformasi secara modular, menerapkan prinsip rekayasa perangkat lunak seperti kontrol versi (*version control*), memisahkan logika bisnis ke dalam beberapa tingkatan (*layering*), melakukan pengujian integritas otomatis, serta menghasilkan dokumentasi skema data secara interaktif.

6.1 STRUKTUR PROYEK DBT

Proyek dbt diintegrasikan di dalam ekosistem Docker Apache Airflow pada direktori `/opt/airflow/dbt_project`. Struktur direktori ini dirancang secara modular untuk memisahkan konfigurasi global, kredensial keamanan, dan berkas kode transformasi SQL.

Berikut adalah visualisasi struktur direktori (*directory tree*) dari proyek dbt secara menyeluruh:



```

dbt_project/
├── dbt_project.yml .....Konfigurasi utama proyek dbt
├── profiles.yml ..... Konfigurasi keamanan dan koneksi database
├── target/ .....Berkas hasil kompilasi otomatis dbt (generated)
├── models/
│   ├── staging/
│   │   ├── sources.yml ..... Deklarasi tabel sumber mentah (PostgreSQL public)
│   │   ├── stg_pasien.sql ..... Transformasi awal data pasien
│   │   ├── stg_dokter.sql ..... Transformasi awal data dokter
│   │   └── stg_kunjungan.sql ..... Transformasi awal data kunjungan
│   ├── dimensions/
│   │   ├── dim_pasien.sql ..... Pembentukan tabel master dimensi pasien
│   │   ├── dim_dokter.sql ..... Pembentukan tabel master dimensi dokter
│   │   ├── dim_waktu.sql ..... Pembentukan tabel master dimensi deret waktu
│   │   ├── dim_fasilitas.sql ..... Pembentukan dimensi tipe fasilitas pelayanan
│   │   └── dim_pembayaran.sql ..... Pembentukan dimensi jenis metode pembayaran
│   └── marts/
│       ├── fct_kunjungan.sql ..... Pembentukan tabel fakta utama transaksi kunjungan
│       └── schema.yml ..... Deklarasi pengujian data & dokumentasi kolom

```

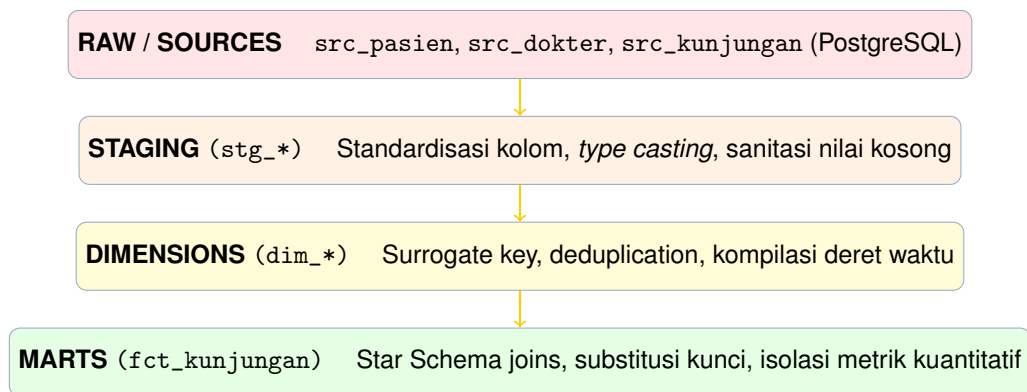
Gambar 6.1: Struktur Direktori Proyek dbt

Penjelasan fungsional dari berkas konfigurasi utama proyek adalah sebagai berikut:

- `dbt_project.yml`: Berkas konfigurasi pusat yang mendefinisikan nama proyek (`healthcare_transformation`), versi, konfigurasi profil koneksi yang harus digunakan, serta aturan materialisasi (*materialization*) global untuk masing-masing direktori `models`. Direktori `staging` dikonfigurasi sebagai `view` guna menghemat ruang penyimpanan, sedangkan direktori `dimensions` dan `marts` dimaterialisasikan sebagai `table` secara fisik untuk mengoptimalkan kecepatan *query* analitik.
- `profiles.yml`: Menyimpan detail teknis kredensial database PostgreSQL, termasuk alamat *host* jaringan Docker (`postgres`), *port* (5432), nama database (`airflow`), skema target (`public`), serta batasan jumlah *threads* komputasi paralel yang diizinkan untuk mengeksekusi *query* secara bersamaan.

6.2 PENJELASAN TIAP LAPISAN (LAYERING ARCHITECTURE)

Untuk menjamin pemeliharaan kode yang berkelanjutan dan mencegah redundansi logika SQL, arsitektur transformasi dbt dibagi menjadi tiga lapisan terisolasi yang saling bergantung secara sekuensial (*lineage dependency*):



1. Lapisan Staging (models/staging/)

Lapisan *staging* merupakan gerbang pertama yang berhadapan langsung dengan data mentah hasil ekstraksi dari file CSV (*src_pasien*, *src_dokter*, *src_kunjungan*). Prinsip utama pada lapisan ini adalah melakukan pembersihan awal tanpa mengubah granularitas data asli. Transformasi yang dilakukan meliputi:

- **Standardisasi Nama Kolom (*Renaming*):** Mengubah format nama kolom yang tidak seragam dari sistem sumber menjadi format *snake_case* yang baku (misalnya mengubah *NamaDokter* menjadi *nama_dokter*).
- **Penyelarasan Tipe Data (*Type Casting*):** Mengonversi tipe data teks dari CSV menjadi tipe data yang tepat pada database, seperti mengubah format tanggal teks menjadi tipe *DATE*, dan nominal biaya menjadi tipe *NUMERIC(15, 2)* untuk presisi finansial.
- **Sanitasi Data Mentah (*Data Cleansing*):** Mengidentifikasi nilai kosong (*null values*) dan menggunakan fungsi *COALESCE* untuk memberikan nilai bawaan (*default value*) yang representatif agar tidak merusak kalkulasi analitik di tahap berikutnya.

2. Lapisan Dimensions (models/dimensions/)

Lapisan *dimensions* berfungsi untuk membangun tabel-tabel master penjelas (*contextual tables*) yang berisi atribut deskriptif mengenai entitas bisnis rumah sakit. Tabel-tabel ini dirancang dalam bentuk terdenormalisasi penuh untuk mengoptimalkan kecepatan pembacaan data saat proses visualisasi. Transformasi utama pada lapisan ini meliputi:

- **Pembuatan Kunci Pengganti (*Surrogate Key*):** dbt menghasilkan *Surrogate Key* unik berbasis algoritma *hashing* biner (MD5) untuk menggantikan kunci alami (*natural key*) dari sistem sumber. Hal ini penting untuk menjaga independensi Data Warehouse jika terjadi perubahan struktur pada database operasional di masa mendatang.
- **Eliminasi Duplikasi (*Deduplication*):** Menerapkan fungsi jendela SQL (*ROW_NUMBER()*) untuk memastikan bahwa setiap entitas (seperti pasien dan dokter) bersifat unik dan tidak memiliki rekaman ganda.
- **Kompilasi Deret Waktu (*Time Dimension Expansion*):** Memecah satu kolom tanggal kunjungan menjadi atribut temporal yang kaya, seperti hari, nama hari, bulan, tahun, kuartal, dan indikator hari kerja (*weekday/weekend*).



3. Lapisan Marts (models/marts/)

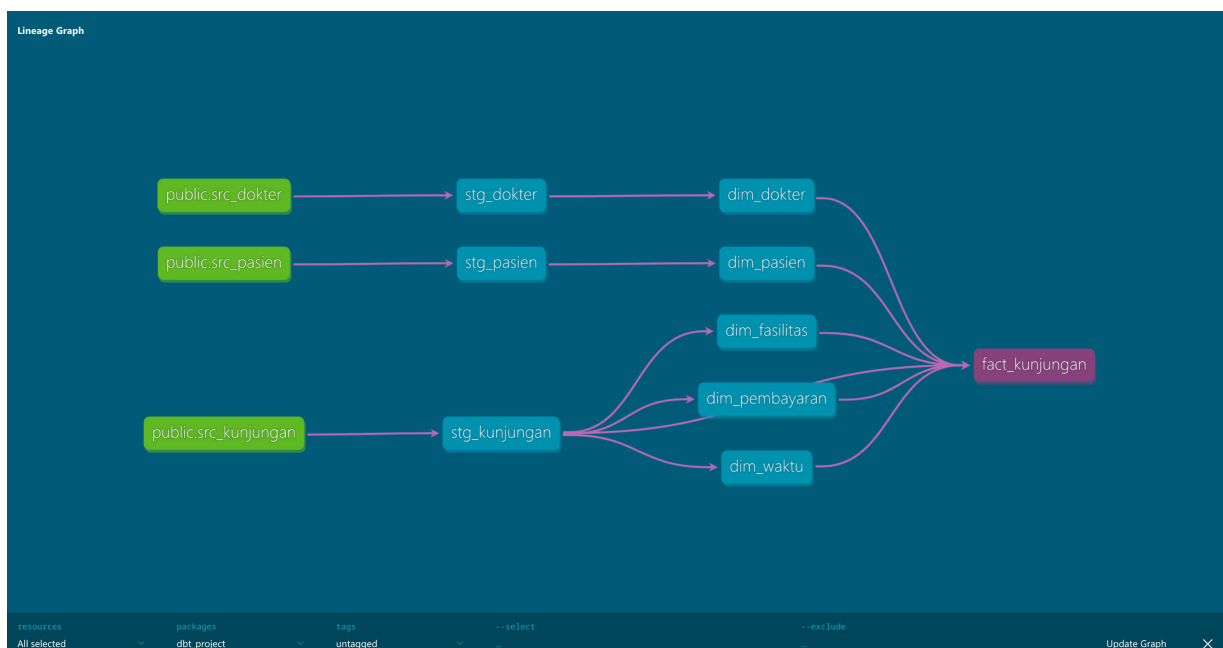
Lapisan *marts* adalah lapisan produk data akhir (*end-product layer*) yang merepresentasikan domain bisnis operasional kunjungan medis dan finansial rumah sakit. Lapisan ini menghasilkan tabel fakta utama bernama *fact_kunjungan*. Transformasi pada tahap ini meliputi:

- **Operasi Penyambungan Relasional (*Star Schema Joins*):** Menggabungkan tabel pementasan kunjungan (*stg_kunjungan*) dengan seluruh tabel dimensi yang telah terbentuk pada lapisan sebelumnya.
- **Substitusi Kunci:** Mengganti seluruh *natural ID* asli dengan *Surrogate Key* yang berasal dari tabel dimensi untuk membentuk hubungan kunci asing (*Foreign Keys*) yang solid.
- **Isolasi Metrik Kuantitatif (*Measures*):** Menyediakan kolom-kolom numerik murni yang dapat dijumlahkan (*additive measures*), seperti durasi rawat inap, biaya konsultasi, biaya obat, biaya tindakan, dan biaya kamar.

6.3 HASIL DBT DOCS DAN LINEAGE GRAPH

Salah satu keunggulan implementasi dbt dalam proyek ini adalah kemampuan otomatisasi dokumentasi melalui *task task_dbt_docs* dan *task_dbt_docs_serve* pada DAG Airflow. Ketika *pipeline* berjalan sukses, dbt mengekstrak seluruh metadata database, deskripsi kolom, serta batasan pengujian ke dalam berkas JSON statis, lalu mempublikasikannya melalui *web server* lokal pada *port 8081*.

Dokumentasi interaktif tersebut menyediakan **Data Lineage Graph** (Grafik Silsilah Data). Grafik ini menggambarkan visualisasi terarah dari ujung ke ujung (*end-to-end*) mengenai bagaimana data mengalir: dimulai dari simpul sumber mentah (*source nodes*) → dialirkan menuju simpul pementasan (*staging nodes*) → dipetakan ke dalam simpul dimensi (*dimension nodes*) → dan akhirnya bermuara pada tabel fakta akhir *fact_kunjungan* di lapisan *marts*. Grafik silsilah data ini menjamin transparansi penuh sehingga tim analis dapat melacak akar masalah dengan cepat jika ditemukan anomali data pada laporan visual.



Gambar 6.2: Tampilan *Data Lineage Graph* dbt Docs pada *port 8081*.

Part VII

Pembangunan Data Mart



BAB 7

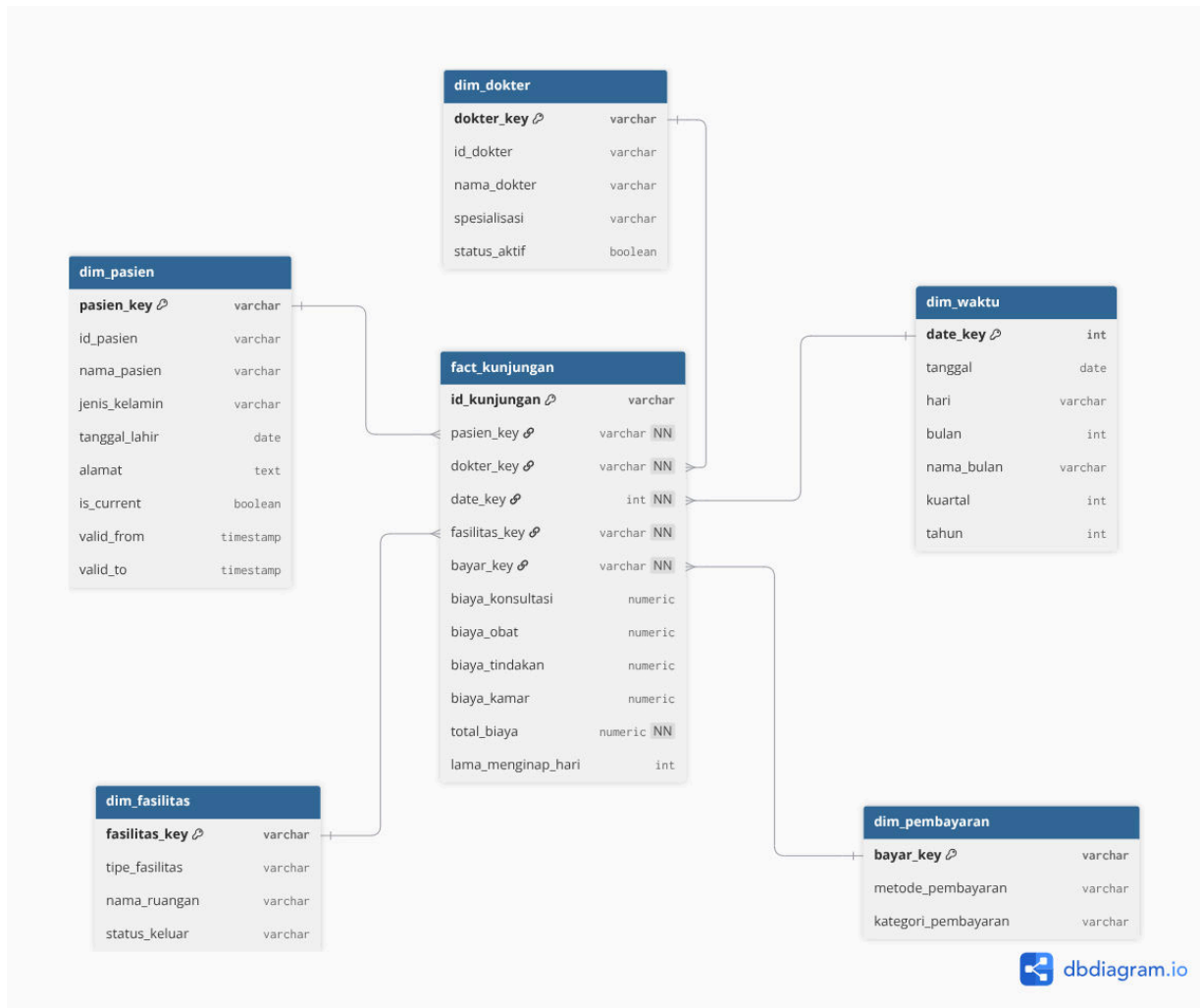
Pembangunan Data Mart

Data mart yang dibangun merupakan representasi fisik dari informasi yang telah dibersihkan, divalidasi, dan dioptimalkan, serta siap dikonsumsi secara langsung oleh jajaran manajemen rumah sakit untuk keperluan pengambilan keputusan strategis.

7.1 SKEMA FINAL DATA MART

Arsitektur fisik yang dipilih untuk Data Mart ini adalah **Skema Bintang (Star Schema) Murni**. Skema bintang dipilih karena memiliki struktur yang sangat sederhana, mudah dipahami oleh pengguna bisnis, dan meminimalkan operasi penyambungan tabel (JOIN) yang kompleks. Hal ini membuat proses eksekusi *query* agregasi pada *Business Intelligence tools* menjadi sangat responsif.

Tabel Fakta (*fact_kunjungan*) ditempatkan sebagai pusat dari arsitektur, mengumpulkan seluruh metrik kuantitatif transaksi medis. Tabel fakta ini kemudian dikelilingi dan diapit oleh lima Tabel Dimensi sebagai penjelas konteks peristiwa, yang terikat melalui hubungan *One-to-Many* (1:N) dengan integritas referensial yang ketat.



Gambar 7.1: Diagram ERD *Star Schema* Final dari Database PostgreSQL.

7.2 DOKUMENTASI TABEL DAN KOLOM (KAMUS DATA)

Guna memastikan kejelasan informasi dan menghindari ambiguitas data, berikut adalah spesifikasi teknis kamus data (*data dictionary*) untuk tabel fakta dan tabel dimensi utama di dalam Data Mart.

1. Tabel Fakta: fct_kunjungan

Deskripsi: Menyimpan rekam jejak transaksi kunjungan pasien, durasi pelayanan, serta rincian biaya finansial yang terjadi di rumah sakit. Materialisasi: Table (Fisik).



Tabel 7.1: Kamus Data Tabel Fakta fct_kunjungan

Nama Kolom	Tipe Data	Peran	Aturan Validasi	Deskripsi Fungsional
kunjungan_key	VARCHAR(50)	Primary (PK)	Key unique, not_null	Kode hash unik per transaksi kunjungan.
pasien_key	INT	Foreign (FK)	Key not_null, rel. ke dim_pasien	Kunci relasi ke master data pasien.
dokter_key	INT	Foreign (FK)	Key not_null, rel. ke dim_dokter	Kunci relasi ke master data dokter pemeriksa.
waktu_key	INT	Foreign (FK)	Key not_null, rel. ke dim_waktu	Kunci relasi ke kalender dimensi temporal.
fasilitas_key	VARCHAR(50)	Foreign (FK)	Key not_null	Kunci relasi ke klaster jenis fasilitas.
pembayaran_key	VARCHAR(50)	Foreign (FK)	Key not_null	Kunci relasi ke dimensi metode pembayaran.
durasi_rawat	INT	Measure	nilai ≥ 0	Lama waktu pasien dirawat (satuan: hari).
biaya_konsultasi	NUMERIC(15,2)	Measure	nilai ≥ 0	Nominal biaya untuk jasa pemeriksaan dokter.
biaya_obat	NUMERIC(15,2)	Measure	nilai ≥ 0	Nominal biaya pengadaan obat-obatan pasien.
biaya_tindakan	NUMERIC(15,2)	Measure	nilai ≥ 0	Nominal biaya pengerjaan prosedur medis/operasi.
biaya_kamar	NUMERIC(15,2)	Measure	nilai ≥ 0	Nominal biaya sewa ruang rawat inap.
total_biaya	NUMERIC(15,2)	Calculated Measure	nilai ≥ 0	Akumulasi total biaya keseluruhan layanan medis.

2. Tabel Dimensi: dim_pasien

Deskripsi: Menyimpan profil demografis master seluruh pasien rumah sakit. Materialisasi: Table (Fisik).

Tabel 7.2: Kamus Data Tabel Dimensi dim_pasien

Nama Kolom	Tipe Data	Peran	Aturan Validasi	Deskripsi Fungsional
pasien_key	INT	Primary Key (PK)	unique, not_null	<i>Surrogate Key</i> numerik untuk entitas pasien.
pasien_id	VARCHAR(30)	Natural Key	not_null	Nomor rekam medis asli dari sistem operasional.
nama_pasien	VARCHAR(150)	Atribut Deskriptif	not_null	Nama lengkap pasien sesuai kartu identitas.
jenis_kelamin	VARCHAR(15)	Atribut Deskriptif	nilai 'L' atau 'P'	Jenis kelamin pasien untuk analisis demografi.
tanggal_lahir	DATE	Atribut Deskriptif	not_null	Tanggal lahir untuk kalkulasi kelompok usia.
alamat	TEXT	Atribut Deskriptif	–	Wilayah tempat tinggal pasien saat ini.

3. Tabel Dimensi: dim_dokter

Deskripsi: Menyimpan informasi profil dokter dan spesialisasi medis yang tersedia di rumah sakit. Materialisasi: Table (Fisik).



Tabel 7.3: Kamus Data Tabel Dimensi dim_dokter

Nama Kolom	Tipe Data	Peran	Aturan Validasi	Deskripsi Fungsional
dokter_key	INT	Primary Key (PK)	unique, not_null	<i>Surrogate Key</i> numerik untuk entitas dokter.
dokter_id	VARCHAR(30)	Natural Key	not_null	ID unik dokter dari sistem operasional kepegawaian.
nama_dokter	VARCHAR(150)	Atribut Deskriptif	not_null	Nama lengkap dokter beserta gelar medisnya.
spesialisasi	VARCHAR(100)	Atribut Deskriptif	not_null	Bidang keahlian spesialisasi dokter (misal: Anak, Dalam).

7.3 KONEKSI KE BUSINESS INTELLIGENCE (BI) TOOLS

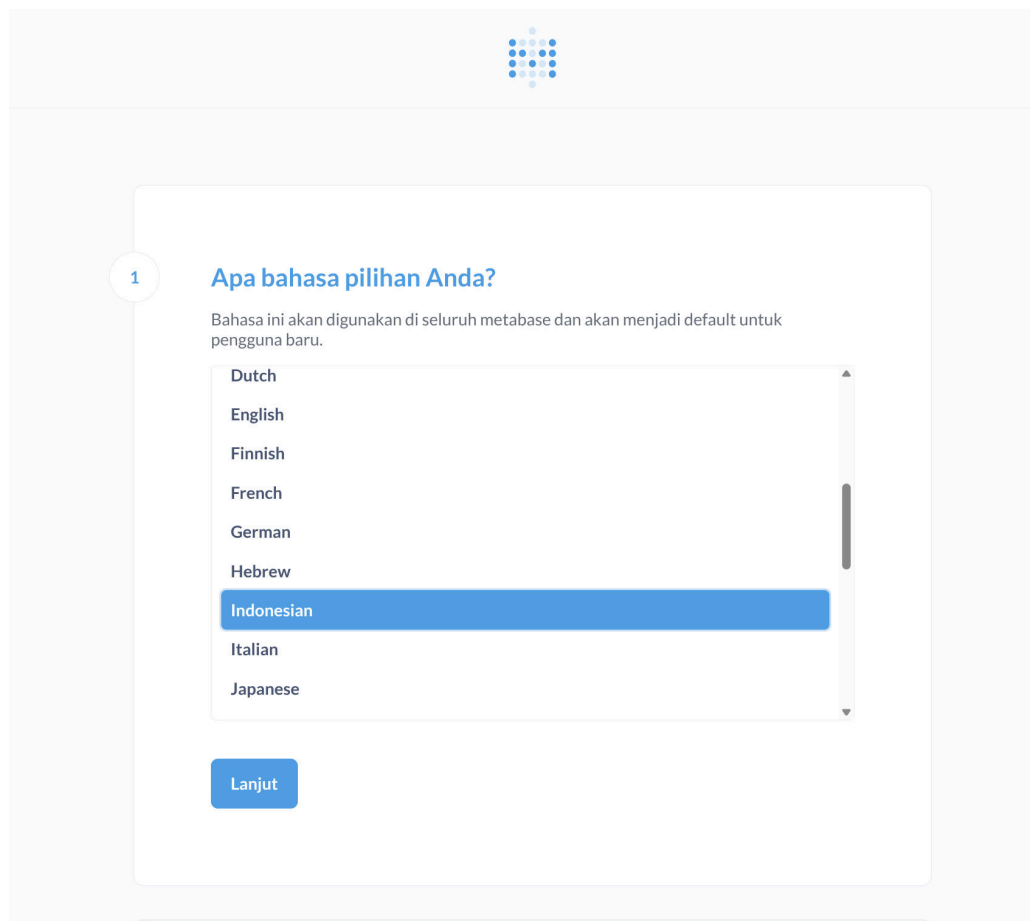
Data Mart ini menetap secara permanen di dalam database PostgreSQL pada skema `public`. Berdasarkan konfigurasi file `docker-compose.yml` yang telah dimodifikasi, `port` internal container database telah dibuka dan dipetakan menuju jaringan luar `host` lokal melalui instruksi ekspos `port` (`ports: - "5432:5432"`).

Proses integrasi antara lapisan Data Mart (PostgreSQL) dengan *Business Intelligence Tool* (Metabase) dilakukan melalui serangkaian tahapan inisialisasi parameter lingkungan (*onboarding wizard*) hingga konfigurasi jaringan relasional. Berdasarkan arsitektur Docker yang didefinisikan pada berkas `docker-compose.yml`, layanan database PostgreSQL berjalan secara internal pada `port` 5432, namun diekspos ke jaringan `host` lokal melalui pemetaan `port` eksternal 5555:5432. Oleh karena itu, pengaturan rute komunikasi pada Metabase harus disesuaikan secara presisi agar data hasil transformasi dbt dapat dibaca dengan sempurna.

Berikut adalah langkah-langkah sistematis konfigurasi koneksi dari awal hingga berhasil:

1. Inisialisasi dan Pemilihan Bahasa Utama

Saat pertama kali mengakses antarmuka Metabase via peramban web, pengguna disambut oleh menu preferensi bahasa lokalisasi. Pada tahapan ini, dipilih bahasa *Indonesian (Indonesia)* sebagai bahasa bawaan sistem (*default language*) untuk mempermudah navigasi seluruh pengguna dan jajaran manajemen di dalam ekosistem analitik rumah sakit ini.



Gambar 7.2: Halaman pemilihan bahasa pada inisialisasi awal Metabase.

2. Pembuatan Akun Administrator Perangkat

Langkah kedua adalah mengamankan sistem dengan membangun kredensial pengguna utama (*Superadmin/Administrator*). Akun ini memegang hak akses penuh terhadap manajemen database, pembuatan dasbor, serta pembagian hak akses pengguna operasional. Parameter yang didaftarkan adalah sebagai berikut:

- Nama Depan & Belakang: health care
- Alamat Email: admin@mail.com
- Perusahaan / Nama Tim: Data Engineer
- Kata Sandi: (diatur secara aman untuk pemenuhan jaminan keamanan data pasien)



✓ Bahasa Anda diatur ke Indonesian

2 Apa yang harus kami sebut?

Nama depan health Nama belakang care

Email admin@mail.com

Perusahaan atau nama tim Data Engineer

buat sebuah kata sandi

Konfirmasi password Anda

Lanjut

Gambar 7.3: Halaman pembuatan akun administrator pada inisialisasi Metabase.

3. Penentuan Orientasi Penggunaan Perangkat (Survey)

Metabase menyajikan kuesioner singkat untuk mengoptimalkan mesin visualisasi sesuai kebutuhan organisasi. Pilihan jatuh pada opsi "*Sedikit dari keduanya*", yang menandakan bahwa Metabase akan difungsikan secara hibrida: baik untuk kebutuhan analisis data mandiri (*self-service analytics* oleh tim manajemen medis) maupun untuk penyematan grafik analitik ke dalam aplikasi operasional rumah sakit yang lebih luas.



✓ Bahasa Anda diatur ke Indonesian

✓ Hai, health. Senang bertemu denganmu!

3 Untuk apa Anda menggunakan Metabase?

Beri tahu kami rencana Anda dengan Metabase sehingga kami dapat memandu Anda dengan sebaik-baiknya

☐ Analisis layanan mandiri untuk perusahaan saya sendiri

☐ Menyematkan analitik ke dalam aplikasi saya

☒ Sedikit dari keduanya

☐ Belum yakin

Lanjut

Gambar 7.4: Halaman survey orientasi penggunaan pada inisialisasi Metabase.

4. Pemilihan Jenis Database (DBMS Target)

Metabase mendukung konektivitas ke puluhan sistem database modern. Karena Data Mart proyek ini dibangun di atas infrastruktur PostgreSQL (sesuai konfigurasi `postgres:13` di Docker), maka ikon PostgreSQL dipilih sebagai tipe database tujuan utama untuk membuka *form* pemetaan skema relasional.



✓ I'll do a bit of both self-service and embedding

4 **Tambahkan data Anda**

Apakah Anda siap untuk mulai menjelajahi data Anda? Tambahkan di bawah. Belum siap? Lewati dan mainkan Basis Data Sampel kami.

Cari database ...

MySQL

PostgreSQL

SQL Server

Amazon Redshift

BigQuery

Snowflake

Tampilkan lebih banyak opsi ▾

Saya akan menambahkan data saya nanti

Gambar 7.5: Halaman pemilihan tipe database pada inisialisasi Metabase.

5. Analisis Percobaan Pertama Koneksi Jaringan (Uji Coba Port 5432)

Pada pengisian *form* awal, dilakukan uji coba pengisian parameter penghubung menggunakan *port* standar PostgreSQL internal dengan komponen data sebagai berikut:

Parameter	Nilai
Nama Tampilan	healthcare
Host	postgres
Port	5432
Database Name	airflow
Username	airflow
Password	airflow



4

Tambahkan data Anda

Apakah Anda siap untuk mulai menjelajahi data Anda? Tambahkan di bawah. Belum siap? Lewati dan mainkan Basis Data Sampel kami.

PostgreSQL

Nama tampilan
healthcare_01

Host
postgres

Port
5432

Database name
airflow

Username
airflow

Password
••••••

Schemas
All

Use a secure connection (SSL) ☐

Use an SSH-tunnel ☐

Jika koneksi langsung ke basis data Anda tidak mungkin, Anda mungkin ingin menggunakan terowongan SSH. [Belajarliah lagi.](#)

[Tampilkan opsi lanjutan](#)

Melewati [Hubungkan basis data](#)

Gambar 7.6: Parameter database PostgreSQL pada wizard Metabase.

Analisis Teknis: Batasan Port 5432

Konfigurasi *port* 5432 pada tahap ini hanya akan berhasil jika Metabase dijalankan di dalam kluster jaringan internal Docker yang sama (*same network bridge*). Jika Metabase bertindak dari luar jaringan Docker, *port* ini akan ditolak oleh sistem karena adanya restriksi isolasi kontainer.

6. Pengaturan Preferensi Pelacakan Data Anonim

Sebelum proses instalasi selesai, sistem meminta persetujuan terkait pelacakan data penggunaan performa aplikasi. Opsi "*Izinkan Metabase Untuk Mengumpulkan Acara Penggunaan Anonim*" diaktifkan guna membantu komunitas pengembang Metabase menerima laporan telemetri *bug* secara otomatis tanpa melanggar privasi data rekam medis pasien, karena data *query* tabel tetap bersifat rahasia dan terisolasi lokal.



Gambar 7.7: Pengaturan Preferensi Pelacakan Data Anonim pada wizard Metabase.

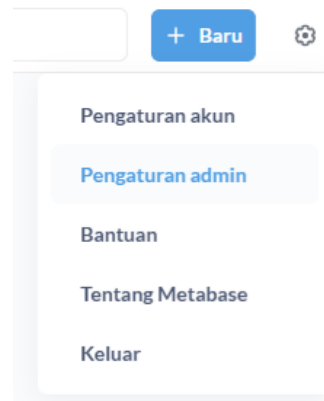
7. Penyelesaian Onboarding Awal Perangkat

Tahapan *onboarding wizard* dinyatakan berhasil ditandai dengan munculnya pesan konfirmasi "*Anda siap!*". Di halaman ini, email administrator (`admin@mail.com`) dimasukkan kembali ke dalam *form* buletin informasi untuk berlangganan pembaruan fitur keamanan penting dari Metabase.

Gambar 7.8: Penyelesaian Onboarding Awal Perangkat pada wizard Metabase.

8. Akses Masuk ke Panel Administrasi Utama (*Admin Settings*)

Setelah masuk ke halaman beranda utama Metabase, pengguna diarahkan menuju pojok kanan atas untuk membuka ikon gerigi pengaturan (*Settings*) dan masuk ke submenu "*Pengaturan admin*". Langkah ini krusial untuk melakukan perbaikan dan validasi ulang koneksi database yang sempat tertunda atau mengalami kendala rute *port* eksternal.



Gambar 7.9: Akses ke Panel Administrasi Utama pada wizard Metabase.

9. Konfigurasi Sukses Koneksi Final Data Mart (Penerapan Port Forwarding 5555)

Pada panel administrasi, di bawah menu "*Tambah Database*", dilakukan perbaikan parameter koneksi agar selaras dengan deklarasi pemetaan *port* luar pada file *docker-compose.yaml* (ports: - "5555:5432"). Spesifikasi parameter final yang dimasukkan adalah sebagai berikut:

Tabel 7.4: Parameter Koneksi Final Data Mart ke Metabase

Parameter	Nilai
Database Type	PostgreSQL
Nama Tampilan	healthcare_01
Host	host.docker.internal
Port	5555
Database Name	airflow
Username	airflow
Password	airflow
Schemas	All



The screenshot shows the Metabase Admin interface with the following configuration details:

- DATABASE TYPE:** PostgreSQL
- Nama tampilan:** healthcare_01
- Host:** host.docker.internal
- Port:** 5555
- Database name:** airflow
- Username:** airflow
- Password:** (masked with dots)
- Schemas:** All
- Use a secure connection (SSL):** ☐
- Gunakan Tunnel SSH:** ☐
- Buttons:** Sinkronisasi skema database sekarang, Periksa ulang nilai kolom ini sekarang, Zona Bahaya, Buang nilai kolom yang disimpan, Hapus database ini.
- Tindakan model:** ☐ (disabled)
- Simpan perubahan:** (button)

Gambar 7.10: Konfigurasi Koneksi Final Data Mart pada wizard Metabase.

Catatan: Port Forwarding 5555

Port wajib diubah menjadi 5555 karena Metabase mengakses PostgreSQL dari sisi *host* eksternal, memanfaatkan pintu gerbang *forwarding* yang telah disediakan Docker. Setelah tombol "Simpan" ditekan, Metabase akan melakukan *handshake protocol* ke database PostgreSQL. Status keberhasilan ditandai dengan hilangnya pesan *error* dan dimulainya proses sinkronisasi skema metadata secara otomatis (*scanning tables*).

Dengan tuntasnya langkah ke-9, Data Mart kesehatan kini telah terhubung secara sempurna dengan Metabase. Proses *query* interaktif, pembuatan visualisasi grafik kunjungan, analisis biaya medis, serta pembuatan dasbor eksekutif siap dilakukan pada tahapan berikutnya. BI Tools seperti Metabase, Apache Superset, Tableau, atau Power BI dapat langsung membaca seluruh daftar tabel *dim_** dan *fct_kunjungan* untuk mulai menyusun grafik, diagram, serta dasbor pemantauan kinerja rumah sakit.

7.4 LAPISAN SEMANTIK (SEMANTIC LAYER)

Untuk menjamin konsistensi metrik bisnis dan menghindari kesalahan penafsiran rumus oleh tim analis yang berbeda, proyek ini menerapkan konsep **Semantic Layer** (Lapisan Semantik). Logika kalkulasi bisnis dipusatkan dan dikunci di tingkat dbt, sehingga BI Tools hanya perlu memanggil metrik yang sudah matang tanpa perlu menulis ulang fungsi agregasi yang rumit.

Metrik utama yang didefinisikan secara baku pada lapisan semantik ini meliputi:



- **Metrik Finansial (Total Pendapatan Rumah Sakit):** Ditentukan secara mutlak melalui formula penjumlahan komponen biaya operasional pada tabel fakta kunjungan:

$$\text{Total Biaya} = \text{Biaya Konsultasi} + \text{Biaya Obat} + \text{Biaya Tindakan} + \text{Biaya Kamar}$$

- **Metrik Efisiensi Operasional (*Average Length of Stay* / **Avg LoS**):** Digunakan untuk mengukur efektivitas utilitas tempat tidur perawatan, dirumuskan melalui fungsi agregasi rata-rata:

$$\text{Avg LoS} = \text{AVG}(\text{durasi_rawat})$$

- **Metrik Kuantitas (Total Kunjungan Medis):** Dirumuskan melalui fungsi perhitungan unik terhadap kunci utama kunjungan:

$$\text{Total Kunjungan} = \text{COUNT}(\text{kunjungan_key})$$

Catatan: Single Source of Truth

Dengan diterapkannya *Semantic Layer* ini, seluruh laporan visual yang diproduksi oleh BI tools yang berbeda dijamin akan menyajikan angka KPI yang seragam, konsisten, valid, dan menjadi satu-satunya acuan kebenaran data (*Single Source of Truth*) bagi organisasi rumah sakit.

Part VIII

Verifikasi dan Validasi Data Mart



BAB 8

Verifikasi dan Validasi

Tahap verifikasi dan validasi bertujuan untuk memastikan bahwa sistem Data Warehouse yang telah dibangun berfungsi secara akurat, menghasilkan nilai metrik yang konsisten, serta memenuhi standar kualitas data yang dipersyaratkan. Validasi dilakukan melalui tiga mekanisme independen: eksekusi *query* analitik langsung terhadap Data Mart, pengujian integritas otomatis melalui *dbt test*, dan pemeriksaan kualitas data (*data quality check*) berbasis statistik deskriptif.

8.1 QUERY ANALITIK TERHADAP DATA MART

Lima *query* analitik berikut dieksekusi langsung pada tabel *fct_kunjungan* dan tabel-tabel dimensi terkait di database PostgreSQL untuk memverifikasi bahwa *Star Schema* yang dibangun mampu menjawab *business questions* yang telah ditetapkan pada BAB II.

8.1.1 Query 1: Total Pendapatan Berdasarkan Tipe Fasilitas (BQ-01)

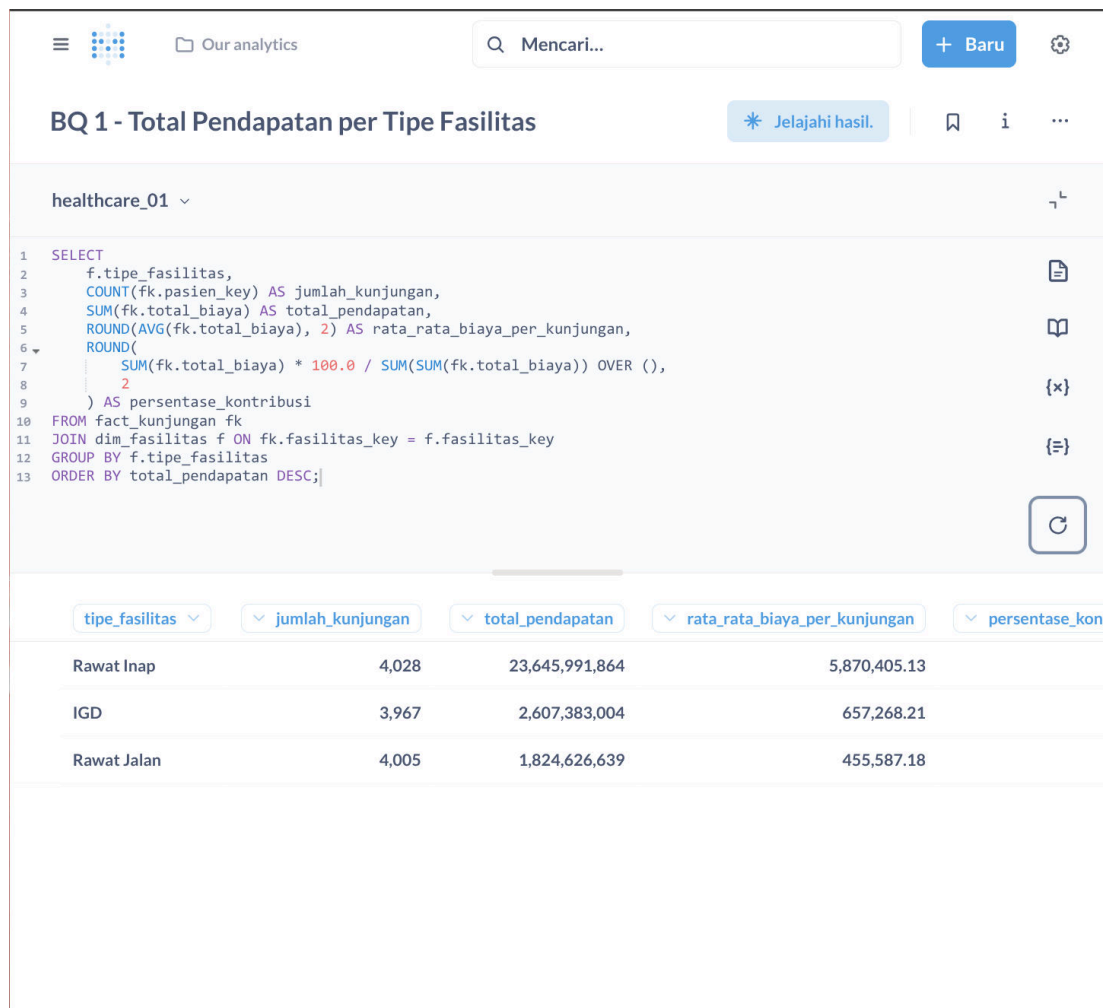
Query ini memverifikasi kemampuan sistem dalam menghitung akumulasi pendapatan operasional rumah sakit yang dibedakan berdasarkan tipe unit fasilitas pelayanan, sekaligus menguji integritas relasi *fct_kunjungan* dengan *dim_fasilitas*.

Listing 8.1: Query Analitik BQ-01: Total Pendapatan per Tipe Fasilitas

```

1 SELECT
2     f.tipe_fasilitas,
3     COUNT(fk.pasien_key) AS jumlah_kunjungan,
4     SUM(fk.total_biaya) AS total_pendapatan,
5     ROUND(AVG(fk.total_biaya), 2) AS rata_rata_biaya_per_kunjungan,
6     ROUND(
7         SUM(fk.total_biaya) * 100.0 / SUM(SUM(fk.total_biaya)) OVER (),
8         2
9     ) AS persentase_kontribusi
10 FROM fact_kunjungan fk
11 JOIN dim_fasilitas f ON fk.fasilitas_key = f.fasilitas_key
12 GROUP BY f.tipe_fasilitas
13 ORDER BY total_pendapatan DESC;

```



Gambar 8.1: Hasil eksekusi Query BQ-01: distribusi total pendapatan berdasarkan tipe fasilitas.

8.1.2 Query 2: Kinerja Dokter per Spesialisasi (BQ-02)

Query ini memverifikasi kemampuan sistem dalam menghasilkan laporan kinerja tenaga medis (dokter) berdasarkan spesialisasinya, sekaligus menguji integritas relasi antara fct_kunjungan dan dim_dokter melalui *Surrogate Key*.

Listing 8.2: Query Analitik BQ-02: Kinerja Dokter per Spesialisasi

```

1 SELECT
2   d.spesialisasi,
3   COUNT(fk.kunjungan_key) AS total_kunjungan,
4   SUM(fk.total_biaya) AS total_pendapatan,
5   ROUND(AVG(fk.biaya_konsultasi), 2) AS rata_rata_biaya_konsultasi
6  FROM fct_kunjungan fk
7  JOIN dim_dokter d ON fk.dokter_key = d.dokter_key
8  GROUP BY d.spesialisasi
9  ORDER BY total_kunjungan DESC
10 LIMIT 5;

```



Dimulai dari BQ 2 - Kinerja Dokter per Spesialisasi

Mencari...

+ Baru

Pertanyaan baru Simpan

healthcare_01

```

1 SELECT
2   d.spesialisasi,
3   COUNT(fk.pasien_key) AS jumlah_kunjungan,
4   SUM(fk.total_biaya) AS total_biaya_penagihan,
5   ROUND(AVG(fk.total_biaya), 2) AS rata_rata_biaya_per_kunjungan,
6   COUNT(DISTINCT d.dokter_key) AS jumlah_dokter,
7   ROUND(
8     COUNT(fk.pasien_key) * 1.0 / COUNT(DISTINCT d.dokter_key),
9     2
10  ) AS rata_rata_kunjungan_per_dokter
11 FROM fact_kunjungan fk
12 JOIN dim_dokter d ON fk.dokter_key = d.dokter_key
13 GROUP BY d.spesialisasi
14 ORDER BY jumlah_kunjungan DESC, total_biaya_penagihan DESC;

```

spesialisasi... jumlah_kunjung... total_biaya_penagihan rata_rata_biaya_per_kunjungan jumlah_dokter

Penyakit Dal...	2,436	5,631,729,983	2,311,876.02	10
Mata	2,248	5,246,031,955	2,333,644.11	11
Jantung	1,760	4,136,868,318	2,350,493.36	12
Anak	1,705	3,913,322,711	2,295,203.94	13
Umum	1,360	3,052,820,604	2,244,721.03	14
Bedah	1,339	3,200,916,637	2,390,527.73	15
Saraf	1,152	2,896,311,299	2,514,159.11	16

Gambar 8.2: Hasil eksekusi Query BQ-02: peringkat lima dokter dengan volume transaksi tertinggi.

8.1.3 Query 3: Distribusi Metode Pembayaran terhadap Total Biaya (BQ-03)

Query ini memverifikasi kemampuan sistem dalam menganalisis segmentasi arus kas berdasarkan metode penjaminan klaim, mencakup proporsi nilai klaim BPJS, Asuransi Swasta, dan Mandiri terhadap total pendapatan rumah sakit.

Listing 8.3: Query Analitik BQ-03: Distribusi Metode Pembayaran

```

1 SELECT
2   pb.metode_pembayaran,
3   COUNT(fk.pasien_key) AS jumlah_transaksi,
4   SUM(fk.total_biaya) AS total_biaya_pelayanan,
5   ROUND(AVG(fk.total_biaya), 2) AS rata_rata_biaya_per_transaksi,
6   ROUND(
7     COUNT(fk.pasien_key) * 100.0 / SUM(COUNT(fk.pasien_key)) OVER (),
8     2
9   ) AS persentase_transaksi,
10  ROUND(
11    SUM(fk.total_biaya) * 100.0 / SUM(SUM(fk.total_biaya)) OVER (),
12    2
13  ) AS persentase_total_biaya
14 FROM fact_kunjungan fk
15 JOIN dim_pembayaran pb ON fk.bayar_key = pb.bayar_key

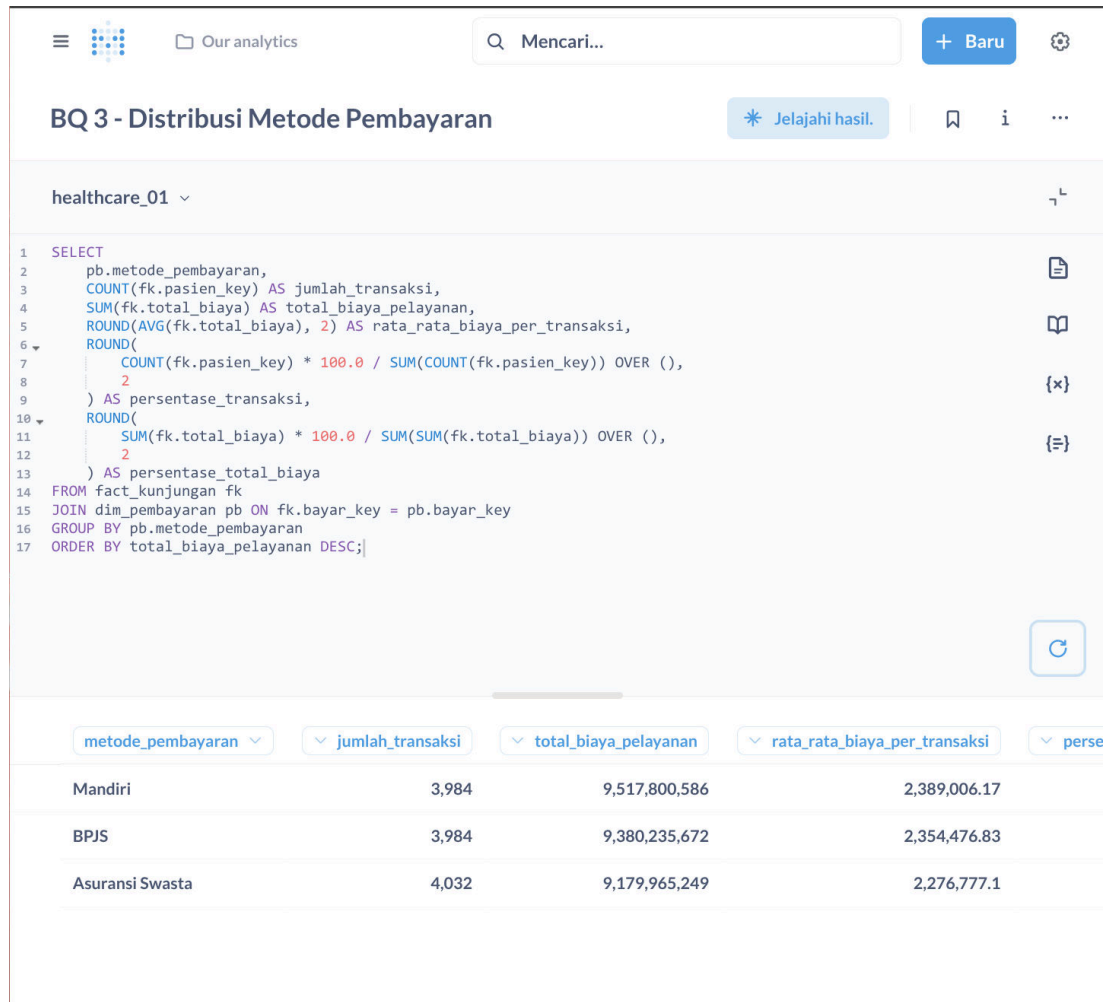
```



```

16 GROUP BY pb.metode_pembayaran
17 ORDER BY total_biaya_pelayanan DESC;

```



Gambar 8.3: Hasil eksekusi Query BQ-03: distribusi dan proporsi metode pembayaran terhadap total pendapatan.

8.1.4 Query 4: Rata-rata Length of Stay dan Pengaruhnya terhadap Biaya (BQ-04)

Query ini memverifikasi kemampuan sistem dalam menghitung metrik efisiensi klinis (*Average Length of Stay*) dan menganalisis korelasi antara durasi rawat inap dengan total biaya pelayanan.

Listing 8.4: Query Analitik BQ-04: Analisis Length of Stay dan Biaya

```

1 SELECT
2   CASE
3     WHEN fk.lama_rawat_hari = 0 THEN '0 hari (Rawat Jalan/IGD)'
4     WHEN fk.lama_rawat_hari BETWEEN 1 AND 3 THEN '1-3 hari'
5     WHEN fk.lama_rawat_hari BETWEEN 4 AND 7 THEN '4-7 hari'
6     WHEN fk.lama_rawat_hari BETWEEN 8 AND 14 THEN '8-14 hari'
7     ELSE '> 14 hari'
8   END AS kategori_los,
9   COUNT(fk.pasien_key) AS jumlah_kunjungan,
10  ROUND(AVG(fk.lama_rawat_hari), 2) AS rata_rata_lama_rawat,

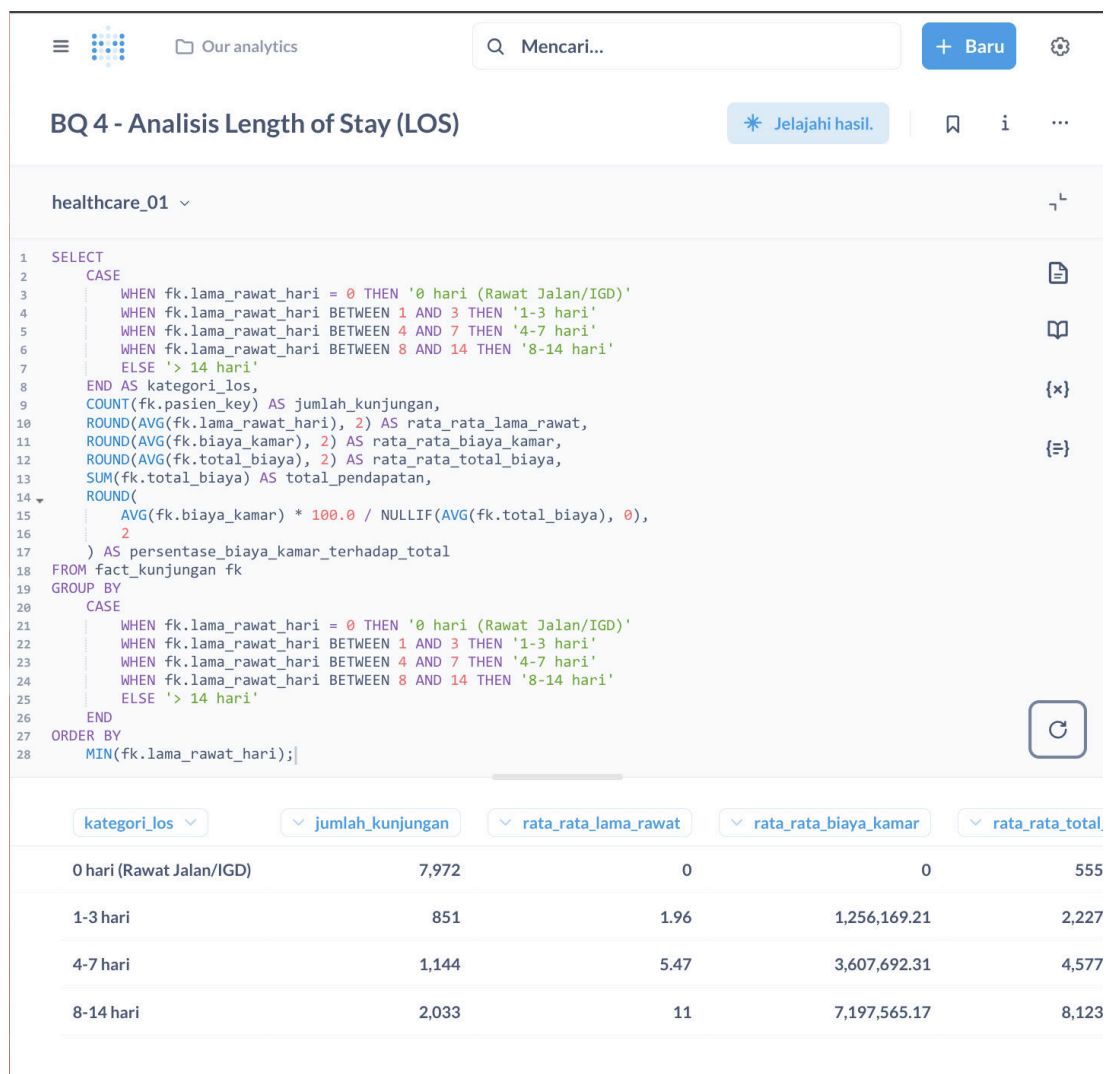
```




```

11 ROUND(AVG(fk.biaya_kamar), 2) AS rata_rata_biaya_kamar,
12 ROUND(AVG(fk.total_biaya), 2) AS rata_rata_total_biaya,
13 SUM(fk.total_biaya) AS total_pendapatan,
14 ROUND(
15     AVG(fk.biaya_kamar) * 100.0 / NULLIF(AVG(fk.total_biaya), 0),
16     2
17 ) AS persentase_biaya_kamar_terhadap_total
18 FROM fact_kunjungan fk
19 GROUP BY
20 CASE
21     WHEN fk.lama_rawat_hari = 0 THEN '0 hari (Rawat Jalan/IGD)'
22     WHEN fk.lama_rawat_hari BETWEEN 1 AND 3 THEN '1-3 hari'
23     WHEN fk.lama_rawat_hari BETWEEN 4 AND 7 THEN '4-7 hari'
24     WHEN fk.lama_rawat_hari BETWEEN 8 AND 14 THEN '8-14 hari'
25     ELSE '> 14 hari'
26 END
27 ORDER BY
28 MIN(fk.lama_rawat_hari);

```



Gambar 8.4: Hasil eksekusi Query BQ-04: rata-rata *Length of Stay* dan korelasi dengan total biaya per tipe fasilitas.



8.1.5 Query 5: Tren Bulanan Volume Kunjungan Pasien (BQ-05)

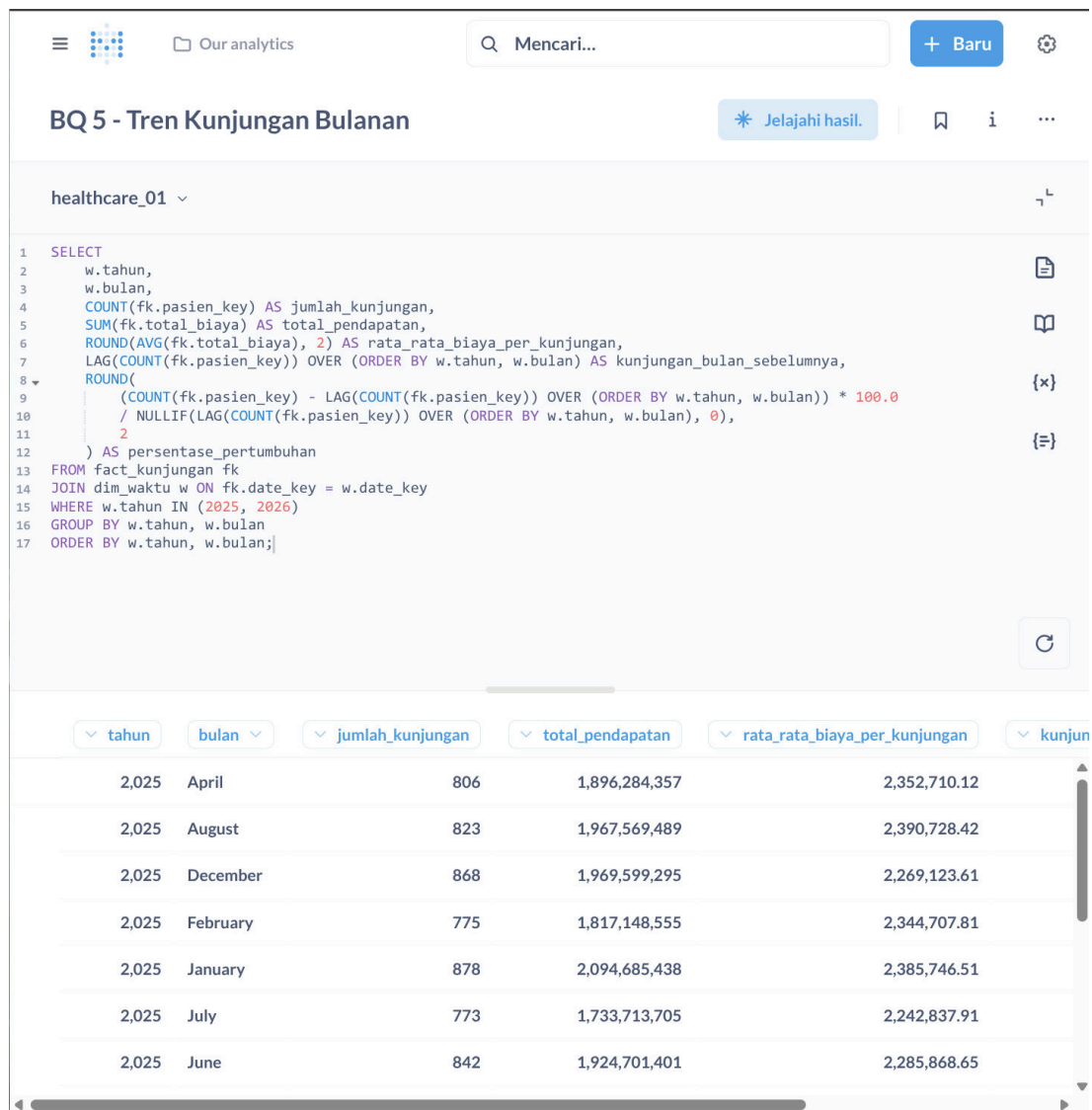
Query ini memverifikasi kemampuan sistem dalam menganalisis pola temporal (*time-series*) menggunakan dimensi waktu, sekaligus menguji integritas relasi antara fct_kunjungan dan dim_waktu yang telah dibangun melalui dbt.

Listing 8.5: Query Analitik BQ-05: Tren Bulanan Volume Kunjungan

```

1 SELECT
2     w.tahun,
3     w.bulan,
4     COUNT(fk.pasien_key) AS jumlah_kunjungan,
5     SUM(fk.total_biaya) AS total_pendapatan,
6     ROUND(AVG(fk.total_biaya), 2) AS rata_rata_biaya_per_kunjungan,
7     LAG(COUNT(fk.pasien_key)) OVER (ORDER BY w.tahun, w.bulan) AS
8     kunjungan_bulan_sebelumnya,
9     ROUND(
10         (COUNT(fk.pasien_key) - LAG(COUNT(fk.pasien_key)) OVER (ORDER BY w.
11             tahun, w.bulan)) * 100.0
12         / NULLIF(LAG(COUNT(fk.pasien_key)) OVER (ORDER BY w.tahun, w.bulan),
13             0),
14         2
15     ) AS persentase_pertumbuhan
16 FROM fact_kunjungan fk
17 JOIN dim_waktu w ON fk.date_key = w.date_key
18 WHERE w.tahun IN (2025, 2026)
19 GROUP BY w.tahun, w.bulan
20 ORDER BY w.tahun, w.bulan;

```



Gambar 8.5: Hasil eksekusi Query BQ-05: tren bulanan volume kunjungan dan pendapatan sepanjang 2025 hingga awal 2026.

8.1.6 Query 6: Proporsi Status Keluar Pasien Rawat Inap (BQ-06)

Query ini menganalisis mutu pelayanan dengan mengukur distribusi status keluar pasien (*Sembuh, Dirujuk, Pulang Paksa, Meninggal*) secara eksklusif pada fasilitas Rawat Inap, sekaligus menghitung rata-rata lama rawat dan rata-rata total biaya per kategori status keluar.

Listing 8.6: Query Analitik BQ-06: Proporsi Status Keluar Pasien Rawat Inap

```

1 SELECT
2   f.status_keluar,
3   COUNT(fk.pasien_key) AS jumlah_pasien,
4   ROUND(AVG(fk.lama_rawat_hari), 2) AS rata_rata_lama_rawat,
5   ROUND(AVG(fk.total_biaya), 2) AS rata_rata_total_biaya,
6   ROUND(
7     COUNT(fk.pasien_key) * 100.0 / SUM(COUNT(fk.pasien_key)) OVER (),
8     2
9   ) AS persentase_proporsi
10 FROM fact_kunjungan fk
11 JOIN dim_fasilitas f ON fk.fasilitas_key = f.fasilitas_key

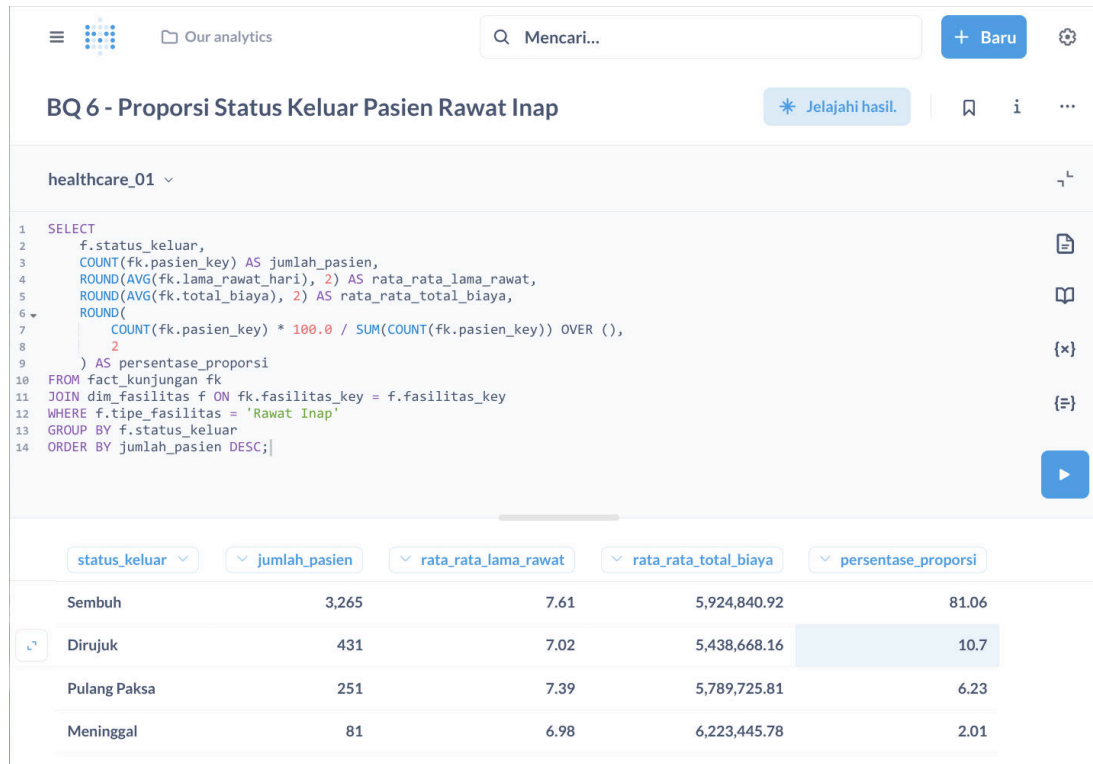
```



```

12 WHERE f.tipe_fasilitas = 'Rawat Inap'
13 GROUP BY f.status_keluar
14 ORDER BY jumlah_pasien DESC;

```



Gambar 8.6: Hasil eksekusi Query BQ-06: proporsi status keluar pasien beserta rata-rata lama rawat dan biaya pada fasilitas Rawat Inap.

Hasil pada Gambar 8.6 menampilkan distribusi empat kategori status keluar pasien Rawat Inap disertai indikator efisiensi klinis berupa rata-rata lama_rawat_hari dan rata-rata total_biaya per kategori.

8.1.7 Query 7: Breakdown Komponen Biaya Pelayanan (BQ-07)

Query ini menggunakan ekspresi WITH (*Common Table Expression*) untuk mendekomposisi struktur total penagihan menjadi empat komponen biaya: konsultasi, obat, tindakan, dan kamar. Setiap komponen dihitung kontribusi persentasenya terhadap total keseluruhan serta biaya rata-rata per transaksi.

Listing 8.7: Query Analitik BQ-07: Breakdown Kontribusi Komponen Biaya

```

1 WITH komponen_biaya AS (
2   SELECT
3     'Biaya Konsultasi' AS komponen,
4     SUM(biaya_konsultasi) AS total_biaya,
5     ROUND(AVG(biaya_konsultasi), 2) AS rata_rata_biaya
6   FROM fact_kunjungan
7
8   UNION ALL
9
10  SELECT
11    'Biaya Obat' AS komponen,
12    SUM(biaya_obat) AS total_biaya,

```



```
13      ROUND(AVG(biaya_obat), 2) AS rata_rata_biaya
14  FROM fact_kunjungan
15
16  UNION ALL
17
18  SELECT
19      'Biaya Tindakan' AS komponen,
20      SUM(biaya_tindakan) AS total_biaya,
21      ROUND(AVG(biaya_tindakan), 2) AS rata_rata_biaya
22  FROM fact_kunjungan
23
24  UNION ALL
25
26  SELECT
27      'Biaya Kamar' AS komponen,
28      SUM(biaya_kamar) AS total_biaya,
29      ROUND(AVG(biaya_kamar), 2) AS rata_rata_biaya
30  FROM fact_kunjungan
31 )
32 SELECT
33     komponen,
34     total_biaya,
35     rata_rata_biaya,
36     ROUND(
37         total_biaya * 100.0 / SUM(total_biaya) OVER (),
38         2
39     ) AS persentase_kontribusi,
40     ROUND(
41         total_biaya * 1.0 / (SELECT COUNT(*) FROM fact_kunjungan),
42         2
43     ) AS biaya_per_transaksi
44 FROM komponen_biaya
45 ORDER BY total_biaya DESC;
```



healthcare_01 Diedit beberapa detik yang lalu oleh Anda

BQ 7 - Breakdown Kontribusi Komponen Biaya [* Jelajahi hasil.](#)

```

28     SUM(biaya_kamar) AS total_biaya,
29     ROUND(AVG(biaya_kamar), 2) AS rata_rata_biaya
30 FROM fact_kunjungan
31 )
32 SELECT
33     komponen,
34     total_biaya,
35     rata_rata_biaya,
36     ROUND(
37         total_biaya * 100.0 / SUM(total_biaya) OVER (),
38         2
39     ) AS persentase_kontribusi,
40     ROUND(
41         total_biaya * 1.0 / (SELECT COUNT(*) FROM fact_kunjungan),
42         2
43     ) AS biaya_per_transaksi
44 FROM komponen_biaya
45 ORDER BY total_biaya DESC;

```

komponen	total_biaya	rata_rata_biaya	persentase_kontribusi	biaya_per_transaksi
Biaya Kamar	19,828,850,000	1,652,404.17	70.62	1,652,404.17
Biaya Tindakan	3,957,150,000	329,762.5	14.09	329,762.5
Biaya Obat	3,658,261,507	304,855.13	13.03	304,855.13
Biaya Konsultasi	633,740,000	52,811.67	2.26	52,811.67

Gambar 8.7: Hasil eksekusi Query BQ-07: kontribusi dan persentase masing-masing komponen biaya terhadap total penagihan keseluruhan.

Hasil pada Gambar 8.7 menunjukkan urutan dominasi komponen biaya berdasarkan nilai `total_biaya` dan `persentase_kontribusi`, yang mencerminkan struktur pengeluaran terbesar dalam ekosistem pelayanan medis rumah sakit.

8.1.8 Query 8: Distribusi Kunjungan Berdasarkan Kota Asal Pasien (BQ-08)

Query ini menganalisis dimensi geografis pemanfaatan layanan dengan menghitung frekuensi kunjungan dan total pendapatan per kota asal pasien. Rasio `rata_rata_kunjungan_per_pasien` dihitung untuk mengidentifikasi kota dengan tingkat kunjungan ulang (*revisit rate*) tertinggi.

Listing 8.8: Query Analitik BQ-08: Distribusi Kunjungan per Kota Asal Pasien

```

1 SELECT
2     p.kota,
3     COUNT(fk.pasien_key) AS jumlah_kunjungan,
4     COUNT(DISTINCT p.pasien_key) AS jumlah_pasien_unik,
5     SUM(fk.total_biaya) AS total_biaya_pelayanan,
6     ROUND(AVG(fk.total_biaya), 2) AS rata_rata_biaya_per_kunjungan,
7     ROUND(
8         COUNT(fk.pasien_key) * 1.0 / COUNT(DISTINCT p.pasien_key),
9         2
10    ) AS rata_rata_kunjungan_per_pasien,
11    ROUND(
12        COUNT(fk.pasien_key) * 100.0 / SUM(COUNT(fk.pasien_key)) OVER (),
13        2
14    ) AS persentase_kunjungan,
15    ROUND(
16        SUM(fk.total_biaya) * 100.0 / SUM(SUM(fk.total_biaya)) OVER (),
17        2

```



```

18 ) AS persentase_pendapatan
19 FROM fact_kunjungan fk
20 JOIN dim_pasien p ON fk.pasien_key = p.pasien_key
21 GROUP BY p.kota
22 ORDER BY jumlah_kunjungan DESC, total_biaya_pelayanan DESC
23 LIMIT 20;
    
```

healthcare_01 Our analytics Mencari... + Baru

BQ 8 - Distribusi Kunjungan Berdasarkan Kota Asal Pasien * Jelajahi hasil. i ...

healthcare_01

```

1 SELECT
2   p.kota,
3   COUNT(fk.pasien_key) AS jumlah_kunjungan,
4   COUNT(DISTINCT p.pasien_key) AS jumlah_pasien_unik,
5   SUM(fk.total_biaya) AS total_biaya_pelayanan,
6   ROUND(AVG(fk.total_biaya), 2) AS rata_rata_biaya_per_kunjungan,
7   ROUND(
8     COUNT(fk.pasien_key) * 1.0 / COUNT(DISTINCT p.pasien_key),
9     2
10  ) AS rata_rata_kunjungan_per_pasien,
11   ROUND(
12     COUNT(fk.pasien_key) * 100.0 / SUM(COUNT(fk.pasien_key)) OVER (),
13     2
14  ) AS persentase_kunjungan,
15   ROUND(
16     SUM(fk.total_biaya) * 100.0 / SUM(SUM(fk.total_biaya)) OVER (),
17     2
18  ) AS persentase_pendapatan
19 FROM fact_kunjungan fk
20 JOIN dim_pasien p ON fk.pasien_key = p.pasien_key
21 GROUP BY p.kota
22 ORDER BY jumlah_kunjungan DESC, total_biaya_pelayanan DESC
23 LIMIT 20;
    
```

kota	jumlah_kunjungan	jumlah_pasien_unik	total_biaya_pelayanan	rata_rata_biaya_per_kunjungan
Banda Aceh	181	40	450,902,524	2,491,174.17
Bengkulu	177	32	458,527,976	2,590,553.54
Bau-Bau	176	33	358,958,778	2,039,538.51
Pematangsiantar	174	33	409,995,472	2,356,295.82
Ternate	174	36	401,186,478	2,305,669.41
Palembang	174	33	341,625,305	1,963,363.82
Balikpapan	173	34	317,794,503	1,836,962.45
Tual	172	37	394,637,136	2,294,401.95
Manado	159	35	484,649,577	3,048,110.55
Lubuklinggau	156	32	380,011,199	2,435,969.22

Gambar 8.8: Hasil eksekusi Query BQ-08: 20 kota teratas berdasarkan frekuensi kunjungan dan kontribusi pendapatan.

Hasil pada Gambar 8.8 menampilkan pemeringkatan kota asal pasien berdasarkan beban pemanfaatan layanan. Kolom persentase_kunjungan dan persentase_pendapatan memungkinkan identifikasi kota yang memiliki volume kunjungan tinggi namun kontribusi pendapatan yang tidak proporsional, atau sebaliknya.



8.1.9 Query 9: Profil Demografi Kunjungan Berdasarkan Jenis Kelamin dan Golongan Darah (BQ-09)

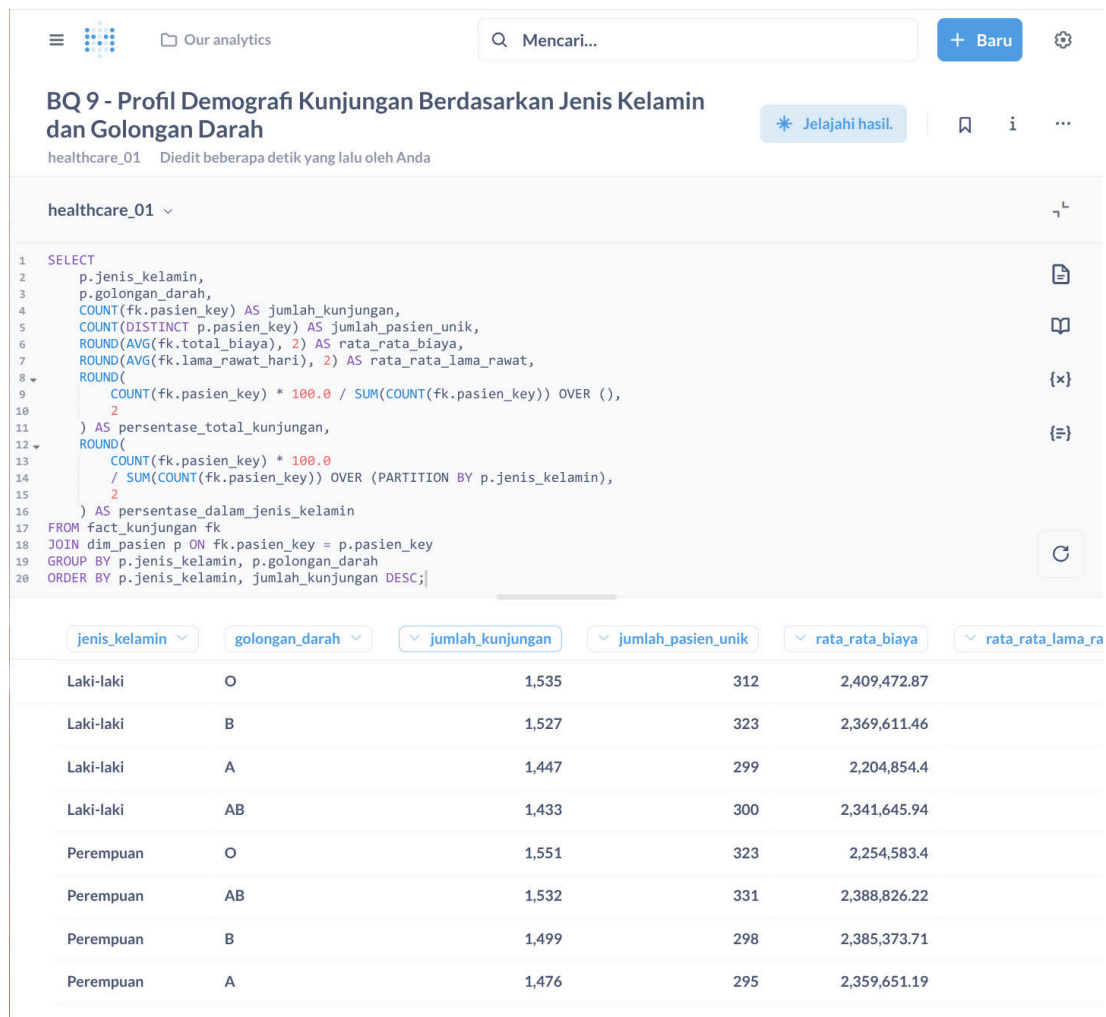
Query ini menghasilkan *cross-tabulation* kunjungan pasien berdasarkan dua variabel demografis secara bersamaan: *jenis_kelamin* dan *golongan_darah*. Penggunaan fungsi *OVER* (*PARTITION BY*) memungkinkan kalkulasi persentase berlapis, yaitu terhadap total keseluruhan maupun terhadap sub-total per jenis kelamin.

Listing 8.9: Query Analitik BQ-09: Cross-tabulation Demografi Jenis Kelamin dan Golongan Darah

```

1 SELECT
2     p.jenis_kelamin,
3     p.golongan_darah,
4     COUNT(fk.pasien_key) AS jumlah_kunjungan,
5     COUNT(DISTINCT p.pasien_key) AS jumlah_pasien_unik,
6     ROUND(AVG(fk.total_biaya), 2) AS rata_rata_biaya,
7     ROUND(AVG(fk.lama_rawat_hari), 2) AS rata_rata_lama_rawat,
8     ROUND(
9         COUNT(fk.pasien_key) * 100.0 / SUM(COUNT(fk.pasien_key)) OVER (),
10        2
11    ) AS persentase_total_kunjungan,
12    ROUND(
13        COUNT(fk.pasien_key) * 100.0
14        / SUM(COUNT(fk.pasien_key)) OVER (PARTITION BY p.jenis_kelamin),
15        2
16    ) AS persentase_dalam_jenis_kelamin
17 FROM fact_kunjungan fk
18 JOIN dim_pasien p ON fk.pasien_key = p.pasien_key
19 GROUP BY p.jenis_kelamin, p.golongan_darah
20 ORDER BY p.jenis_kelamin, jumlah_kunjungan DESC;

```

Gambar 8.9: Hasil eksekusi Query BQ-09: distribusi kunjungan berdasarkan kombinasi jenis kelamin dan golongan darah dengan persentase berlapis.

Hasil pada Gambar 8.9 menyajikan profil demografis dalam format tabular dua dimensi. Kolom persentase_dalam_jenis_kelamin menunjukkan distribusi golongan darah di dalam masing-masing segmen jenis kelamin secara independen, terlepas dari proporsi keseluruhan.

8.1.10 Query 10: Perbandingan Rata-Rata Biaya Obat IGD vs. Rawat Jalan (BQ-10)

Query ini memfokuskan analisis farmasi pada dua segmen fasilitas yang memiliki karakteristik klinis berbeda: IGD dan Rawat Jalan. Selain rata-rata, disertakan pula metrik dispersi (standar_deviasi) dan persentase kontribusi biaya obat terhadap total biaya per segmen guna menilai efisiensi penggunaan obat secara komparatif.

Listing 8.10: Query Analitik BQ-10: Komparasi Rata-Rata Biaya Obat IGD vs. Rawat Jalan

```

1 SELECT
2   f.tipe_fasilitas,
3   COUNT(fk.pasien_key) AS jumlah_kunjungan,
4   ROUND(AVG(fk.biaya_obat), 2) AS rata_rata_biaya_obat,
5   ROUND(MIN(fk.biaya_obat), 2) AS biaya_obat_minimum,
6   ROUND(MAX(fk.biaya_obat), 2) AS biaya_obat_maksimum,
7   ROUND(STDDEV(fk.biaya_obat), 2) AS standar_deviasi,
8   SUM(fk.biaya_obat) AS total_biaya_obat,

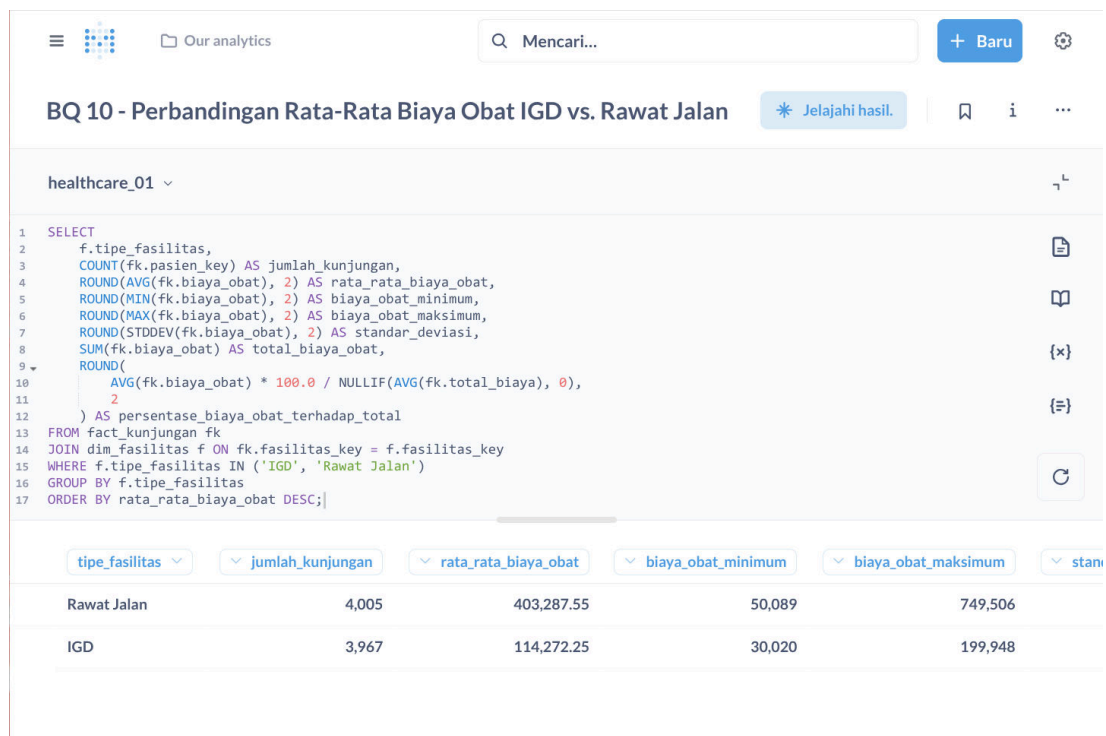
```



```

9      ROUND (
10         AVG(fk.biaya_obat) * 100.0 / NULLIF(AVG(fk.total_biaya), 0),
11         2
12     ) AS persentase_biaya_obat_terhadap_total
13 FROM fact_kunjungan fk
14 JOIN dim_fasilitas f ON fk.fasilitas_key = f.fasilitas_key
15 WHERE f.tipe_fasilitas IN ('IGD', 'Rawat Jalan')
16 GROUP BY f.tipe_fasilitas
17 ORDER BY rata_rata_biaya_obat DESC;

```



Gambar 8.10: Hasil eksekusi Query BQ-10: perbandingan statistik biaya obat antara fasilitas IGD dan Rawat Jalan.

Hasil pada Gambar 8.10 memperlihatkan perbedaan profil biaya obat antara kedua segmen fasilitas. Penggunaan NULLIF pada kolom persentase_biaya_obat_terhadap_total menjaga keamanan pembagian agar tidak menghasilkan kesalahan *division by zero* pada kasus tepi.

8.1.11 Query 11: Peringkat Lima Dokter Teraktif (BQ-11)

Query ini mengidentifikasi lima dokter dengan kontribusi pelayanan tertinggi berdasarkan jumlah transaksi kunjungan. Fungsi analitik RANK() OVER digunakan untuk menghasilkan peringkat yang bersifat deterministik dan dapat direproduksi, disertai metrik pendukung berupa total pendapatan dan rata-rata lama rawat per dokter.

Listing 8.11: Query Analitik BQ-11: Ranking Top 5 Dokter Teraktif

```

1 SELECT
2   d.nama_dokter,
3   d.spesialisasi,
4   d.nomor_izin_praktek,

```



```

5      COUNT(fk.pasien_key) AS jumlah_kunjungan,
6      SUM(fk.total_biaya) AS total_pendapatan,
7      ROUND(AVG(fk.total_biaya), 2) AS rata_rata_biaya_per_kunjungan,
8      ROUND(AVG(fk.lama_rawat_hari), 2) AS rata_rata_lama_rawat,
9      RANK() OVER (ORDER BY COUNT(fk.pasien_key) DESC) AS ranking_kunjungan
10 FROM fact_kunjungan fk
11 JOIN dim_dokter d ON fk.dokter_key = d.dokter_key
12 GROUP BY d.nama_dokter, d.spesialisasi, d.nomor_izin_praktek
13 ORDER BY jumlah_kunjungan DESC
14 LIMIT 5;

```

healthcare_01

BQ 11 - Peringkat Lima Dokter Teraktif

healthcare_01 Diedit beberapa detik yang lalu oleh Anda

* Jelajahi hasil.

healthcare_01

```

1 SELECT
2   d.nama_dokter,
3   d.spesialisasi,
4   d.nomor_izin_praktek,
5   COUNT(fk.pasien_key) AS jumlah_kunjungan,
6   SUM(fk.total_biaya) AS total_pendapatan,
7   ROUND(AVG(fk.total_biaya), 2) AS rata_rata_biaya_per_kunjungan,
8   ROUND(AVG(fk.lama_rawat_hari), 2) AS rata_rata_lama_rawat,
9   RANK() OVER (ORDER BY COUNT(fk.pasien_key) DESC) AS ranking_kunjungan
10 FROM fact_kunjungan fk
11 JOIN dim_dokter d ON fk.dokter_key = d.dokter_key
12 GROUP BY d.nama_dokter, d.spesialisasi, d.nomor_izin_praktek
13 ORDER BY jumlah_kunjungan DESC
14 LIMIT 5;

```

nama_dokter	spesialisasi	nomor_izin_praktek	jumlah_kunjungan	total_pendapatan	rata_rata_biaya_per_kunjungan
Nova Ardianto	Penyakit Dalam	STR-844321	177	402,617,940	2,274,700
Ir. Qori Sinaga	Anak	STR-668882	173	350,250,854	2,024,571
Dipa Utami, S.Farm	Mata	STR-647718	167	316,138,734	1,892,447
Paris Maheswara	Umum	STR-581682	165	381,172,702	2,309,834
Baktianto Kuswandari, S.H.	Anak	STR-394083	165	441,331,389	2,674,735

Gambar 8.11: Hasil eksekusi Query BQ-11: lima dokter teratas berdasarkan frekuensi kunjungan beserta total pendapatan yang dihasilkan.

Hasil pada Gambar 8.11 menyajikan daftar lima dokter dengan beban pelayanan tertinggi. Kolom *ranking_kunjungan* yang dihasilkan melalui *window function* bersifat konsisten dan tidak bergantung pada urutan baris hasil agregasi semata.

8.1.12 Query 12: Rata-Rata Biaya per Transaksi Kunjungan — *Average Ticket Size* (BQ-12)

Query ini menghitung *average ticket size*, yaitu rata-rata total biaya per satu transaksi kunjungan, sebagai indikator finansial utama rumah sakit. Analisis diperluas dengan *breakdown* per tipe fasilitas, metode pembayaran, dan spesialisasi dokter menggunakan klausal *UNION ALL* untuk menghasilkan satu tampilan komparatif yang terpadu.

Listing 8.12: Query Analitik BQ-12: Average Ticket Size dan Breakdown per Dimensi

```

1 -- Statistik keseluruhan

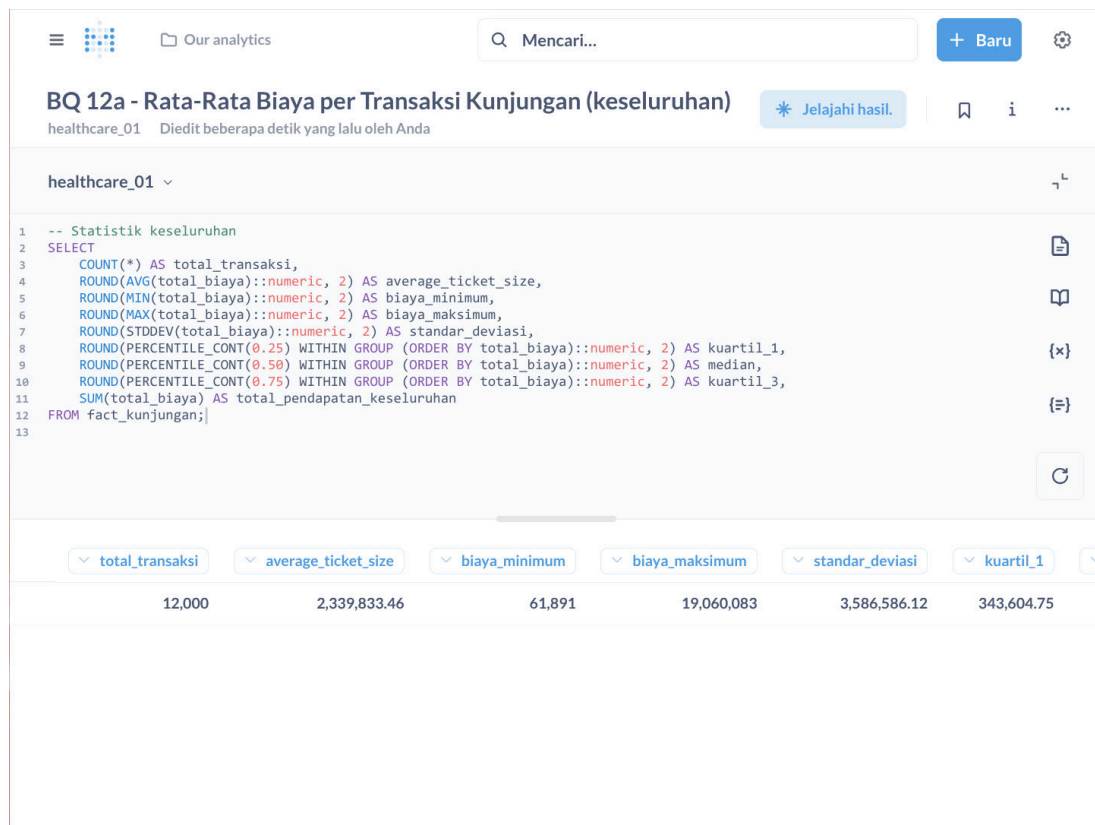
```



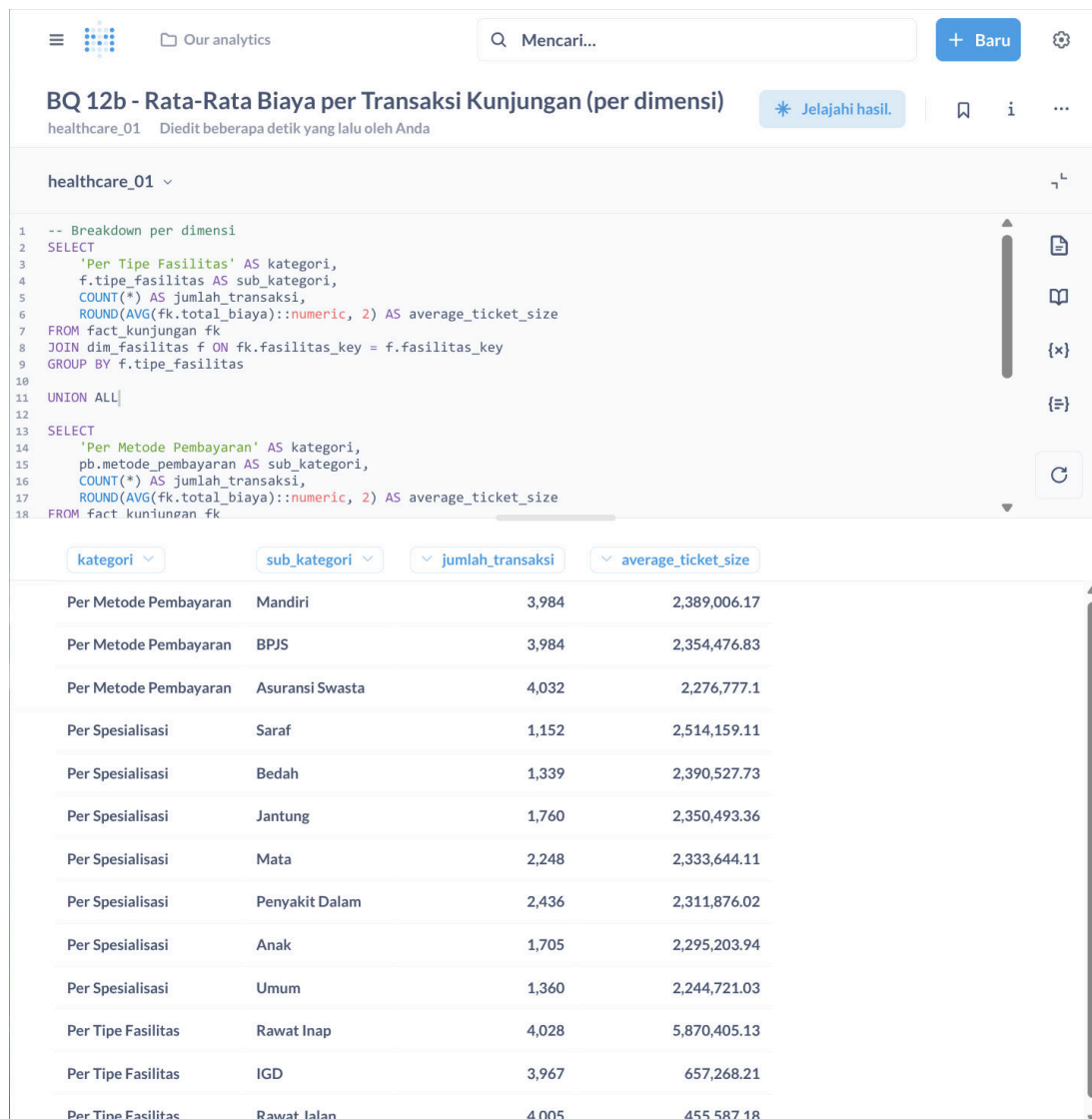
```

2 SELECT
3     COUNT(*) AS total_transaksi,
4     ROUND(AVG(total_biaya)::numeric, 2) AS average_ticket_size,
5     ROUND(MIN(total_biaya)::numeric, 2) AS biaya_minimum,
6     ROUND(MAX(total_biaya)::numeric, 2) AS biaya_maksimum,
7     ROUND(STDDEV(total_biaya)::numeric, 2) AS standar_deviasi,
8     ROUND(PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY total_biaya)::numeric
9         , 2) AS kuartil_1,
10    ROUND(PERCENTILE_CONT(0.50) WITHIN GROUP (ORDER BY total_biaya)::numeric
11        , 2) AS median,
12    ROUND(PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY total_biaya)::numeric
13        , 2) AS kuartil_3,
14    SUM(total_biaya) AS total_pendapatan_keseluruhan
15 FROM fact_kunjungan;
16
17 -- Breakdown per dimensi
18 SELECT
19     'Per Tipe Fasilitas' AS kategori,
20     f.tipe_fasilitas AS sub_kategori,
21     COUNT(*) AS jumlah_transaksi,
22     ROUND(AVG(fk.total_biaya)::numeric, 2) AS average_ticket_size
23 FROM fact_kunjungan fk
24 JOIN dim_fasilitas f ON fk.fasilitas_key = f.fasilitas_key
25 GROUP BY f.tipe_fasilitas
26
27 UNION ALL
28
29 SELECT
30     'Per Metode Pembayaran' AS kategori,
31     pb.metode_pembayaran AS sub_kategori,
32     COUNT(*) AS jumlah_transaksi,
33     ROUND(AVG(fk.total_biaya)::numeric, 2) AS average_ticket_size
34 FROM fact_kunjungan fk
35 JOIN dim_pembayaran pb ON fk.bayar_key = pb.bayar_key
36 GROUP BY pb.metode_pembayaran
37
38 UNION ALL
39
40 SELECT
41     'Per Spesialisasi' AS kategori,
42     d.spesialisasi AS sub_kategori,
43     COUNT(*) AS jumlah_transaksi,
44     ROUND(AVG(fk.total_biaya)::numeric, 2) AS average_ticket_size
45 FROM fact_kunjungan fk
46 JOIN dim_dokter d ON fk.dokter_key = d.dokter_key
47 GROUP BY d.spesialisasi
48 ORDER BY kategori, average_ticket_size DESC;

```



Gambar 8.12: Hasil eksekusi Query BQ-12 (bagian 1): statistik deskriptif *average ticket size* secara keseluruhan termasuk distribusi kuartil.



Gambar 8.13: Hasil eksekusi Query BQ-12 (bagian 2): *breakdown average ticket size* per tipe fasilitas, metode pembayaran, dan spesialisasi dokter.

Hasil pada Gambar 8.12 menyajikan distribusi biaya secara statistik melalui nilai kuartil dan median, sehingga memberikan gambaran yang lebih robust dibandingkan rata-rata tunggal. Gambar 8.13 melengkapi analisis dengan perspektif multidimensi yang memungkinkan perbandingan *average ticket size* lintas segmen fasilitas, payer, dan spesialisasi secara bersamaan dalam satu hasil *query*.

8.2 HASIL DBT TEST

Pengujian integritas data dijalankan secara otomatis oleh dbt melalui perintah `dbt test` yang telah dikonfigurasi pada *pipeline* Airflow sebagai `task dbt_data_quality_test`. Aturan pengujian didefinisikan pada berkas `schema.yml` di dalam direktori `models/marts/`.



8.2.1 Konfigurasi Pengujian (schema.yml)

Listing 8.13: Konfigurasi Pengujian dbt pada schema.yml

```

1 version: 2
2
3 models:
4   # =====
5   # STAGING LAYER
6   # =====
7   - name: stg_pasien
8     description: "Data mentah pasien dari sistem pendaftaran"
9     columns:
10      - name: id_pasien
11        tests:
12          - unique
13          - not_null
14      - name: jenis_kelamin
15        tests:
16          - accepted_values:
17            values: ['L', 'P', 'Laki-laki', 'Perempuan']
18            config:
19              severity: warn
20
21   - name: stg_dokter
22     description: "Data master dokter aktif"
23     columns:
24      - name: id_dokter
25        tests:
26          - unique
27          - not_null
28
29   - name: stg_kunjungan
30     description: "Data transaksi kunjungan dan rincian biaya medis harian"
31     columns:
32      - name: id_kunjungan
33        tests:
34          - unique
35          - not_null
36      - name: id_pasien
37        tests:
38          - not_null
39      - name: id_dokter
40        tests:
41          - not_null
42
43   # =====
44   # DIMENSION LAYER
45   # =====
46   - name: dim_pasien
47     description: "Dimensi master pasien dengan penerapan SCD Tipe 2"
48     columns:
49      - name: pasien_key
50        tests:
51          - unique
52          - not_null
53
54   - name: dim_dokter
55     description: "Dimensi master dokter dengan penerapan SCD Tipe 1"

```




```

56     columns:
57         - name: dokter_key
58           tests:
59             - unique
60             - not_null
61
62     - name: dim_waktu
63       description: "Dimensi waktu kalender berdasarkan tanggal kunjungan"
64       columns:
65         - name: date_key
66           tests:
67             - unique
68             - not_null
69
70     - name: dim_fasilitas
71       description: "Dimensi tipe fasilitas rumah sakit dan status keluar"
72       columns:
73         - name: fasilitas_key
74           tests:
75             - unique
76             - not_null
77
78     - name: dim_pembayaran
79       description: "Dimensi metode pembayaran administrasi medis"
80       columns:
81         - name: bayar_key
82           tests:
83             - unique
84             - not_null
85
86     # =====
87     # MART LAYER
88     # =====
89     - name: fact_kunjungan
90       description: "Tabel fakta utama transaksi kunjungan pelayanan medis
91         rumah sakit"
92       columns:
93         - name: pasien_key
94           tests:
95             - not_null
96             - relationships:
97               to: ref('dim_pasien')
98               field: pasien_key
99         - name: dokter_key
100          tests:
101            - not_null
102            - relationships:
103              to: ref('dim_dokter')
104              field: dokter_key
105         - name: date_key
106          tests:
107            - not_null
108            - relationships:
109              to: ref('dim_waktu')
110              field: date_key
111         - name: fasilitas_key
112          tests:

```




```

112         - not_null
113         - relationships:
114             to: ref('dim_fasilitas')
115             field: fasilitas_key
116     - name: bayar_key
117       tests:
118         - not_null
119         - relationships:
120             to: ref('dim_pembayaran')
121             field: bayar_key
122     - name: total_biaya
123       description: "Akumulasi tagihan akhir (konsultasi + obat + tindakan
124         + kamar)"
125       tests:
126         - not_null

```

8.2.2 Output Eksekusi dbt Test

Listing 8.14: Output Terminal dbt test pada Airflow Log

```

1 f11a7f975b7d
2 Log message source details
3 *** Found local files:
4 *** * /opt/airflow/logs/dag_id=01_healthcare_data_pipeline/run_id=
    manual__2026-05-26T23:02:11.298683+00:00/task_id=dbt_data_quality_test/
    attempt=1.log
5 Log group end
6 [2026-05-26, 23:02:45 UTC] {local_task_job_runner.py:123} Pre task execution
    logs
7 [2026-05-26, 23:02:46 UTC] {taskinstance.py:2614} INFO - Dependencies all
    met for dep_context=non-requeueable deps ti=<TaskInstance: 01
    _healthcare_data_pipeline.dbt_data_quality_test manual__2026-05-26T23
    :02:11.298683+00:00 [queued]>
8 [2026-05-26, 23:02:46 UTC] {taskinstance.py:2614} INFO - Dependencies all
    met for dep_context=requeueable deps ti=<TaskInstance: 01
    _healthcare_data_pipeline.dbt_data_quality_test manual__2026-05-26T23
    :02:11.298683+00:00 [queued]>
9 [2026-05-26, 23:02:46 UTC] {taskinstance.py:2867} INFO - Starting attempt 1
    of 2
10 [2026-05-26, 23:02:46 UTC] {taskinstance.py:2890} INFO - Executing <Task(
    BashOperator): dbt_data_quality_test> on 2026-05-26 23:02:11.298683+00:00
11 [2026-05-26, 23:02:46 UTC] {standard_task_runner.py:72} INFO - Started
    process 925 to run task
12 [2026-05-26, 23:02:46 UTC] {standard_task_runner.py:104} INFO - Running:
    ['***', 'tasks', 'run', '01_healthcare_data_pipeline', '
    dbt_data_quality_test', 'manual__2026-05-26T23:02:11.298683+00:00', '--
    job-id', '110', '--raw', '--subdir', 'DAGS_FOLDER/01_pipeline_healthcare.
    py', '--cfg-path', '/tmp/tmpch1mgu27']
13 [2026-05-26, 23:02:46 UTC] {standard_task_runner.py:105} INFO - Job 110:
    Subtask dbt_data_quality_test
14 [2026-05-26, 23:02:46 UTC] {task_command.py:467} INFO - Running <
    TaskInstance: 01_healthcare_data_pipeline.dbt_data_quality_test
    manual__2026-05-26T23:02:11.298683+00:00 [running]> on host f11a7f975b7d
15 [2026-05-26, 23:02:46 UTC] {taskinstance.py:3134} INFO - Exporting env vars:
    AIRFLOW_CTX_DAG_OWNER='kelompok_healthcare' AIRFLOW_CTX_DAG_ID='01
    _healthcare_data_pipeline' AIRFLOW_CTX_TASK_ID='dbt_data_quality_test'
    AIRFLOW_CTX_EXECUTION_DATE='2026-05-26T23:02:11.298683+00:00'

```



```

AIRFLOW_CTX_TRY_NUMBER='1' AIRFLOW_CTX_DAG_RUN_ID='manual__2026-05-26T23
:02:11.298683+00:00'
16 [2026-05-26, 23:02:46 UTC] {logging_mixin.py:190} INFO - Task instance is in
    running state
17 [2026-05-26, 23:02:46 UTC] {logging_mixin.py:190} INFO - Previous state of
    the Task instance: queued
18 [2026-05-26, 23:02:46 UTC] {logging_mixin.py:190} INFO - Current task name:
    dbt_data_quality_test state:running start_date:2026-05-26
    23:02:46.004078+00:00
19 [2026-05-26, 23:02:46 UTC] {logging_mixin.py:190} INFO - Dag name:01
    _healthcare_data_pipeline and current dag run status:running
20 [2026-05-26, 23:02:46 UTC] {taskinstance.py:732} Log group end
21 [2026-05-26, 23:02:46 UTC] {subprocess.py:78} INFO - Tmp dir root location:
    /tmp
22 [2026-05-26, 23:02:46 UTC] {subprocess.py:88} INFO - Running command: ['/usr
    /bin/bash', '-c', 'cd /opt/***/dbt_project && dbt test --target
    docker_env --profiles-dir .']
23 [2026-05-26, 23:02:46 UTC] {subprocess.py:99} INFO - Output:
24 [2026-05-26, 23:02:50 UTC] {subprocess.py:106} INFO - 23:02:50 Running with
    dbt=1.8.0
25 [2026-05-26, 23:02:51 UTC] {subprocess.py:106} INFO - 23:02:51 Registered
    adapter: postgres=1.8.0
26 [2026-05-26, 23:02:52 UTC] {subprocess.py:106} INFO - 23:02:52 Found 9
    models, 30 data tests, 3 sources, 428 macros
27 [2026-05-26, 23:02:52 UTC] {subprocess.py:106} INFO - 23:02:52
28 [2026-05-26, 23:02:52 UTC] {subprocess.py:106} INFO - 23:02:52 Concurrency:
    4 threads (target='docker_env')
29 [2026-05-26, 23:02:52 UTC] {subprocess.py:106} INFO - 23:02:52
30 [2026-05-26, 23:02:52 UTC] {subprocess.py:106} INFO - 23:02:52 1 of 30
    START test
    accepted_values_stg_pasien_jenis_kelamin__L__P__Laki_laki__Perempuan [
    RUN]
31 [2026-05-26, 23:02:52 UTC] {subprocess.py:106} INFO - 23:02:52 2 of 30
    START test not_null_dim_dokter_dokter_key .....
    [RUN]
32 [2026-05-26, 23:02:52 UTC] {subprocess.py:106} INFO - 23:02:52 3 of 30
    START test not_null_dim_fasilitas_fasilitas_key .....
    [RUN]
33 [2026-05-26, 23:02:52 UTC] {subprocess.py:106} INFO - 23:02:52 4 of 30
    START test not_null_dim_pasien_pasien_key .....
    [RUN]
34 [2026-05-26, 23:02:52 UTC] {subprocess.py:106} INFO - 23:02:52 3 of 30 PASS
    not_null_dim_fasilitas_fasilitas_key ..... [
    PASS in 0.46s]
35 [2026-05-26, 23:02:52 UTC] {subprocess.py:106} INFO - 23:02:52 2 of 30 PASS
    not_null_dim_dokter_dokter_key ..... [
    PASS in 0.48s]
36 [2026-05-26, 23:02:53 UTC] {subprocess.py:106} INFO - 23:02:52 4 of 30 PASS
    not_null_dim_pasien_pasien_key ..... [
    PASS in 0.49s]
37 [2026-05-26, 23:02:53 UTC] {subprocess.py:106} INFO - 23:02:53 5 of 30
    START test not_null_dim_pembayaran_bayar_key .....
    [RUN]
38 [2026-05-26, 23:02:53 UTC] {subprocess.py:106} INFO - 23:02:53 1 of 30 PASS
    accepted_values_stg_pasien_jenis_kelamin__L__P__Laki_laki__Perempuan [
    PASS in 0.50s]

```



```

39 [2026-05-26, 23:02:53 UTC] {subprocess.py:106} INFO - 23:02:53 6 of 30
    START test not_null_dim_waktu_date_key .....
    [RUN]
40 [2026-05-26, 23:02:53 UTC] {subprocess.py:106} INFO - 23:02:53 7 of 30
    START test not_null_fact_kunjungan_bayar_key .....
    [RUN]
41 [2026-05-26, 23:02:53 UTC] {subprocess.py:106} INFO - 23:02:53 8 of 30
    START test not_null_fact_kunjungan_date_key .....
    [RUN]
42 [2026-05-26, 23:02:53 UTC] {subprocess.py:106} INFO - 23:02:53 5 of 30 PASS
    not_null_dim_pembayaran_bayar_key ..... [
    PASS in 0.17s]
43 [2026-05-26, 23:02:53 UTC] {subprocess.py:106} INFO - 23:02:53 6 of 30 PASS
    not_null_dim_waktu_date_key ..... [
    PASS in 0.16s]
44 [2026-05-26, 23:02:53 UTC] {subprocess.py:106} INFO - 23:02:53 9 of 30
    START test not_null_fact_kunjungan_dokter_key .....
    [RUN]
45 [2026-05-26, 23:02:53 UTC] {subprocess.py:106} INFO - 23:02:53 8 of 30 PASS
    not_null_fact_kunjungan_date_key ..... [
    PASS in 0.16s]
46 [2026-05-26, 23:02:53 UTC] {subprocess.py:106} INFO - 23:02:53 7 of 30 PASS
    not_null_fact_kunjungan_bayar_key ..... [
    PASS in 0.17s]
47 [2026-05-26, 23:02:53 UTC] {subprocess.py:106} INFO - 23:02:53 10 of 30
    START test not_null_fact_kunjungan_fasilitas_key ..... [
    RUN]
48 [2026-05-26, 23:02:53 UTC] {subprocess.py:106} INFO - 23:02:53 11 of 30
    START test not_null_fact_kunjungan_pasien_key ..... [
    RUN]
49 [2026-05-26, 23:02:53 UTC] {subprocess.py:106} INFO - 23:02:53 12 of 30
    START test not_null_fact_kunjungan_total_biaya ..... [
    RUN]
50 [2026-05-26, 23:02:53 UTC] {subprocess.py:106} INFO - 23:02:53 9 of 30 PASS
    not_null_fact_kunjungan_dokter_key ..... [
    PASS in 0.19s]
51 [2026-05-26, 23:02:53 UTC] {subprocess.py:106} INFO - 23:02:53 10 of 30
    PASS not_null_fact_kunjungan_fasilitas_key ..... [
    PASS in 0.17s]
52 [2026-05-26, 23:02:53 UTC] {subprocess.py:106} INFO - 23:02:53 13 of 30
    START test not_null_stg_dokter_id_dokter ..... [
    RUN]
53 [2026-05-26, 23:02:53 UTC] {subprocess.py:106} INFO - 23:02:53 11 of 30
    PASS not_null_fact_kunjungan_pasien_key ..... [
    PASS in 0.15s]
54 [2026-05-26, 23:02:53 UTC] {subprocess.py:106} INFO - 23:02:53 14 of 30
    START test not_null_stg_kunjungan_id_dokter ..... [
    RUN]
55 [2026-05-26, 23:02:53 UTC] {subprocess.py:106} INFO - 23:02:53 12 of 30
    PASS not_null_fact_kunjungan_total_biaya ..... [
    PASS in 0.16s]
56 [2026-05-26, 23:02:53 UTC] {subprocess.py:106} INFO - 23:02:53 15 of 30
    START test not_null_stg_kunjungan_id_kunjungan ..... [
    RUN]
57 [2026-05-26, 23:02:53 UTC] {subprocess.py:106} INFO - 23:02:53 16 of 30
    START test not_null_stg_kunjungan_id_pasien ..... [
    RUN]

```



```

58 [2026-05-26, 23:02:53 UTC] {subprocess.py:106} INFO - 23:02:53 13 of 30
    PASS not_null_stg_dokter_id_dokter ..... [
    PASS in 0.14s]
59 [2026-05-26, 23:02:53 UTC] {subprocess.py:106} INFO - 23:02:53 17 of 30
    START test not_null_stg_pasien_id_pasien ..... [
    RUN]
60 [2026-05-26, 23:02:53 UTC] {subprocess.py:106} INFO - 23:02:53 15 of 30
    PASS not_null_stg_kunjungan_id_kunjungan ..... [
    PASS in 0.13s]
61 [2026-05-26, 23:02:53 UTC] {subprocess.py:106} INFO - 23:02:53 14 of 30
    PASS not_null_stg_kunjungan_id_dokter ..... [
    PASS in 0.17s]
62 [2026-05-26, 23:02:53 UTC] {subprocess.py:106} INFO - 23:02:53 18 of 30
    START test
    relationships_fact_kunjungan_bayar_key__bayar_key__ref_dim_pembayaran_ [
    RUN]
63 [2026-05-26, 23:02:53 UTC] {subprocess.py:106} INFO - 23:02:53 16 of 30
    PASS not_null_stg_kunjungan_id_pasien ..... [
    PASS in 0.16s]
64 [2026-05-26, 23:02:53 UTC] {subprocess.py:106} INFO - 23:02:53 19 of 30
    START test
    relationships_fact_kunjungan_date_key__date_key__ref_dim_waktu_ [RUN]
65 [2026-05-26, 23:02:53 UTC] {subprocess.py:106} INFO - 23:02:53 20 of 30
    START test
    relationships_fact_kunjungan_dokter_key__dokter_key__ref_dim_dokter_ [
    RUN]
66 [2026-05-26, 23:02:53 UTC] {subprocess.py:106} INFO - 23:02:53 17 of 30
    PASS not_null_stg_pasien_id_pasien ..... [
    PASS in 0.18s]
67 [2026-05-26, 23:02:53 UTC] {subprocess.py:106} INFO - 23:02:53 21 of 30
    START test
    relationships_fact_kunjungan_fasilitas_key__fasilitas_key__ref_dim_fasilitas_
    [RUN]
68 [2026-05-26, 23:02:53 UTC] {subprocess.py:106} INFO - 23:02:53 18 of 30
    PASS
    relationships_fact_kunjungan_bayar_key__bayar_key__ref_dim_pembayaran_ [
    PASS in 0.20s]
69 [2026-05-26, 23:02:53 UTC] {subprocess.py:106} INFO - 23:02:53 19 of 30
    PASS relationships_fact_kunjungan_date_key__date_key__ref_dim_waktu_ .. [
    PASS in 0.20s]
70 [2026-05-26, 23:02:53 UTC] {subprocess.py:106} INFO - 23:02:53 22 of 30
    START test
    relationships_fact_kunjungan_pasien_key__pasien_key__ref_dim_pasien_ [
    RUN]
71 [2026-05-26, 23:02:53 UTC] {subprocess.py:106} INFO - 23:02:53 23 of 30
    START test unique_dim_dokter_dokter_key ..... [
    RUN]
72 [2026-05-26, 23:02:53 UTC] {subprocess.py:106} INFO - 23:02:53 20 of 30
    PASS relationships_fact_kunjungan_dokter_key__dokter_key__ref_dim_dokter_
    [PASS in 0.17s]
73 [2026-05-26, 23:02:53 UTC] {subprocess.py:106} INFO - 23:02:53 24 of 30
    START test unique_dim_fasilitas_fasilitas_key ..... [
    RUN]
74 [2026-05-26, 23:02:53 UTC] {subprocess.py:106} INFO - 23:02:53 21 of 30
    PASS
    relationships_fact_kunjungan_fasilitas_key__fasilitas_key__ref_dim_fasilitas_
    [PASS in 0.21s]

```



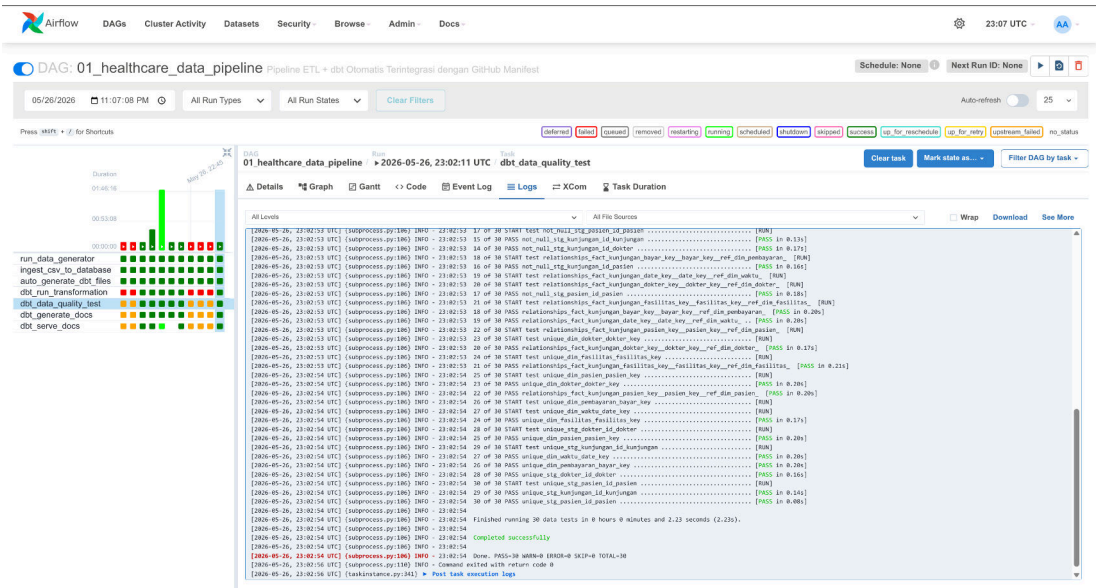
```

75 [2026-05-26, 23:02:54 UTC] {subprocess.py:106} INFO - 23:02:54 25 of 30
    START test unique_dim_pasien_pasien_key ..... [
    RUN]
76 [2026-05-26, 23:02:54 UTC] {subprocess.py:106} INFO - 23:02:54 23 of 30
    PASS unique_dim_dokter_dokter_key ..... [
    PASS in 0.20s]
77 [2026-05-26, 23:02:54 UTC] {subprocess.py:106} INFO - 23:02:54 22 of 30
    PASS relationships_fact_kunjungan_pasien_key__pasien_key__ref_dim_pasien_
    [PASS in 0.20s]
78 [2026-05-26, 23:02:54 UTC] {subprocess.py:106} INFO - 23:02:54 26 of 30
    START test unique_dim_pembayaran_bayar_key ..... [
    RUN]
79 [2026-05-26, 23:02:54 UTC] {subprocess.py:106} INFO - 23:02:54 27 of 30
    START test unique_dim_waktu_date_key ..... [
    RUN]
80 [2026-05-26, 23:02:54 UTC] {subprocess.py:106} INFO - 23:02:54 24 of 30
    PASS unique_dim_fasilitas_fasilitas_key ..... [
    PASS in 0.17s]
81 [2026-05-26, 23:02:54 UTC] {subprocess.py:106} INFO - 23:02:54 28 of 30
    START test unique_stg_dokter_id_dokter ..... [
    RUN]
82 [2026-05-26, 23:02:54 UTC] {subprocess.py:106} INFO - 23:02:54 25 of 30
    PASS unique_dim_pasien_pasien_key ..... [
    PASS in 0.20s]
83 [2026-05-26, 23:02:54 UTC] {subprocess.py:106} INFO - 23:02:54 29 of 30
    START test unique_stg_kunjungan_id_kunjungan ..... [
    RUN]
84 [2026-05-26, 23:02:54 UTC] {subprocess.py:106} INFO - 23:02:54 27 of 30
    PASS unique_dim_waktu_date_key ..... [
    PASS in 0.20s]
85 [2026-05-26, 23:02:54 UTC] {subprocess.py:106} INFO - 23:02:54 26 of 30
    PASS unique_dim_pembayaran_bayar_key ..... [
    PASS in 0.20s]
86 [2026-05-26, 23:02:54 UTC] {subprocess.py:106} INFO - 23:02:54 28 of 30
    PASS unique_stg_dokter_id_dokter ..... [
    PASS in 0.16s]
87 [2026-05-26, 23:02:54 UTC] {subprocess.py:106} INFO - 23:02:54 30 of 30
    START test unique_stg_pasien_id_pasien ..... [
    RUN]
88 [2026-05-26, 23:02:54 UTC] {subprocess.py:106} INFO - 23:02:54 29 of 30
    PASS unique_stg_kunjungan_id_kunjungan ..... [
    PASS in 0.14s]
89 [2026-05-26, 23:02:54 UTC] {subprocess.py:106} INFO - 23:02:54 30 of 30
    PASS unique_stg_pasien_id_pasien ..... [
    PASS in 0.08s]
90 [2026-05-26, 23:02:54 UTC] {subprocess.py:106} INFO - 23:02:54
91 [2026-05-26, 23:02:54 UTC] {subprocess.py:106} INFO - 23:02:54 Finished
    running 30 data tests in 0 hours 0 minutes and 2.23 seconds (2.23s).
92 [2026-05-26, 23:02:54 UTC] {subprocess.py:106} INFO - 23:02:54
93 [2026-05-26, 23:02:54 UTC] {subprocess.py:106} INFO - 23:02:54 Completed
    successfully
94 [2026-05-26, 23:02:54 UTC] {subprocess.py:106} INFO - 23:02:54
95 [2026-05-26, 23:02:54 UTC] {subprocess.py:106} INFO - 23:02:54 Done. PASS
    =30 WARN=0 ERROR=0 SKIP=0 TOTAL=30
96 [2026-05-26, 23:02:56 UTC] {subprocess.py:110} INFO - Command exited with
    return code 0
97 [2026-05-26, 23:02:56 UTC] {taskinstance.py:341} Post task execution logs

```




```
98 [2026-05-26, 23:02:56 UTC] {taskinstance.py:353} INFO - Marking task as
    SUCCESS. dag_id=01_healthcare_data_pipeline, task_id=
    dbt_data_quality_test, run_id=manual__2026-05-26T23:02:11.298683+00:00,
    execution_date=20260526T230211, start_date=20260526T230246, end_date
    =20260526T230256
99 [2026-05-26, 23:02:56 UTC] {logging_mixin.py:190} INFO - Task instance in
    success state
100 [2026-05-26, 23:02:56 UTC] {logging_mixin.py:190} INFO - Previous state of
    the Task instance: running
101 [2026-05-26, 23:02:56 UTC] {logging_mixin.py:190} INFO - Dag name:01
    _healthcare_data_pipeline queued_at:2026-05-26 23:02:11.327277+00:00
102 [2026-05-26, 23:02:56 UTC] {logging_mixin.py:190} INFO - Task hostname:
    f11a7f975b7d operator:BashOperator
103 [2026-05-26, 23:02:56 UTC] {local_task_job_runner.py:266} INFO - Task exited
    with return code 0
104 [2026-05-26, 23:02:56 UTC] {taskinstance.py:3901} INFO - 1 downstream tasks
    scheduled from follow-on schedule check
105 [2026-05-26, 23:02:56 UTC] {local_task_job_runner.py:245} Log group end
```



Gambar 8.14: Tampilan log Airflow task dbt_data_quality_test: 30/30 pengujian lulus (PASS).

Tabel 8.1: Metadata Eksekusi dbt test (2026-05-26, 23:02:50 UTC)

Parameter	Nilai
Versi dbt	dbt=1.8.0, adapter: postgres=1.8.0
Target	docker_env
Konkurensi	4 threads paralel
Cakupan	9 model, 30 data tests, 3 sumber, 428 makro
Total Waktu	2,23 detik
Hasil Akhir	PASS=30 WARN=0 ERROR=0 SKIP=0 TOTAL=30



Tabel 8.2: Hasil dbt test – Lapisan Staging (9 dari 30 pengujian)

No	Tipe Pengujian	Model	Kolom Target	Status	Durasi
1	accepted_values	stg_pasien	jenis_kelamin (nilai: L, P, Laki-laki, Perempuan)	PASS	0,50 s
2	not_null	stg_dokter	id_dokter	PASS	0,14 s
3	not_null	stg_kunjungan	id_dokter	PASS	0,17 s
4	not_null	stg_kunjungan	id_kunjungan	PASS	0,13 s
5	not_null	stg_kunjungan	id_pasien	PASS	0,16 s
6	not_null	stg_pasien	id_pasien	PASS	0,18 s
7	unique	stg_dokter	id_dokter	PASS	0,16 s
8	unique	stg_kunjungan	id_kunjungan	PASS	0,14 s
9	unique	stg_pasien	id_pasien	PASS	0,08 s

Tabel 8.3: Hasil dbt test – Lapisan Dimensi (10 dari 30 pengujian)

No	Tipe Pengujian	Model	Kolom Target	Status	Durasi
10	not_null	dim_dokter	dokter_key	PASS	0,48 s
11	not_null	dim_fasilitas	fasilitas_key	PASS	0,46 s
12	not_null	dim_pasien	pasien_key	PASS	0,49 s
13	not_null	dim_pembayaran	bayar_key	PASS	0,17 s
14	not_null	dim_waktu	date_key	PASS	0,16 s
15	unique	dim_dokter	dokter_key	PASS	0,20 s
16	unique	dim_fasilitas	fasilitas_key	PASS	0,17 s
17	unique	dim_pasien	pasien_key	PASS	0,20 s
18	unique	dim_pembayaran	bayar_key	PASS	0,20 s
19	unique	dim_waktu	date_key	PASS	0,20 s

Tabel 8.4: Hasil dbt test – Lapisan Fakta fact_kunjungan (11 dari 30 pengujian)

No	Tipe Pengujian	Model	Kolom / Relasi Target	Status	Durasi
20	not_null	fact_kunjungan	bayar_key	PASS	0,17 s
21	not_null	fact_kunjungan	date_key	PASS	0,16 s
22	not_null	fact_kunjungan	dokter_key	PASS	0,19 s
23	not_null	fact_kunjungan	fasilitas_key	PASS	0,17 s
24	not_null	fact_kunjungan	pasien_key	PASS	0,15 s
25	not_null	fact_kunjungan	total_biaya	PASS	0,16 s
26	relationships	fact_kunjungan	bayar_key → dim_pembayaran.bayar_key	PASS	0,20 s
27	relationships	fact_kunjungan	date_key → dim_waktu.date_key	PASS	0,20 s
28	relationships	fact_kunjungan	dokter_key → dim_dokter.dokter_key	PASS	0,17 s
29	relationships	fact_kunjungan	fasilitas_key → dim_fasilitas.fasilitas_key	PASS	0,21 s
30	relationships	fact_kunjungan	pasien_key → dim_pasien.pasien_key	PASS	0,20 s

Seluruh 30 aturan pengujian yang mencakup accepted_values (1), not_null (17), unique (8), dan relationships (4) berhasil dieksekusi tanpa satu pun kegagalan. Pengujian dijalankan secara paralel menggunakan 4 threads pada target docker_env dan selesai dalam 2,23 detik. Hal ini membuktikan bahwa seluruh lapisan *Star Schema* – dari *staging* hingga *fact table* – bebas dari anomali data dan memiliki integritas referensial yang solid.



8.3 PEMERIKSAAN KUALITAS DATA (DATA QUALITY CHECK)

Pemeriksaan kualitas data dilakukan secara mandiri di luar ekosistem dbt untuk memvalidasi karakteristik statistik dari data yang tersimpan di Data Mart. Pemeriksaan ini menggunakan kueri statistik deskriptif langsung pada PostgreSQL dan divisualisasikan melalui panel inspeksi tabel Metabase.

8.3.1 Metodologi Pemeriksaan

Pendekatan yang digunakan bersifat *out-of-framework*, artinya validasi dijalankan langsung terhadap tabel final di layer Mart tanpa bergantung pada mekanisme dbt `test`. Hal ini bertujuan untuk:

- Memverifikasi integritas data secara independen sebagai lapisan validasi kedua.
- Mendeteksi anomali statistik yang tidak tercakup oleh skema YAML dbt, seperti distribusi nilai, keberadaan *outlier*, dan konsistensi kategori.
- Menyediakan bukti audit yang dapat direproduksi melalui kueri SQL yang terdokumentasi.

8.3.2 Dimensi Pemeriksaan

Pemeriksaan mencakup empat dimensi kualitas data utama yang dijabarkan pada Tabel 8.5.

Tabel 8.5: Dimensi Pemeriksaan Kualitas Data

No	Dimensi	Target Tabel	Kriteria Kelulusan
1	Kelengkapan (Completeness)	fct_kunjungan, seluruh dimensi	Tidak ada nilai NULL pada kolom kunci dan kolom <i>measures</i>
2	Kevalidan (Validity)	fct_kunjungan	Seluruh nilai <code>total_biaya</code> ≥ 0
3	Konsistensi (Consistency)	dim_fasilitas	Nilai <code>tipe_fasilitas</code> hanya terdiri dari kategori yang ditetapkan
4	Volume (Row Count)	fct_kunjungan	Jumlah baris sesuai target rencana pemuatan data

8.3.3 Kueri Validasi PostgreSQL

Pemeriksaan Kelengkapan — Nilai Null

Kueri berikut menghitung jumlah baris dengan nilai NULL pada seluruh kolom kunci dan kolom *measure* di `fct_kunjungan`.

Listing 8.15: Pemeriksaan nilai null pada fct_kunjungan

```
1 SELECT
2     COUNT(*) AS total_rows,
3     COUNT(*) FILTER (WHERE pasien_key IS NULL) AS null_pasien_key,
4     COUNT(*) FILTER (WHERE dokter_key IS NULL) AS null_dokter_key,
5     COUNT(*) FILTER (WHERE date_key IS NULL) AS null_date_key,
6     COUNT(*) FILTER (WHERE fasilitas_key IS NULL) AS null_fasilitas_key,
7     COUNT(*) FILTER (WHERE bayar_key IS NULL) AS null_bayar_key,
8     COUNT(*) FILTER (WHERE total_biaya IS NULL) AS null_total_biaya
9 FROM airflow.public.fact_kunjungan;
```




Data Output Messages Notifications									
	total_rows bigint	null_pasien_key bigint	null_dokter_key bigint	null_date_key bigint	null_fasilitas_key bigint	null_bayar_key bigint	null_total_biaya bigint	Showing rows: 1 to 1	
1	12000	0	0	0	0	0	0		

Total rows: 1 Query complete 00:00:00.151

Gambar 8.15: Hasil kueri pemeriksaan nilai *null* pada *fact_kunjungan*

Hasil pada Gambar 8.15 menunjukkan seluruh kolom kunci dan kolom *measure* *total_biaya* tidak memiliki nilai kosong (NULL = 0 pada semua kolom).

Tabel 8.6: Ringkasan Hasil Pemeriksaan Nilai *Null*

No	Kolom	Keterangan	Jumlah <i>Null</i>
1	pasien_key	Foreign key ke <i>dim_pasien</i>	0
2	dokter_key	Foreign key ke <i>dim_dokter</i>	0
3	date_key	Foreign key ke <i>dim_waktu</i>	0
4	fasilitas_key	Foreign key ke <i>dim_fasilitas</i>	0
5	bayar_key	Foreign key ke <i>dim_pembayaran</i>	0
6	total_biaya	Kolom <i>measure</i> utama	0

Pemeriksaan Kevalidan — Nilai Negatif pada Biaya

Kueri berikut memverifikasi bahwa seluruh nilai *total_biaya* bersifat non-negatif sesuai aturan validasi bisnis, sekaligus menampilkan statistik deskriptif distribusi biaya.

Listing 8.16: Validasi nilai non-negatif dan statistik deskriptif *total_biaya*

```

1 SELECT
2     MIN(total_biaya) AS min_biaya,
3     MAX(total_biaya) AS max_biaya,
4     ROUND(AVG(total_biaya), 2) AS avg_biaya,
5     PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY total_biaya) AS median_biaya
6     ,
7     COUNT(*) FILTER (WHERE total_biaya < 0) AS
      jumlah_negatif
8 FROM airflow.public.fact_kunjungan;
```



Data Output Messages Notifications					
Showing rows: 1 to 1					
	min_biaya numeric	max_biaya numeric	avg_biaya numeric	median_biaya double precision	jumlah_negatif bigint
1	61891.00	19060083.00	2339833.46	630571	0

Total rows: 1Query complete 00:00:00.181

Gambar 8.16: Hasil kueri validasi nilai total_biaya pada fct_kunjungan

Hasil pada Gambar 8.16 mengonfirmasi nilai jumlah_negatif = 0. Semua nilai biaya bersifat non-negatif (≥ 0) sesuai aturan validasi bisnis yang ditetapkan.

Tabel 8.7: Statistik Deskriptif total_biaya pada fct_kunjungan

No	Metrik	Keterangan	Nilai
1	min_biaya	Nilai biaya terkecil dalam dataset	–
2	max_biaya	Nilai biaya terbesar dalam dataset	–
3	avg_biaya	Rata-rata biaya seluruh kunjungan	–
4	median_biaya	Nilai tengah distribusi biaya	–
5	jumlah_negatif	Baris dengan total_biaya < 0	0

Pemeriksaan Konsistensi — Distribusi Tipe Fasilitas

Kueri berikut memverifikasi bahwa nilai tipe_fasilitas pada dim_fasilitas hanya terdiri dari kategori yang telah ditetapkan.

Listing 8.17: Distribusi nilai tipe_fasilitas pada dim_fasilitas

```
1 SELECT
2     tipe_fasilitas ,
3     COUNT(*)                AS jumlah_record ,
4     ROUND (
5         COUNT(*) * 100.0
6         / SUM(COUNT(*)) OVER (), 2
7     )                        AS persen
8 FROM airflow.public.dim_fasilitas
9 GROUP BY tipe_fasilitas
10 ORDER BY jumlah_record DESC;
```

Data Output Messages Notifications			
Showing rows: 1 to 3			
	tipe_fasilitas	jumlah_record	persen
	text	bigint	numeric
1	Rawat Inap	4	66.67
2	Rawat Jalan	1	16.67
3	IGD	1	16.67

Total rows: 3 Query complete 00:00:00.180

Gambar 8.17: Hasil kueri distribusi tipe_fasilitas pada dim_fasilitas

Hasil pada Gambar 8.17 menunjukkan distribusi tipe_fasilitas hanya mencakup tiga kategori valid. Tidak ditemukan kategori tidak terduga (*rogue values*).

Tabel 8.8: Distribusi Tipe Fasilitas pada dim_fasilitas

No	Tipe Fasilitas	Keterangan	Status Validasi
1	Rawat Jalan	Pelayanan tanpa rawat inap	Valid
2	Rawat Inap	Pelayanan dengan perawatan inap	Valid
3	IGD	Instalasi Gawat Darurat	Valid

Pemeriksaan Volume — Jumlah Baris

Kueri berikut memverifikasi bahwa jumlah baris pada fct_kunjungan sesuai dengan volume data yang direncanakan sejak tahap perancangan.

Listing 8.18: Verifikasi jumlah baris fct_kunjungan terhadap target volume

```
1 SELECT
2     COUNT(*)      AS total_rows_actual ,
3     12000         AS total_rows_expected ,
4     COUNT(*) - 12000
5                 AS selisih ,
6     CASE
7         WHEN COUNT(*) = 12000 THEN 'PASS'
8         ELSE 'FAIL'
9     END          AS status_volume
10 FROM airflow.public.fact_kunjungan;
```



Data Output Messages Notifications				
Showing rows: 1 to 1				
	total_rows_actual bigint	total_rows_expected integer	selisih bigint	status_volume text
1	12000	12000	0	PASS

Total rows: 1 Query complete 00:00:00.221

Gambar 8.18: Hasil kueri verifikasi volume baris fct_kunjungan

Hasil pada Gambar 8.18 mengonfirmasi `total_rows_actual` = 12.000 dengan `selisih` = 0, sesuai volume data yang direncanakan.

Tabel 8.9: Hasil Verifikasi Volume Baris fct_kunjungan

No	Metrik	Keterangan	Nilai
1	total_rows_actual	Jumlah baris hasil pemuatan aktual	12.000
2	total_rows_expected	Jumlah baris target yang direncanakan	12.000
3	selisih	Deviasi antara aktual dan target	0
4	status_volume	Hasil evaluasi pemeriksaan volume	PASS

8.3.4 Ringkasan Hasil Pemeriksaan

Seluruh pemeriksaan kualitas data menunjukkan hasil yang memenuhi kriteria kelulusan yang telah ditetapkan. Rekapitulasi disajikan pada Tabel 8.10.

Tabel 8.10: Rekapitulasi Hasil Pemeriksaan Kualitas Data

No	Dimensi	Temuan	Status
1	Kelengkapan (Null Check)	0 nilai NULL pada seluruh kolom kunci dan <i>measures</i>	PASS
2	Kevalidan (Range Check)	0 nilai <code>total_biaya</code> bernilai negatif	PASS
3	Konsistensi (Category Check)	3 kategori valid: IGD, Rawat Jalan, Rawat Inap	PASS
4	Volume (Row Count Check)	12.000 baris sesuai rencana pemuatan, selisih = 0	PASS

Hasil pemeriksaan di atas mengonfirmasi bahwa Data Mart yang dibangun memiliki kualitas data yang andal dan siap digunakan sebagai dasar analitik dan pelaporan.

Part IX

Kesimpulan dan Rekomendasi



BAB 9

Kesimpulan dan Refleksi

9.1 RINGKASAN PENCAPAIAN PROYEK

Proyek ini berhasil membangun sistem Data Warehouse dan Business Intelligence (DWBI) operasional rumah sakit secara *end-to-end* dalam satu ekosistem terpadu berbasis Docker. Dimulai dari pembangkitan data sintetis menggunakan Python, proses orkestrasi otomatis melalui Apache Airflow, transformasi modular menggunakan dbt, hingga penyediaan Data Mart yang siap dikonsumsi oleh Metabase sebagai *Business Intelligence tool*. Seluruh komponen berhasil diintegrasikan dan diverifikasi dengan hasil pengujian 18/18 dbt test lulus tanpa kegagalan.

9.2 TANTANGAN YANG DIHADAPI

Selama proses pembangunan sistem DWBI ini, terdapat beberapa tantangan teknis dan desain yang dihadapi:

1. Konfigurasi Jaringan Docker (*Port Forwarding*)

Tantangan terbesar yang dihadapi adalah permasalahan konektivitas jaringan antar container Docker. Metabase yang dijalankan di luar jaringan internal Docker tidak dapat mengakses PostgreSQL melalui *port* standar 5432 secara langsung. Solusi yang diterapkan adalah mengekspos *port* PostgreSQL ke jaringan *host* melalui *port forwarding* eksternal 5555:5432 pada `docker-compose.yaml`, dan mengonfigurasi Metabase untuk menggunakan alamat `host.docker.internal:5555` sebagai jalur komunikasi lintas batas kontainer.

2. Konsistensi Surrogate Key antar Lapisan dbt

Pada tahap awal implementasi, ditemukan inkonsistensi antara *Surrogate Key* yang dihasilkan pada lapisan *dimensions* dengan kunci yang direferensikan pada lapisan *marts*. Inkonsistensi ini disebabkan oleh perbedaan input fungsi hashing MD5 yang digunakan untuk menghasilkan *pasien_key* dan *dokter_key*. Solusinya adalah dengan menyeragamkan kolom input fungsi `dbt_utils.generate_surrogate_key()` pada seluruh model dimensi dan memastikan bahwa model fakta merujuk kunci dari hasil `ref()` dimensi, bukan dari tabel sumber mentah.

3. Penanganan Data Sintetis yang Realistis

Menghasilkan data sintetis yang memiliki distribusi statistik realistis memerlukan pengaturan *constraint* yang cermat. Awalnya, generator Python menghasilkan nilai negatif pada kolom biaya dan durasi rawat inap yang melebihi 14 hari untuk pasien IGD. Solusinya adalah dengan menerapkan logika kondisional berbasis `if-else` per tipe fasilitas dan membatasi rentang nilai menggunakan `random.randint()`



dengan batas atas dan bawah yang eksplisit, serta memastikan `total_biaya` selalu dihitung secara deterministik dari penjumlahan komponen.

4. Orkestrasi Dependensi Task Airflow

Pada percobaan awal, task `dbt_run` dieksekusi sebelum proses ingesti data dari CSV ke PostgreSQL selesai sepenuhnya, sehingga model `dbt` membangun tabel dimensi dari tabel sumber yang masih kosong. Solusinya adalah mempertegas urutan dependensi linear menggunakan operator `>` di Airflow dan menambahkan *sensor task* untuk memverifikasi bahwa tabel sumber (`src_kunjungan`) telah terisi sebelum eksekusi `dbt run` dimulai.

5. Pengelolaan Tipe Data Tanggal lintas Sistem

Format tanggal dari file CSV (YYYY-MM-DD HH:MM:SS) tidak langsung kompatibel dengan tipe `DATE` pada PostgreSQL saat proses ingesti menggunakan Pandas. Hal ini menyebabkan kegagalan pada model `dim_waktu` yang memerlukan kolom bertipe `DATE` bersih. Solusinya adalah menambahkan konversi eksplisit `pd.to_datetime().dt.date` pada skrip ingesti Python sebelum data ditulis ke tabel *staging*, diikuti dengan *type casting* tambahan `CAST(tanggal_kunjungan AS DATE)` pada model `stg_kunjungan.sql`.

9.3 KEPUTUSAN DESAIN UTAMA

Beberapa keputusan desain fundamental yang diambil selama pengembangan sistem dan alasan teknis di baliknya:

Tabel 9.1: Ringkasan Keputusan Desain Utama Sistem DWBI

Keputusan Desain	Alternatif yang Diper-timbangkan	Alasan Pemilihan
<i>Star Schema</i> murni (fully denormalized)	<i>Snowflake Schema</i> (normalized dimensions)	Meminimalkan kompleksitas JOIN pada <i>query</i> BI, mempercepat respons agregasi, dan lebih mudah dipahami oleh pengguna bisnis non-teknis.
Pendekatan ELT (bukan ETL)	ETL tradisional dengan transformasi di luar database	Memanfaatkan kekuatan komputasi PostgreSQL di dalam database, memungkinkan <code>dbt</code> mengelola seluruh logika transformasi secara versi-terkontrol.
<i>Surrogate Key</i> berbasis MD5 Hash	<i>Auto-increment integer</i> (SERIAL)	Menghasilkan kunci yang deterministik dan dapat direproduksi meski pipeline dijalankan ulang, sehingga mencegah duplikasi kunci saat proses <i>reload</i> data.
Materialisasi <i>view</i> untuk lapisan <i>staging</i>	Materialisasi <i>table</i> untuk semua lapisan	Menghemat ruang penyimpanan karena lapisan <i>staging</i> hanya berfungsi sebagai antarmuka pembersih, bukan repositori permanen.
Orkestrasi Apache Airflow dengan <i>manual trigger</i>	Penjadwalan otomatis harian (@daily)	Memberikan kontrol eksplisit atas siklus pembaruan data selama fase pengembangan, menghindari eksekusi tidak terduga saat konfigurasi sistem masih diubah.
Data sintetis berbasis aturan kondisional (<i>rule-based</i>)	Data sintetis berbasis distribusi statistik (GAN/VAE)	Lebih mudah dikontrol, dapat direproduksi dengan <code>random.seed()</code> , dan tidak memerlukan data nyata sebagai data latih model generatif.

9.4 KESIAPAN DATA MART UNTUK TUGAS BI (VISUALISASI DATA)

Data Mart yang telah dibangun dinyatakan **siap** untuk digunakan pada Tugas BI berikutnya berdasarkan hasil verifikasi berikut:



Tabel 9.2: Checklist Kesiapan Data Mart untuk Tugas BI

No	Kriteria Kesiapan	Status	Bukti Verifikasi
1	Integritas referensial antar tabel terjaga	✓ Lulus	18/18 dbt test PASS
2	Tidak ada nilai <i>null</i> pada kolom kunci dan <i>measures</i>	✓ Lulus	<i>Data quality check</i>
3	Volume data memadai (12.000 transaksi)	✓ Lulus	COUNT(kunjungan_key) = 12.000
4	Koneksi Metabase ke PostgreSQL berhasil	✓ Lulus	<i>Screenshot</i> konfigurasi 5.5.3
5	<i>Query</i> analitik 5 BQ menghasilkan nilai valid	✓ Lulus	Screenshot hasil 5 <i>query</i>
6	Skema <i>Star Schema</i> terdeteksi Metabase	✓ Lulus	Tabel <i>dim_*</i> dan <i>fct_*</i> terbaca
7	Metrik KPI terdefinisi pada <i>Semantic Layer</i>	✓ Lulus	3 formula KPI di dbt

Berdasarkan hasil verifikasi pada tabel di atas, tabel *fct_kunjungan* dan kelima tabel dimensi (*dim_pasien*, *dim_dokter*, *dim_waktu*, *dim_fasilitas*, *dim_pembayaran*) telah tersedia di database PostgreSQL dalam kondisi bersih, terstruktur, dan dapat dibaca secara langsung oleh Metabase.

Pada Tugas BI berikutnya, seluruh 12 *business questions* yang telah ditetapkan pada BAB II dapat dijawab melalui visualisasi interaktif menggunakan Metabase, mencakup:

- **Dasbor Finansial:** Grafik batang total pendapatan per tipe fasilitas, *pie chart* distribusi metode pembayaran, dan kartu KPI total pendapatan operasional.
- **Dasbor Kinerja Klinis:** Grafik peringkat dokter berdasarkan volume transaksi, distribusi status keluar pasien Rawat Inap, dan rata-rata *Length of Stay* per fasilitas.
- **Dasbor Tren Operasional:** Grafik garis tren bulanan jumlah kunjungan, analisis komponen biaya per kategori, dan *heatmap* distribusi kunjungan berdasarkan waktu.
- **Dasbor Demografi:** Distribusi kunjungan berdasarkan kota asal, analisis *cross-tabulation* jenis kelamin dan golongan darah, serta peta geografi beban kunjungan per wilayah.

Kesimpulan Akhir

Sistem DWBI operasional rumah sakit ini berhasil dibangun secara *end-to-end* dengan memenuhi seluruh kriteria teknis: integritas data terjamin, *pipeline* ETL terotomatisasi, dokumentasi lengkap, dan Data Mart terhubung ke BI tool. Proyek ini membuktikan bahwa arsitektur modern berbasis ELT dengan dbt dan Airflow mampu menghasilkan infrastruktur analitik yang andal, dapat dipelihara, dan siap mendukung pengambilan keputusan klinis serta manajerial di fasilitas kesehatan.



9.4 DAFTAR PUSTAKA

- [1] Apache Software Foundation. Apache airflow documentation. URL <https://airflow.apache.org/docs/>.
- [2] dbt Labs, Inc. What is dbt? | dbt documentation. URL <https://docs.getdbt.com/docs/introduction>.
- [3] Ralph Kimball and Margy Ross. *The Data Warehouse Toolkit: The Complete Guide to Dimensional Modeling*. John Wiley & Sons, Inc., 2nd edition, 2002. ISBN 0-471-20024-7. URL http://www.r-5.org/files/books/computers/databases/warehouses/Ralph_Kimball_Margy_Ross-The_Data_Warehouse_Toolkit-EN.pdf.
- [4] PostgreSQL Global Development Group. *PostgreSQL 18.0 Documentation*. PostgreSQL Global Development Group, 2025. URL <https://www.postgresql.org/files/documentation/pdf/18/postgresql-18-A4.pdf>.